PERQemu User's Guide

For *PERQemu* version 0.9.x

S. Boondoggle

12-May-2026

PRELIMINARY

Boondoggle Heavy Industries, Ltd.
1060 West Addison
Chicago, IL 60613
(800) 555-1212

Many of the dubious modifications or enhancements to the software discursively described here were contributed by S. Boondoggle, Boondoggle Heavy Industries, Ltd.

Boondoggle Heavy Industries will make every effort to keep customers apprised of all documentation changes as quickly as possible. A Reader's Comments card will be distributed with this document to request users' critical evaluation to assist us in preparing future documentation.

Table of Contents

1. Introduction **14**
Where to Obtain *PERQemu* 15
Software Requirements 16
System Requirements 17
Host System Recommendations 17
Roadmap and Release Notes 19
What's New in *PERQemu* v0.9.0 19
2. The Emulator **20**
Getting Started 20
Running *PERQemu* 21
The Command Line Interface 22
Getting Help 22
Tab Completion 23
Command Arguments 24
Editing and History 26
The Configurator 27
Choosing a Predefined Configuration 27
Creating Custom Configurations 28
Chassis Types 29
Processor Type 30
Configuring Memory 30
I/O Board Types 31
Display and Tablet Options 32
Assigning a Custom Keyboard Map 32
I/O Options 33
Status of I/O Option Development 34
Configuring Sub-options 35
Serial Ports 35
Network Address 36
Disks, Floppies, and Tapes 36
Checking for Errors 36
Naming and Saving Configurations 38

Working with Storage Devices 39
Supported Devices 39
Unit Numbers 40
The Disks Directory 41
Supported Media Formats 41
Loading, Saving, and Unloading Media 42
Showing Storage Device Status 44
Changing Formats, Making Backups 44
Creating and Formatting New Media 45
Using Hardware Disk Emulators 46
Importing Disk Images 47
Exporting Disk Images 47
Safe Mode 48
Import/Export Notes 48
Working with Communication Devices 49
Serial Devices 49
Live Updates 51
Permissions and Performance 51
The RSX: Pseudo-device 51
"Speech" Output 51
Ethernet 52
Encapsulation Options and Administrative Privileges 54
Network Address Translation and the *PERQemu* Virtual Hub 55
Current Problems and Limitations 55
Working with Output Devices 57
Canon Laser Printers 57
Canon Runtime Options 58
Multi-page Documents 59
Managing the Virtual Machine 60
Starting and Stopping the PERQ 60
The Diagnostic Display (DDS) 62
Boot Selection 63
Bootstrapping in Depth 64
Automating Actions with Scripts 66

Limitations and Implementation Notes 66
Setting User Preferences 67
Autosave Options 67
Device Assignment Options 67
Performance Options 67
Miscellaneous Settings 69
Let's Do The Time Warp Again 69
Keyboard Options 70
Debugging Features 70
3. PERQ Operations 71
Virtual Machine Interface 71
Special Keys 71
Mouse Input 72
Button Mapping 73
Display Window 73
Floppy Formats 74
Hard Disk Formats 75
Tape Formats 75
Supported Operating Systems 77
POS 78
Availability 78
Configuration 79
Login and User Interface 79
File Transfer via RSX: 81
File Transfer using RS-232 82
File Transfer using Floppies 82
RT-11 Floppy Limitations 82
File Transfer using Tapes 83
Laser Printing 83
Logout and Shutdown 83
MPOS 84
Availability 84
Configuration 84
Login and User Interface 84

Logout and Shutdown 84
Accent 85
Availability 85
Configuration 85
Login and User Interface 86
Using the Accent Shell 88
Laser Printing 88
Logout and Shutdown 88
Limitations 89
Interim Ethernet Support 89
Live Ethernet Support 89
PNX 91
Availability 91
Configuration 91
Login and User Interface 92
Logout and Shutdown 94
FLEX 95
Availability 95
Configuration 95
HCR Unix 96
4. Command Reference **97**
Top-level Commands 97
about [libraries] 97
bootchar [*char*] 97
configure [...] 97
debug [...] 97
go 97
help [*keyword*] 98
history 98
keymap [...] 98
load (floppy | harddisk | tape) *filename* [*unit*] 98
load paper [size] 98
power (on | off) 98
quit [without save] 98

reset 99
save (floppy | harddisk | tape) [as *filename* [*format*]] 99
save history 99
save screenshot [*filename*] 99
settings [...] 99
start 99
status 99
stop 99
storage [...] 100
unload floppy 100
unload harddisk [*unit*] 100
unload tape 100
Settings Commands 101
assign audio device *hostDevice* 101
assign audio option *opt val* 101
assign ethernet device *hostDevice* 101
assign (rs232a | rs232b) device *hostDevice* [*baud*] [*bits*] [*parity*] [*stopBits*] 101
assign (rs232a | rs232b) option *flag* 101
autosave (floppy | harddisk | tape) (yes | no | maybe) 101
canon output format (raw | png | tiff | jpeg) 101
canon paper size (nocassette | a4 | b5 | uslegal | usletter) 102
canon resolution (240 | 300) 102
default 102
display cursor (crosshairs | defaultarrow | none) 102
keymap (*mapName* | none) 102
load 102
output directory *dir* 102
pause on reset (true | false) 102
pause when minimized (true | false) 102
rate limit *option* 103
rtc offset *year* 103
save 103
show 103
show audio devices 103

show ethernet devices 103
unassign audio device 103
unassign ethernet device 103
unassign (rs232a | rs232b) device 103
Configuration Commands 104
assign *filename* [*unit*] 104
chassis *type* 104
check 104
cpu *type* 104
default 104
description *string* 104
disable (rs232a | rs232b | speech) 104
display (portrait | landscape) 104
enable (rs232a | rs232b) 105
enable speech 105
ethernet address *address* 105
io board *type* 105
keymap *mapName* 105
keymap (default | none) 105
list 105
load *config* 105
memory *capacity* 106
name *string* 106
option board *type* 106
option add *opt* 106
option remove *opt* 106
rtc offset *year* 106
save 106
show [*config*] 106
tablet (bitpad | kriz | both | none) 106
unassign *filename* 107
unassign unit *unit* 107
Storage Commands 108
create *driveType filename* 108

define *driveClass driveType* 108
export unit *unit imgDataFile* [*metaDataFile*] 108
export disk *diskFile imgDataFile* [*metaDataFile*] 108
import *diskFile imgDataFile* [*metaDataFile*] 109
list 109
safe 109
show 109
show aliases 109
show devices 109
show specifications *driveType* 109
status [*unit*] 109
unsafe 109
Keyboard Mapping Commands 110
apply 110
define *mapName* 110
edit [*mapName*] 110
list 110
load *mapName* 110
show [(running | summary | *mapName*)] 110
save 110
Keymap Editor 111
apply 111
default 111
description *string* 111
map *hostKey perqKey* 111
unmap *hostKey* 111
save 111
show [(summary | host key *hostKey* | mapped key *perqKey*)] 111
Debugging Commands 112
clear breakpoint *type watchpoint* 112
clear interrupt *irq* 112
clear logging *category* 112
disable breakpoints 112
disable console logging 112

disable file logging 113
disassemble microcode [*address*] [*length*] 113
dump item 113
edit breakpoint *type watchpoint* 113
enable breakpoints 113
enable console logging 113
enable file logging 113
find memory *address value* 113
inst 114
jump *address* 114
load microcode *binfile* 114
load qcodes *codeset* 114
raise interrupt *irq* 114
reset breakpoints *type* 114
set breakpoint *type watchpoint* [*pause*] 114
set console loglevel *severity* 115
set execution mode (asynchronous | synchronous) 115
set file loglevel *severity* 115
set logging *category* 115
set memory *address value* 115
set xy register *reg value* 115
show 115
show breakpoints [*type*] 115
show cpu registers 115
show cstack 116
show estack 116
show execution mode 116
show memory *address* [*length*] 116
show memory state 116
show opfile 116
show variables 116
show xy register [*reg*] 116
step 116
z80 [...] 116

Z80 Debugging Commands 117
clear breakpoint *type watch* 117
dump *item* 117
edit breakpoint *type watch* 117
inst 117
reset breakpoints *type* 117
set breakpoint *type watch* [*pause*] 117
show breakpoints [*type*] 117
show memory *address* [*length*] 118
show registers 118
top 118
Appendix A: A Brief History of PERQ **119**
Appendix B: Bibliography **120**
POS Documentation 120
Accent Documentation 120
PNX Documentation 121
Additional Documentation 121
Applications 121
Appendix C: Additional Resources **122**
PERQdisk: Filesystem Munger Tool *122*
Stut: The Streamer Utility *122*
Appendix D: Keyboard Maps **123**
Console vs. Display Input 123
PERQemu and SDL2 Keycodes *124*
Reserved Keys 127
Using the Keymap Editor 129
Editing Keymaps for Fun and Profit 129
Using an External Editor 131
Other Features, Quirks and Notes 131
Appendix E: Release History **133**
What's New in *PERQemu* v0.8.5 133
What's New in *PERQemu* v0.7.8 133
What's New in *PERQemu* v0.7.5 133
What's New in *PERQemu* v0.6.5 134

What's New in *PERQemu* v0.5.8 134
What's New in *PERQemu* v0.5.5 135
What's New in *PERQemu* v0.5.0 135

List of Tables

Table 1: Chassis types. 29
Table 2: CPU board types. 30
Table 3: I/O board types. 31
Table 4: I/O Option board types. 34
Table 5: I/O sub-options. 35
Table 6: Special key mapping. 71
Table 7: POS hardware requirements. 79
Table 8: Printing characters. 125
Table 9: Editing and special keys. 126
Table 10: Reserved keys. 127
Table 11: Unmapped keys. 128

List of Figures

Figure 1: Renaming a Windows network adapter. 54
Figure 2: Console window with diagnostic display. 62
Figure 3: Host to PERQ mouse button mapping. 73
Figure 4: Accent startup screen. 86
Figure 5: PNX 1.3 graphical environment. 93
Figure 6: Screen capture of the PERQ-1 keyboard layout. 132
Figure 7: Screen capture of the PERQ-2 VT-100 style layout. 132

1. Introduction

PERQemu is a software emulator of the venerable PERQ workstations, graphics computers designed and sold in the early 1980s by Three Rivers Computer Corporation (3RCC) of Pittsburgh, PA. It is intended to provide a *nearly* cycle-accurate hardware emulation such that all software that runs on the PERQ hardware can be run unmodified on the emulator. For a brief background on the PERQ's origins, please see Appendix A.

The following hardware is emulated by *PERQemu*:

- A custom bit-slice, microcoded CPU with 20-bit physical addressing
- 4K or 16K of writable control store (48-bit microinstruction words)
- 256KB to 2MB of RAM (in 16-bit words)
- A high resolution bitmapped display at 768 x 1024 pixels (monochrome)
- Custom RasterOp hardware to accelerate bitmap operations

The original PERQ features a standard set of peripherals:

- A 12- or 24MB Shugart SA4000-series hard disk (14" platters)
- A Shugart SA851 8" double-sided floppy drive
- A GPIB interface, typically used to connect a Summagraphics BitPadOne digitizer tablet
- One programmable RS232 port, up to 9600 baud
- A CVSD chip for audio output

As of version 0.9.x, all of the standard peripherals *including* audio output are emulated!

Optional I/O boards can also be fitted, which provide:

- 3Mbit or 10Mbit Ethernet interfaces
- A laser printer interface (Canon LBP-10, or later Canon CX)
- A quarter inch cartridge tape connection (Archive Sidewinder)

At this time, the streaming tape and Canon laser printer I/O options have been fully implemented. 10Mbit Ethernet is available for experimentation and still is in active development; all three interfaces are still undergoing testing and refinement with additional software support coming soon.

The PERQ-2 series, introduced in 1983, extends the original design in a few significant ways and adds a number of additional I/O options:

- A larger landscape display (1280 x 1024, monochrome)
- Faster, higher capacity disk drives (8" Micropolis, or up to two 5.25" MFM/ST-506)
- A faster I/O board with integrated Ethernet, second RS-232 port, real-time clock
- An extended 24-bit CPU with larger memory addressing range

Support for the PERQ-2 features, including the 24-bit CPU, was incorporated as of v0.7.8. While there are a few more esoteric hardware options available, they fall outside the scope of what *PERQemu* can realistically emulate. Full support for the PERQ-1 and PERQ-2 enables the preservation of *five* complete operating systems and the vast majority of surviving software.

Where to Obtain *PERQemu*

For the most up-to-date information, source code and binary downloads, please consult the following Github repositories:

https://github.com/jdersch/PERQemu
: The master branch and definitive source for *PERQemu*;

https://github.com/skeezicsb/PERQemu
: Experimental branch and source of never ending amusement for the nerdiest or most hardcore PERQ fanatic.

The basic *PERQemu* packages include a number of hard disk images and software to get started. Two additional sources are:

https://github.com/skeezicsb/PERQmedia
: A library of curated media files (floppy, hard disk and tape images) that will grow over time; additional materials of historical interest (marketing images, clippings, academic papers and more) may be included as well;

https://bitsavers.org
: The *excellent* Bitsavers archive is a treasure trove of computing history and also features quite a bit of PERQ software and documentation.

Software Requirements

PERQemu is written in C#, targeting the Microsoft .NET Framework v4.8. For Linux and MacOS X this is roughly equivalent to the Mono 6.12.0.*x* runtime/SDK. In addition, the *SDL2-CS* package provides a wrapper around the SDL2 media library, while the *Z80dotNet* package provides Z80 CPU emulation. *SharpPcap* is used for host Ethernet connectivity. These are bundled with the *PERQemu* binary distribution or are installed via NuGet when building from source.

The emulator should run on any Windows, Mac or Linux environment that can support the appropriate .NET or Mono runtime environment. You may also need to install a current SDL2 package appropriate for your OS (available for download from `libsdl.org`).

- *Note:* For MacOS X installations the pre-packaged SDL2 download that installs in `Library/Frameworks` does *not* work with the SDL2-CS package; it has to find the individual `SDL2` and `SDL2_image` libraries. Building from source or installing with a package manager should drop these into `/usr/local/lib` (or another common directory in your linker's library path) so Mono can find them.

Installation assistance for SDL2 can be found on the web at https://wiki.libsdl.org/SDL2/Installation.

Windows users may need to install Npcap (preferred over the older WinPcap) to access Ethernet. Download and documentation information is at https://npcap.com/.

The "`skeezicsb/experiments`" branch is developed and tested with Xamarin Studio Community 6.3 on MacOS X 10.13 (High Sierra) and verified on Visual Studio for Mac 2019. It has also been confirmed to run on macOS 12 (Monterey) with Visual Studio for Mac 2022.

This branch has also been tested for compatibility with MS Visual Studio 2022 on Windows 10, with Linux testing on Ubuntu 20.0.4 LTS and Mint (Cinnamon) 22.1[1]. Cross-platform inconsistencies are fairly minor and mostly related to things like window handling; in general *PERQemu* runs the same on all three OS platforms.

[1] *Linux testing is currently performed using the Windows-generated distribution; there is not currently any documented build method using a Linux-hosted .NET / Mono toolchain.*

System Requirements

PERQemu is a nearly cycle-accurate, register-level emulation of a complex microcoded processor *and* a Z80 subsystem – essentially two emulations running side-by-side – driving a full bitmapped graphics display refreshed 60 frames per second. It requires a fair bit of grunt.

To gauge how faithfully your computer is able to emulate the PERQ, the title of the display window updates every few seconds to report the frame rate ("fps") and the average cycle time of the PERQ and Z80 processors. At full speed the PERQ will display 60fps, with the CPU executing at 170ns (~5.88Mhz) and the Z80 at 407ns (2.45Mhz for the PERQ-1) or 250ns (4Mhz for the PERQ-2). The emulator is still in active development and performance tuning is an ongoing concern.

Host System Recommendations

Processor: A "reasonably current" multi-core x64 CPU is the baseline requirement. *PERQemu* is sensitive to host memory speed when running asynchronously, but may be CPU-bound when running synchronously. For reference, without rate limiting enabled a "6th gen" Intel i5-6600 @ 3.3Ghz with 2133Mhz memory achieves 80-90fps at ~ 70% system utilization on Windows 10, while a 2008 Mac Pro (8-core 2.8Ghz Xeon, 800Mhz DDR2 memory) tops out around 35fps. YMMV.

No testing has yet been done on Apple's ARM-based silicon or other non-x64 processors. *Donations gladly accepted!*

Memory, disk space: negligible, by modern standards. The running emulator uses about 100MB of RAM with a typical PERQ-1/Shugart configuration loaded, to 300MB for a large PERQ-2/MFM machine (varies depending on the size of the loaded hard drive). Approximately 200MB of disk space is required for the source tree, or 50MB for the binary distribution with the standard library of bundled disk images.

Display: A typical 1920 x 1080 display is generally adequate to host the PERQ's displays without clipping or scrolling. At full speed the PERQ refreshes its display at 60 frames/second, which should be supported by all modern displays.

PERQemu v0.8.1 adds support for automatic resizing of the window for screens (such as older laptops) that are too short to display the full height, 1024-line display. For hosts with multiple monitors, the

window will automatically resize when dragged between them. Please refer to Chapter 3, *PERQ Operations,* for more information.

- *Note:* There has been no testing on high-DPI monitors.

Keyboard: Any standard keyboard supported by SDL2 should work. The PERQ has some special keys that aren't common on modern keyboards, so these are assigned to function keys based on the "standard" USB HID guidelines that SDL uses, where essentially every keyboard is a PC-101 type. However, *PERQemu* now supports remapping the keyboard based on user preference, or to accommodate a variety of host platforms. Information about keyboard mapping is given throughout this document, but a good starting point is Appendix D, *Keyboard Maps*.

- *Note: PERQemu* has limited support for non-US/English keyboard input in the CLI, but it requires recompiling the source. You may need to set your system's input locale to US (or UK) English to generate input that the console can understand.

Though keys may be dynamically remapped for interacting with the virtual machine, none of the PERQ operating systems offered much in the way of internationalization support.

Mouse: The PERQ expects a 3-button mouse or a 4-button "puck" as its pointing device. For a typical modern mouse, the three buttons are mapped directly, while a key modifier provides the extra button if the 4-button Bitpad is configured. Mice with scroll wheels didn't exist in the early 1980s so *PERQemu* doesn't transmit wheel inputs to the virtual machine; instead, the mouse wheel will scroll the display on short screens, as described above.

Audio: The PERQ now speaks! Currently *PERQemu* relies on SDL2 to select and use the default audio device on the host, with the presumption that it will upsample the 16kHz mono output to meet the (typically) 44.1 or 48kHz stereo output rate that modern computers provide. Some tuning of the audio preferences may/will be added as the implementation matures.

Roadmap and Release Notes

This final pre-release marks the completion of basic emulator development – practically all of the standard features of all PERQ models as shipped are available! Ongoing development effort toward the v1.0 milestone is focused on further refinement of the Ethernet implementation so that virtual PERQs can reliably use the network, emulation accuracy and efficiency, and additional user interface/usability improvements. The `ChangeLog` and `Readme` files in the *PERQemu* Github repository are the best source for up-to-the-minute updates and status.

What's New in *PERQemu* v0.9.0

Changes since the v0.8.5 interim release:

- PERQ "speech" output is now enabled: emulation of the MC3417 CVSD chip provides 16kHz, monaural audio output to the default host audio device;
- Major re-work of the RS-232 / Z80 SIO chip low-level implementation makes using the host serial ports *much* more robust, on all platforms, with live configuration updates;
- Assignment of `RSX:` to RS-232 port B on PERQ-2 is discouraged/disallowed (POS doesn't use it, so it's just ignored if assigned);
- Settings and Configurator command line syntax changes to accommodate the RS-232 and Speech refactoring;
- Changes to the Scheduler and Z80 reset code to correct obscure bugs while diagnosing yet another gratuitous incompatibility introduced by the "PNX Installation Test" floppy;
- The usual assortment of bug fixes and documentation updates.

To support the serial device handling (RS-232 and Speech), the syntax of a number of Settings and Configuration commands was changed. This interim release will still accept the older commands but will rewrite them using the new form when saved. This should happen automatically for the settings file, but changed configurations will be rewritten only when explicitly saved. Please see chapter four, *Command Reference*, for the new syntax.

Release notes for prior releases are preserved in Appendix E.

2. The Emulator

Getting Started

Binary distributions of *PERQemu* can be downloaded from the Releases area on Github (see above). The single ZIP archive may be unpacked anywhere you like; there is no need for an installer and the program does not need any special OS privileges to run – unless Ethernet is enabled.

If you prefer to build from source, a clone or download from the Github repository contains everything you need to load the "`PERQemu.sln`" file into Visual Studio. (Please consult the file "`Readme-source.txt`" for more information about the organization of the source tree.) The source builds end up in `bin/Debug` or `bin/Release`, which should be your working directory when running the executable.

When unpacked the *PERQemu* binary release distribution contains several subdirectories:

`Conf/` Contains a collection of predefined system configurations as well as device data required for operation. By default, all custom PERQ configurations and keyboard maps are also saved to and loaded from this directory.

`Disks/` Contains disk images that the emulator can access. Several "stock" hard drive images are included to get you started; additional media images may be added in future releases. All disk images are saved to and loaded from the `Disks` directory by default.

`Output/` When logging debug output to disk is enabled, those files go here by default. Saved screenshots and printed Canon laser printer pages land here too. Output directory is a settable preference.

`PROM/` Contains dumps of PERQ ROMs necessary for operation.

`Resources/` Contains graphics or other files necessary for operation.

A number of other interesting files and directories are included in the full source distribution, including several "readme" files and additional documentation. (This is all browsable on Github.)

Running *PERQemu*

To launch the program, first set your working directory to where you unpacked the ZIP file. Then just invoke `PERQemu.exe`:

> **Windows**: double-click the `PERQemu.exe` icon or launch it from the task bar. *PERQemu* is a "console application," so a command window will appear and you'll be at the command prompt. Or you may also run the executable from the command line.
>
> **Unix/Linux/Mac**: type "`mono PERQemu.exe`" from the shell prompt in a terminal window. The *PERQemu* command interpreter will announce itself, same as in the Windows version.

At startup you can supply a script to run as an argument. The command syntax is:

```
Usage:  PERQemu [-h] [-v] [<script>]

    -h    print this help message
    -v    print version information
script    read startup commands from file
```

The script you provide must be a plain text file, and it may contain any valid command line you would type yourself. (See *Automating Actions with Scripts* later in this chapter for more information.) In the examples which follow, things you type will be indicated in **bold**:

```
[octothorpe:PERQemu/bin/Release] skeezics% mono PERQemu.exe
Initializing, please wait...
Settings reset to defaults.
Default output directory is now 'Output'.
PERQemu v0.9.0 ('Split for the coast.')
Copyright (c) 2006-2026, J. Dersch (derschjo@gmail.com)
With contributions from S. Boondoggle (skeezicsb@gmail.com)
>
```

- *Note:* The first time you run *PERQemu*, a settings file is created in your home directory. On Windows this file will be called `PERQemu.cfg`; on the Unix platforms this will be `~/.PERQemu_cfg`. This is the only file that *PERQemu* will write outside of the folder where you install it.

The Command Line Interface

PERQemu's top-level or main command prompt is a single '>'. The command-line interface (CLI) now organizes the extensive command set into a hierarchical set of "subsystems." The prompt will change according to the current level in the hierarchy, such as:

```
      configure>
```
or
```
      settings>
```

Essentially a subsystem is just a convenient way to save typing when executing a number of related commands. To return to the previous level, the "`done`" command exits the current subsystem.

Getting Help

The CLI provides some on-line help. At the top-level prompt, the "`help`" command currently provides very little assistance, but it's still a work-in-progress and a more comprehensive help will be forthcoming in a future release.

Typing "`commands`" at any prompt will show a summary of all the commands available at that point in the hierarchy. For example:

```
> settings
settings> commands
   assign ethernet device - Map a network adapter to the PERQ Ethernet device
      assign rs232 device - Map a host device to a PERQ serial port
          autosave floppy - Save modified floppy disks on eject
        autosave harddisk - Save harddisks on shutdown
            autosave tape - Save modified streamer tapes on unload
      canon output format - Set file format for Canon laser printer output
         canon paper size - Set default paper size for the Canon laser printer
                        ...
           pause on reset - Pause emulation after a reset
    pause when minimized - Pause emulation when display window is minimized
              rate limit - Set rate limit option flags
       rate limit default - Set default rate limits
              rtc offset - Set the base year for the EIO RTC chip
                    save - Save current settings
                    show - Show all program settings
    show ethernet devices - List available host Ethernet interfaces
 unassign ethernet device - Unmap a network adapter (disable PERQ Ethernet)
    unassign rs232 device - Unmap a device from a PERQ serial port
```

You can also type:

```
> settings commands
```

at the top-level prompt to see the same list. For some subsystems, *PERQemu* keeps track of unsaved changes, updating the prompt with a '*' to show that something has been modified:

```
> settings
settings> autosave floppy yes
Autosave of floppy disks is now Yes.
settings*> save
Settings saved.
settings>
```

Tab Completion

The general form of a command is a series of one or more plain words followed by zero or more arguments. Each command line is executed when `RETURN` is pressed. At any point, pressing the `TAB` key will show the available options or expected argument based on what you've typed so far. For example, at the main prompt pressing `TAB` shows:

```
> <tab>
Possible completions are:
   about          bootchar       commands       configure...   debug…
   exit           go             help...        history        inst
   keymap...      load...        power...       quit           reset
   save...        settings...    start          status         step
   stop           storage...     unload...
```

Commands that share a common prefix or that lead to subsystems are followed by ellipses (...) to show that additional input will be expected.

The tab completion feature will always try to expand what you've typed when it can:

```
> se<tab>
Possible completions are:
   <CR>           assign...      autosave...    canon...       commands
   default        display        done           keymap         load
   output         pause...       rate           rtc            save
   show           unassign...
> settings
```

In the example above, "`se`" is an unambiguous match for the "`settings`" command, so *PERQemu* shows you what options are available next, and then expands the partial command for you. Since `settings` is also a subsystem, the first option shown is `<CR>` which means you can press `RETURN` at this point and the command is complete. If there are multiple possibilities, the CLI expands the input as far as it can then waits for you to type another character to proceed:

```
> st<tab>
Possible completions are:
      status      start       stop        step        storage...
> sta<tab>
Possible completions are:
      status      start
> sta
```

Note that the `SPACE` character sometimes acts like a `TAB` too. When it makes sense. Mostly.

Some commands use "noise words"[2] to help make the syntax more natural or disambiguate their meaning; *PERQemu* will expand these too:

```
> settings pause <tab>
Possible completions are:
      on          when
> settings pause o<tab>
Possible completions are:
      false       true
> settings pause on reset            ← PERQemu expands both words
> settings pause w<tab>              ← erase "on reset" and type
Possible completions are:
      false       true
> settings pause when minimized      ← PERQemu expands both words
```

Command Arguments

Many commands require arguments, while some accept optional arguments. Tab completion shows what is expected next, prompting you with the type and name of the argument to be entered. Here a string argument named "`file`" is required:

```
> configure assign<tab>
Possible completions are:
      [string] (file)
```

[2] *A paean to the glory days of TOPS-20, for the old timers.*

```
> configure assign g7.prqm<tab>
Possible completions are:
      <CR>        [0..255] (unit)
> configure assign g7.prqm 1<tab>
Possible completions are:
      <CR>
```

Entering `g7.prqm` and pressing `TAB` shows that `<CR>` may be entered to end the command, in which case default values will be used to fill in the remaining arguments, *or* a numeric value for "`unit`" may be entered. Typing the value and pressing `TAB` again now indicates that no more input is required.

- *Note*: Most command processing is case insensitive; you don't have to capitalize command words, although sometimes case matters for arguments. For instance, filenames on most Unix hosts *are* case sensitive; see *Working with Storage Devices* later in this chapter for more info.

The most common argument types are:

`byte`	- a positive 8-bit integer value (0..255)
`ushort`	- a positive 16-bit integer (0..65535)
`uint`	- a positive 32-bit integer
`char`	- a single character (generally alphanumeric)
`string`	- a single word (quoted if the value contains spaces or multiple words!)

- *Note:* String arguments *must* be surrounded by double quotes ("") if they contain spaces. Pressing `TAB` while editing a string generally inserts a space instead, until the closing quote is typed (then completion resumes). Pressing `RETURN` will append a closing quote automatically if needed.

- *Note:* Currently filenames are entered as plain strings, so you have to type them in full.[3]

Each command will do additional checking of its inputs and the numeric range prompted for may be larger than the range of acceptable values in the given circumstance. In other words, while the `unit` number in the example above may be stored as an 8-bit byte, the limit on the number of storage units you can actually assign is much smaller than 255!

[3] *Tab completion for filenames is planned for a future release!*

When entering numeric types, you can also supply the value in any of the common bases using a character prefix or common format:

Base	Prefix	Example
Binary	`b`	`b10001100`
Octal	`o` or `%`	`o377` or `%177600`
Decimal	`d` (or none)	`d12345` or `54321`
Hexadecimal	`x`, `0x` or `$`	`x55aa`, `0xff`, `$80000`

Some types of arguments do provide tab completion. Boolean values are entered as text and expanded to "`true`" or "`false`", while enumerated types are completed like normal command words.

Editing and History

The CLI allows basic editing of the command line as you type. The following keys do what you generally expect them to do in a DOS-style editor; Unix/Emacs-style control character equivalents work as well, assuming your terminal or console application doesn't intercept them or bind them to another function:

`^U` or `ESCAPE`	- erase the entire line
`^H` or `BACKSPACE`	- delete character to the left
`^D` or `DELETE`	- delete character to the right
`^A` or `HOME`	- move cursor to the beginning of the line
`^E` or `END`	- move cursor to the end of the line
`^B` or `LEFT` (←)	- move cursor left one character
`^F` or `RIGHT` (→)	- move cursor right one character

When editing, you may press `RETURN` anywhere on the line to invoke the command. At the end of a line, `RETURN` will also complete any unambiguous final arguments on the line as well:

```
> quit w<return>
> quit without saving
```

PERQemu also keeps a limited command history. The `UP` and `DOWN` arrow keys (or the equivalents `^P` and `^N`) allow you to move forward and backward through the history buffer of previously typed commands; you may then use the editing keys to change the line, or press `RETURN` to execute the same command again.

The Configurator

Beginning with version 0.5.0, *PERQemu* allows dynamic configuration of the PERQ to be emulated. While there aren't a *huge* number of options, previous versions of the program required recompilation to change the size of memory or the CPU type – and very little else was configurable. Now, using the Configuration subsystem, you can specify CPU type, option boards, memory and display size, peripherals to attach and more. The configuration describes a complete "virtual machine" that the emulator can set up and run.

Choosing a Predefined Configuration

Configurations for all of the common machine types sold by Three Rivers are provided with the distribution. The first time you run *PERQemu*, the `default` configuration is selected. This is the only one built-in to the emulator; all the rest are loaded from the `Conf` directory. The default is a PERQ-1A with the POS F.1 image already assigned, so when first installed this version of *PERQemu* runs the same software as prior versions.

To choose a different configuration to run, you might first want to see which ones are available. If you define your own configurations, they'll be stored in the same directory and loaded at startup too. Type:

```
> configure list
Standard configurations:
       default - Default configuration
         PERQ1 - Original PERQ w/4K CPU, 256KB
        PERQ1A - Large PERQ-1A w/16K CPU, CIO, 2MB
         PERQ2 - PERQ-2 w/16K CPU, 1MB
      PERQ2-T1 - PERQ-2/T1 w/16K CPU, 1MB
      PERQ2-T2 - PERQ-2/T2 w/16K CPU, 2MB
      PERQ2-T4 - PERQ-2/T4 w/24-bit 16K CPU, 4MB
Current configuration:
       default - Default configuration
```

To load one, simply type:

```
> configure                          ← enter the subsystem
configure> load perq2-t2             ← load one
Configuration 'PERQ2-T2' selected.
configure> show                      ← take a look at the details

Configuration: PERQ2-T2
Description:   PERQ-2/T2 w/16K CPU, 2MB
Filename:      Conf/perq2-t2.cfg
--------------
```

```
Machine type:  PERQ2Tx
CPU type:      PERQ1A
Memory size:   2MB
Display type:  Landscape
Tablet type:   Kriz
IO board:      EIO
    Speech:    Enabled

Storage configuration:
----------------------
Unit: 0  Type: Floppy    File: <unassigned>
Unit: 1  Type: Disk5Inch File: <unassigned>
Unit: 2  Type: Disk5Inch File: <unassigned>
```

The "`configure show`" command with no argument displays the current configuration. If a name from the list of predefined configurations or a file name is given as an argument, the Configurator will show its details instead without changing the current selection.

Note that most reconfiguration must take place while the virtual machine is powered off. *PERQemu* will prevent you from changing the configuration while the machine is running if doing so would confuse or crash the PERQ, which like most machines in the age before USB didn't appreciate having peripherals suddenly appear or disappear at runtime. Operations on removable media such as loading and unloading tapes or floppy disks are allowed, of course.

Creating Custom Configurations

This section is written as a tutorial to explain how and why certain configuration choices are made. For an itemized breakdown of available configuration commands, their syntax and options, see the *Command Reference* chapter later in this document.

The Configurator allows you to modify, name and save your own PERQ configurations. All of the `configure` subsystem's commands modify the current configuration, so if you want to start from scratch you can use the "`default`" command to begin from a known baseline:

```
> configure default
Setting the machine to defaults.
```

Or you can load an existing configuration that's closer to what you want as your starting point, then just modify the settings you want to change.

Chassis Types

The basic definition of a PERQ starts with the chassis type, to which boards are added, peripherals attached, and other options specified. The "`configure chassis`" command selects the type.

Chassis Type	Description
`PERQ1`	The original PERQ cabinet, housing a single 14" Shugart hard drive, an 8" floppy disk, and an IOB or CIO board. Uses the original PERQ-1 style parallel keyboard and GPIB BitPad.
`PERQ2`	First system to ship with the EIO board. Houses a single 8" Micropolis hard drive and the standard 8" floppy drive. Uses the PERQ-2 style serial keyboard and the Kriz tablet.
`PERQ2Tx`	This ICL-designed cabinet quickly replaced the early PERQ-2. The "T1" model comes with an 8" Micropolis hard drive; the "T2" model with a 5.25" MFM hard disk in various makes and capacities. The 8" floppy, VT100-style serial keyboard and Kriz tablet are standard. The rare "T4" model is essentially a T2 fitted with a 24-bit board set.

Table 1: Chassis types.

Starting with *PERQemu* v0.5.8 the first PERQ-2 model (often referred to as "K1" or "Krismas," sometimes "T0") can be configured. Extremely rare, this updated model was meant to be smaller and quieter than the original cabinet, but is generally characterized as hotter, more cramped, and harder to work on. By some estimates only a few dozen or so were made, and few references exist in any of the surviving literature and marketing material.

For emulation, selecting the `PERQ2` chassis enforces a limit of one 8" Micropolis hard disk and allows both the 4K or 16K CPU. The PERQ-2/T1 is essentially the same machine, but in an ICL-designed cabinet that was common to all of the subsequent PERQ-2 models. As the OS software doesn't seem to distinguish between the T0 and T1, and *PERQemu* doesn't yet have a graphical front end that would show you the difference in cabinet types, the `PERQ2` chassis selection represents both models.

The `PERQ2Tx` designation represents the T2 or T4 models in the ICL-designed cabinet. Selecting this chassis will allow two 5.25" MFM disk drives but limit the processor selection to the 16K or 24-bit CPUs as discussed in the next section.

Processor Type

The system requires a CPU board. The "`configure cpu`" command allows selection from three types, subject to some limitations based on chassis type:

CPU Type	Description
`PERQ1`	The basic 4K, 20-bit CPU works with the PERQ-1 and PERQ-2 chassis, providing for a maximum of 1 megaword (2MB) of memory.
`PERQ1A`	The 16K, 20-bit CPU board is universal, and works with any chassis type. This processor has the widest software compatibility of the available types.
`PERQ24`	This rare processor is an extension of the `PERQ1A`. It offers the same 16K of writable control store, but extends the major datapaths and physical address bus to 24 bits for a much higher memory capacity. For emulation purposes this processor is only supported in the `PERQ2Tx` chassis.

Table 2: CPU board types.

In PERQ terminology, 4K or 16K refers to the size of the writable control store, a block of fast static RAMs which hold the microinstructions the processor executes. There are subtle differences that the microcoder must be aware of, and some software limitations; these are detailed in the *PERQ Operations* chapter.

The original 4K processor is provided for completeness and historical accuracy. As a general configuration guideline the 16K CPU is the best choice, and is required for later versions of Accent, PNX, and FLEX. All versions of POS can run on the 4K processor.

Configuring Memory

A variety of memory boards exist for the PERQ, differing in total capacity and the type of video display they support. *PERQemu* groups all of them into one "universal" memory board which can drive both display types. Though the 24-bit CPU can address up to 16 megawords (32MB) of RAM, the practical limit given the DRAM technology available in 1984 is 4MB. While there are some oblique references to an 8MB configuration, no known version of POS, Accent or PNX can actually use that much memory. *PERQemu* caps support at 8MB, but this can easily be changed in the source.

Use the "`configure memory`" command to select how much RAM is available to the virtual machine. With no arguments, a usage message is printed:

```
> configure memory
Configure the memory capacity, from 256KB to 8MB.  The CPU type determines
the maximum memory capacity, which must be a power of two:
    20-bit CPU: 256, 512, 1024 or 2048 (KB)  or 1, 2 (MB)
    24-bit CPU: 2048, 4096 or 8192 (KB)      or 2, 4, 8 (MB)
PERQemu will round up to the nearest legal value.
> configure memory 2
Memory size set to 2MB.
```

I/O Board Types

There are two basic PERQ I/O boards, each with two variations. The chassis type selected determines which I/O board may be configured, while the software you intend to run determines which variant is required (detailed in the chapter *PERQ Operations*). The following table explains the differences:

I/O Board	Description
`IOB`	The original PERQ-1 I/O board. This board contains a 2.45Mhz Z80 and runs the old v8.7 firmware. It contains a Shugart hard disk controller.
`CIO`	*PERQemu* uses "CIO" to distinguish an IOB running the new v10.17 Z80 firmware. It also encompasses a very rare ICL-produced variant that can drive either a Shugart hard disk *or* an 8" Micropolis hard disk.
`EIO`	The "Ethernet I/O" board, used by all PERQ-2 models. It features a faster 4Mhz Z80, runs v10.17 firmware, has a battery-backed real-time clock chip and a second RS-232 port. The EIO can drive 8" Micropolis *or* 5.25" MFM hard disks, depending on the chassis type.
`NIO`	An "Ethernet I/O" board without the Ethernet.

Table 3: I/O board types.

The "`configure io board`" command is used to make your selection:

```
> configure io board cio
IO Board type CIO selected.
```

When you change the I/O board type, *PERQemu* attempts to remap any assigned media files from the old board selection to the new one. While configuration commands may be given in any order, you may want to assign or load media as the last step; that way the Configurator can verify that the disk controller on your configured I/O board is compatible with the selected disk images.

As of *PERQemu* v0.5.8 all four I/O board types are emulated, enabling most PERQ-1 and PERQ-2 configurations to run. At this time the CIO Micropolis hard disk support is not yet implemented, as a viable OS image has not been found to verify its operation.

Display and Tablet Options

PERQemu supports both of the PERQ's monochrome graphics displays: the 768 x 1024 portrait mode display, and the 1280 x 1024 landscape display. The portrait display is compatible with every model and is supported by every PERQ OS. The landscape display was introduced with the PERQ-2 line, but a few extremely rare PERQ-1 landscape memory boards were produced. *PERQemu* allows you to configure either display, and both have been tested successfully!

The "`configure display`" command does the obvious and expected thing. Refer to the *PERQ Operations* chapter for details about software restrictions, as some older OSes do not have landscape support, and some that do require a separate disk boot be created to enable it.

Two types of digitizing tablet and pointing device are available: the PERQ-1 generally uses a GPIB-connected Summagraphics BitPad One, while the PERQ-2 uses a Three Rivers-produced "Kriz tablet." Named after its creator J. Stanley Kriz, a brilliant hardware engineer and co-founder of the company, the Kriz tablet comes in portrait and landscape variants. *PERQemu* emulates the appropriate version based on the display selection.

The "`configure tablet`" command lets you attach one *or both* types to your PERQ. Although the operating system will only use one or the other, later versions of POS allow you to choose which one to activate when you log in. The chapter *PERQ Operations* explores this in depth.

Assigning a Custom Keyboard Map

New in version v0.8.5, *PERQemu* allows the customization of keyboard mapping between your host environment and the virtual PERQ. This gives new flexibility to accommodate personal preference, to overcome limitations of a given host platform (such as laptops that do not have the keypad that the PERQ 2 layout has), and to allow for limited internationalization for non-English keyboards.

The process required to create a custom keyboard map (or keymap, for short) is slightly cumbersome, with an entirely new subsystem provided to create and manage them. If the default keymap is acceptable, no extra configuration step is needed, and you can skip to the next section. Otherwise, the complete details are given in Appendix D, *Keyboard Maps*.

Keyboards are specific to the type of machine you are configuring: the PERQ-1 keyboard for IOB or CIO configurations, or the PERQ-2 "VT100-style" keyboard for all EIO (or NIO) machines. Because the PERQ-2 layout is a superset of the original, *PERQemu* stores mappings for both types and will only apply the individual mapping commands that are appropriate for the machine type that's configured. Key mappings that don't apply or are unrecognized are quietly ignored.

The Settings subsystem can supply a default keymap that's applied to *all* configurations, which can be useful on a laptop or host with a peculiar keyboard layout. When a new virtual PERQ is started up, the Settings are consulted first but may be overridden by a keymap specified in the configuration. See the section *Setting User Preferences* later in this chapter for more information.

To use a custom keymap with a specific configuration, the "`configure keymap`" command takes one of three parameters:

- the name of a map to use. *PERQemu* loads saved keymaps from the `Conf/` directory at startup and provides tab completion for those that are preloaded, but will fall back to searching for a filename otherwise;
- the reserved name "`default`" to remove a previous selection;
- the reserved name "`none`" to override any Settings preference and use the built-in keymap.

Finally, you can apply a keymap after the PERQ is configured and running by using the new Keymap subsystem. A summary of the new commands is given in chapter four, *Command Reference*.

I/O Options

Every PERQ contains two slots in its card cage for optional boards: a CPU Option and an I/O Option. No specific CPU Option board was ever produced, however, so *PERQemu* combines these into one virtual option board. While some of the optional hardware available is not yet implemented in the emulator, the Configurator allows you to see what should be coming in future releases.

To select an option board, use the "`configure option board`" command to choose which board to install:

Option Board Type	Description
`OIO`	A multi-function board that contains up to four controllers in any combination: `Link`, `Ether`, `Canon` and `Tape`.
`MLO`	The "Multibus-Laser Option" board provides a Multibus-I compatible interface to the PERQ. A `Canon` interface is built-in. Sub-options include: `SMD` and `Tape`.
`Ether3`	This full-size board provides a 3Mbit Ethernet interface.
`None`	Removes any configured board and leaves the Option slot empty.

Table 4: I/O Option board types.

Status of I/O Option Development

Currently only the OIO option board is emulated. This is by far the most common option, as it's the only way to add Ethernet to a PERQ-1. The QIC Tape option was added in v0.4.8; an initial "fake" Ethernet device was added in v0.4.9. A preliminary "real" Ethernet was introduced in v0.5.2, while the Canon interface appeared in v0.5.4.

The MLO was once thought to have been forged from pure *unobtanium*, but several boards have been recovered and source code for the driver appears to be complete. Documentation and schematics exist to some degree. The primary lure of emulating the MLO is to allow PERQ-2 configurations to attach great big SMD hard disks, but verifying emulation accuracy without a working hardware model to compare to will make things difficult.

The "Ether3" board was not officially a product of Three Rivers Computer, as it was mostly used at Carnegie-Mellon University to connect to the early "experimental" Ethernet. (CMU was one of the sites that received a bunch of Xerox Altos with their original Ethernet interfaces in the 1970s.) Only one complete PERQ 3Mbit board is *known* to exist, wire wrapped on a full-size prototyping board. While there are no schematics or documentation for it, software support still exists, so adding a driver to *PERQemu* is a (very) long-term goal.

Configuring Sub-options

The OIO and MLO boards are multifunction boards that come in different varieties. The "`configure option add`" and "`configure option remove`" commands allow you to selectively customize these boards, as described in the table below:

Option	Description
`Link`	Provides a 16-bit DEC-compatible parallel interface that allows one PERQ to connect directly to the control store of another. *(In hardware this is a "universal option" that can be installed in the CPU Option slot of any machine.)*
`Ether`	Provides a 10Mbit Ethernet interface for the OIO board when installed in a PERQ-1. Conflicts with the Ethernet interface on the PERQ-2's EIO board.
`Canon`	Provides a video bitstream interface to the early Canon laser printers, the LBP-10 (240dpi) or the LBP-CX (300dpi). Can be added to the OIO board, and comes built-in on the MLO board.
`Tape`	When added to the OIO *or* MLO board, it provides an interface to the Archive Sidewinder streaming tape drive. *(The streamer interface is also a universal option and can occupy the CPU Option slot.)*
	Far, distant future: the MLO board should also emulate the Ciprico Tapemaster Multibus controller for connecting a 9-track ½" tape drive.
`SMD`	When added to the MLO board, it provides a Multibus Storage Module Disk controller (Interphase SMD 2190).

Table 5: I/O sub-options.

Since the `Link` option is probed as part of the boot process, the emulator currently implements enough to let the microcode know that nothing is connected. Unsupported options for the OIO are quietly ignored. The Configurator will explicitly disallow any configuration choice that would prevent the virtual machine from starting up. It also checks that the options selected are appropriate for the board type configured.

Serial Ports

Every PERQ-1 has a single RS-232 port that runs at up to 9600 baud. The PERQ-2 has two ports and can run them at 19200 baud. These are called RS-232 A (device `RS:` or `RSA:` in POS) and RS-232 B

(device `RSB:`) in most of the operating system literature. The hardware uses the Zilog SIO and CTC chips connected to the Z80 on the I/O board, with software support for programming them directly. On both boards port A can do "high volume" DMA transfers for greater performance; port B cannot.

PERQemu emulates these chips at the register level, providing a translation to either a "real" serial device connected to the host computer, *or* a "pseudo device" called `RSX:` that lets the emulator transfer text files to and from the host by pretending to be a PDP-11. If nothing is configured, a "null port" is attached that simply acts as a data sink. This is explained in detail in the section *Working with Communication Devices* below.

Network Address

Each PERQ can have an Ethernet interface. As an early adopter of Ethernet, most PERQ software was written to display the 48-bit address as three *signed* 16-bit integers, before the formatting convention of six bytes (octets, in network parlance) displayed in hexadecimal separated by ':' or '-' became common. Don't be alarmed if you see a negative number in a PERQ Ethernet address!

PERQemu sets the first three octets to `02:1c:7c` (Three Rivers' prefix assigned by Xerox) and the fourth octet according to the board type. The last two octets are read from the NET PROM on the hardware; they can be configured in the emulator with the "`configure ethernet address`" command. If set to zero or left unconfigured, a random address is generated at runtime. On PERQ-2 machines, the last two octets of the Ethernet address are also the system serial number that can be read from the backplane.

Disks, Floppies, and Tapes

Based on the chassis type and I/O board selected, the Configurator "knows" what type and how many storage devices can be attached. For practical use, every PERQ must have a bootable hard disk or floppy disk loaded. The configuration of storage devices is explored in the section *Working with Storage Devices* below.

Checking for Errors

To make sure your custom configuration is valid, the "`configure check`" command can be run at any time. The Configurator will make sure there are no conflicts, alerting you to problems that must be resolved and issuing warnings about unusual or rare configurations that may have limited software support. An example dialog to illustrate the types of messages produced:

```
configure*> show
Configuration: test
Description:   A test system
Filename:      Conf/test.cfg
--------------
Machine type:  PERQ2
CPU type:      PERQ1A
Memory size:   4MB
Display type:  Portrait
Tablet type:   BitPad
IO board:      CIO
Option board:  OIO  Options: Link, Ether

Storage configuration:
----------------------
Unit: 0  Type: Floppy    File: <unassigned>
Unit: 1  Type: Disk8Inch File: <unassigned>

configure*> check
Configuration is not valid:
PERQ1A (16K) CPU supports a maximum of 2MB of memory.
configure*> memory 2
Memory size set to 2MB.
configure*> check
Configuration is not valid:
IO Board type CIO not compatible with PERQ2 chassis.
configure*> io board eio
IO Board type EIO selected.
Note: Updating storage from CIO to EIO.
configure*> check
Configuration is not valid:
OIO Board Ethernet option conflicts with EIO board.
configure*> option remove ether
IO Option 'Ether' deselected.
configure*> check
Configuration is valid, with warnings:
The PERQ won't boot without a hard or floppy disk present.
configure*> done
Note: the configuration has been modified but not yet saved.
Use the 'configure save' command to save your changes.
>
```

Note that the Configurator doesn't impose any particular order in which commands must be given. It may complain when you enter a value that is in conflict but it doesn't prevent you from doing so. When the `power on` command is given, however, the emulator will refuse to start any configuration that doesn't pass the `check` rules.

Naming and Saving Configurations

Finally, a custom configuration must be named before it can be saved. The "`configure name`" command accepts a string which will form the basis of the saved filename. In general, you should

- use a fairly short, simple name without spaces or special characters;
- let the Configurator save the file in the `Conf` directory automatically.

Of course, you may also choose to provide a pathname to store the configuration wherever you please, but at startup *PERQemu* won't know where to find it and you'll have to type the same pathname in full to reload it.

The "`configure description`" command lets you add a one-line "human readable" description for the configuration. This is optional; it's displayed by the "`configure list`" command to show what each file contains. For example:

```
> configure
configure> name s5lisp
configure*> description "PERQ-1 Landscape with Accent S5, Lisp"
configure*> list
Standard configurations:
     default  - Default configuration
      f16dev  - POS F.16 development machine
         …
Current configuration:
      s5lisp  - PERQ-1 Landscape with Accent S5, Lisp
```

To save the completed configuration, "`configure save`" writes out the file:

```
configure*> save
Configuration saved to 'Conf/s5lisp.cfg'.
configure> done
>
```

That's all there is to it. Easy as pie!

Working with Storage Devices

The PERQ supports a range of storage devices, including floppy disk, hard disk and tape drives. In *PERQemu* these devices are emulated at the block level, stored as "media images" in files on the host computer in a number of different supported formats.

- ***Warning:*** *PERQemu* does not automatically save modifications to loaded storage media! All changes are made to an in-memory copy and are lost when the virtual machine shuts down unless they are explicitly saved. New preferences may be set that affects this; see the section *Setting User Preferences* below.

All PERQ operating systems generally require that a hard disk be present, though you can boot and run the real machine in a *very* limited way from a floppy disk alone. *PERQemu* is not tested without a hard disk present, and the Configurator will complain if one is not attached. The distribution includes many hard disk images for a variety of configurations, and the growing library of software will expand as additional drive types and operating systems are added to the emulator.

Supported Devices

Each PERQ can support one type of internal hard disk based on the chassis type and I/O board selected. The Configurator checks that only the correct type of image is attached to the disk controller in the virtual PERQ. Table 1 shows the maximum number and type of supported devices:

	PERQ-1	**PERQ-2**	**PERQemu**[4]
Floppy drives	1	1	2
Internal hard drives			
14″ Shugart	1	-	2
8″ Micropolis	1	1	2
5.25″ MFM	-	2	4
External hard drives			
SMD	-	4?	4

[4] *PERQemu currently allows only the maximum number of drives that the actual hardware supports, but in future releases (as software and microcode is enhanced) we may expand the limits. This column represents the theoretical maximums.*

	PERQ-1	PERQ-2	PERQemu[4]
Tape drives			
QIC streamer (¼")	1	1	1
9-track (½")	-	1?	1

Table 4: Storage device types and limits

- *Author's note:* While the 8" floppy drive was always listed as an "option," I've never actually come across a PERQ that didn't have one attached. Because it is essentially free to emulate, all standard configurations in *PERQemu* have a floppy drive attached as unit #0. You are welcome to configure a machine without one, but that option has not been exhaustively tested and may result in boot failures or other problems.

Each PERQ model supports only one type of internal hard drive, based on the chassis type and I/O board type selected – so the PERQ-2 models may contain 8" Micropolis *or* 5.25" MFM drives, but not both. As discussed in the *Configurator* section above, external SMD and tape drives are connected by optional I/O boards and are not yet implemented.

Unit Numbers

Each device in *PERQemu* is assigned a "unit #" to uniquely identify it when more than one of a given type is configured. In this release it's a very straightforward mapping, as the options are quite limited:

Unit 0:	Floppy drive	(all configurations)
Unit 1:	Hard drive	(for PERQ-1 or PERQ-2, the only hard drive)
Unit 2:	2nd hard drive	(only in PERQ-2/T*x* models)
Unit 3:	Streamer tape	(optional, when configured)

For compatibility with *PERQemu* versions prior to v0.5.0, the "`load floppy`" and "`load harddisk`" commands are provided as shortcuts for loading units 0 or 1, while new command syntax allows specifying the unit number (in multi-drive configurations). Many commands can automatically determine which unit to assign. When configured with an I/O Option board with the streaming tape, the command "`load tape`" automatically selects unit #3.

The Disks Directory

In the discussion that follows, *PERQemu* assumes that all media files are stored in the `Disks` directory provided in the distribution. This is located in the base directory (where `PERQemu.exe` resides) as discussed in the *Getting Started* section above.

A simple algorithm is used to find files to load, trying each step in turn and returning the first match:

1. Look for the file name exactly as given;
2. If no directory was specified, look in `Disks` for the file name;
3. If the file does not have an extension, retry steps 1 and 2 for each of the known extensions.

While the distribution is currently just a handful of files, an archive of many *hundreds* of floppy, tape and hard disk images is being assembled, so it is recommended that you choose simple names and allow *PERQemu* to apply the correct directory and file extensions when loading or saving media. Future releases may rename `Disks` or add additional user-specified directories to a "search list" to keep things manageable.

For now, any hard disk, floppy or tape images you create or download should be dropped into `Disks` so *PERQemu* can find them without a lot of typing!

Supported Media Formats

Since *PERQemu* v0.5.0, a new common *PERQmedia* file format has been available to store *all* of the supported storage devices. While the world may not need Yet Another File Format, the PERQ universe is fairly static and self-contained; translating images into the new format provides both data compression and a standard 32-bit CRC for verifying the integrity of the archive, features which the older formats do not have. Text and graphical labels may be stored in the file as well, which future versions of *PERQemu* can use to aid in the selection of media to load.

While the *PERQmedia* file format will be used by default, *all* of the previous formats may still be used as well, and utilities are available to transpose between them. *PERQemu* recognizes these file types:

> **PRQM** format: the new unified format; uses the file name extension "`.prqm`". Can be used to store any supported PERQ storage device — floppy, hard disk or tape.

PHD format: files ending in "`.phd`" contain Shugart 12MB or 24MB hard disk images, suitable for use with older versions of *PERQemu*. These drives are only supported on PERQ-1 models.

IMD format: floppy images in IMD format typically have the "`.imd`" (or DOS "`.IMD`") file extension. *PERQemu* can read and write these, and will preserve the media label information when transferred to/from PRQM format.

PFD/raw format: older floppy images might have a "`.raw`" or "`.pfd`" extension. These were likely converted from DMK or IMD images into a "raw" format with no metadata. The PFD format was my brief and ill-advised flirtation with adding a small "cookie" to the first sector of a raw floppy image (undetectable by *PERQemu* or the emulated PERQ) that could provide some hints as to format and contents.

❖ *Note*: While this version of *PERQemu* can read both formats, if asked to write a "raw" floppy it will always include the PFD cookie and will append/replace the extension with "`.pfd`" to more uniquely identify the image.

TAP format: *PERQmedia* now supports the SIMH magtape format for both reading and writing. These files are currently limited to QIC tape formats with fixed 512-byte blocks. The "`Stut`" utility (available for POS and later versions of Accent) provides a way to dump and restore entire partitions at a time. TAP files end in "`.tap`".

Additionally, some floppies archived with a "`.img`" extension may in fact be readable too: just change the extension to "`.raw`" and *PERQemu* will load them if they use the standard IBM-compatible 8" format. In v0.7.7 new commands for accessing hard disk images in "`.img`" format are also provided.[5]

Loading, Saving, and Unloading Media

Each configuration optionally stores the names of media files to load at startup, so that specific peripheral or machine configurations may be paired with the necessary software to support them. The "`configure assign`" command establishes a mapping:

> **`configure assign`** <***file***> [***unit #***]

[5] *The* PERQmedia *library may be updated in a future release to provide seamless reading and writing of IMG files.*

Note that *PERQemu* figures out the appropriate unit number based on the file's contents. In a dual-disk PERQ-2/Tx configuration, unit 1 is default, so you must specify unit 2 to load the second drive. If left unassigned, media files must be selected at runtime after the configuration is loaded.

The "`configure unassign`" command removes the association of image files:

```
> configure unassign <file>
      or
> configure unassign unit <unit #>
```

These assignment commands change the mapping of files in the configuration, but don't affect the loaded media if the machine is running.

The top-level `load`, `save` and `unload` commands can also still be used to update a configuration:

```
> load <device> <file> [unit #]
> save <device>
> unload <device>
```

Where <*device*> is "`floppy`," "`harddisk`" or "`tape`." These commands update the current media assignments in the same way the `assign` command does, but also affect the running machine. Because floppies and tapes are removable media, *PERQemu* allows these commands at any time.[6] The current configuration record is updated to reflect the status of the drive *and* the emulated drive receives a status change signal.

- *Note:* You should take care to not `load`, `save` or `unload` removable media while the operating system is trying to access it, of course, as data corruption may occur (just as if you suddenly eject a floppy from a real drive while it is being written). The emulator now overlays small icons of a floppy or tape cartridge on the lower right corner of the display when those drives access their media, as there is no other visual or auditory cue as with the hardware to let you know when the device is busy.

As the internal hard drives in the PERQ are not removable, a `load` or `unload` of a hard disk image is disallowed when the emulator is running; it must be powered down first (see *Managing the Virtual Machine*, below).

[6] PERQmedia *will accommodate removable hard disks too; SMD drives that use removable "packs" will be supported in a future release.*

Showing Storage Device Status

When the emulator is running, the "`storage status`" command prints a one-line status report for each configured drive and its loaded media, or one particular unit # if specified:

```
> storage status
Drive 0 (Floppy) is online, removable, no media loaded
Drive 1 (Disk14Inch) is online, loaded (Disks/f1.phd), writable
> storage status 1
Drive 1 (Disk14Inch) is online, type Shugart24
Filename:  Disks/newf1.phd (PHDFormat)
Flags:     Loaded, writable.
```

This information is updated in real-time as changes are made, such as loading a floppy or having the PERQ write to the device (setting its "modified" flag). If the media type supports a text label and one is present, it is displayed as well; this is a feature we hope to expand upon in future releases to assist in identifying the contents of media files:

```
> load floppy s5boot
Assigned file 'Disks/s5boot.imd' to drive 0.
Loaded Disks/s5boot.imd.
> storage status 0
Drive 0 (Floppy) is online, type SA851
Filename:  Disks/s5boot.imd (IMDFormat)
Flags:     Loaded, removable, bootable, writable.
FS hint:   POS
Label:
------
621003-00
PQS PERQ SOFTWARE  ACCENT S5 NON-SOURCE
Copyright (C) 1984, PERQ Systems Corporation
single density, double sided

Contents:   ACCENT PERQ1 BOOT NS
------
```

Changing Formats, Making Backups

The `save` commands shown above always update a file in place, using the same name and format as originally loaded. To save a copy of any loaded disk or tape image, the "`save` *`device`* `as`" commands allow you to specify a new filename and, optionally, a different file format:

```
> save harddisk as newf1 <tab>
Possible completions are:
      <CR>        imdformat   phdformat   prqformat   rawformat   tapformat
```

```
> save harddisk as newf1 prqformat
Updating from Disks/f1.phd
Saved Disks/newf1.prqm.
Drive 1 saved to 'Disks/newf1.prqm'.
```

In the example above, *PERQemu* makes a new copy of the "stock" POS F.1 image and writes a new file in the PRQM format. Because a simple name ("`newf1`", rather than a path) was given, the `Disks` directory was prepended and the correct file extension was appended. Advantages of the new format are more apparent when debugging output is enabled:

```
Updating from Disks/test.phd
MediaLoader: Compression ratio = 5.40
MediaLoader: Space savings = 81.47%
MediaLoader: Saving computed CRC: df22731c
Saved Disks/test.prqm.
```

- *Note:* It is recommended that you make a copy of any bundled disk images if you make changes to them and don't want to risk having them overwritten when a new release is installed. This can easily be done using your operating system's native file copy tools outside of *PERQemu*.

Creating and Formatting New Media

The `storage` subsystem allows the creation of new blank media files in a wide range of predefined types. To view the list of available device types, use the "`show devices`" command, as shown in the representative output below:

```
> storage show devices
Drive Type   Drive Class   Description
----------   -----------   ------------------------------------------------
Archive20    TapeQIC       Archive 3020I DC-300XL (20MB) QIC tape
Archive45    TapeQIC       Archive 9045I DC-300XL (45MB) QIC tape
Floppy1024   Floppy        Shugart SA851 DS/DD (1MB) floppy disk
Floppy256    Floppy        Shugart SA851 SS/SD (256KB) floppy disk
Floppy512    Floppy        Shugart SA851 DS/SD (512KB) floppy disk
Shugart12    Disk14Inch    Shugart SA4004 12MB hard disk
Shugart24    Disk14Inch    Shugart SA4008 24MB hard disk
Microp35     Disk8Inch     Micropolis 1202 35MB hard disk
Microp40     Disk5Inch     Micropolis 1304 40MB hard disk
Maxtor71     Disk5Inch     Maxtor XT-1085 71MB hard disk
Maxtor120    Disk5Inch     Maxtor XT-1140 120MB hard disk
Vertex51     Disk5Inch     Priam/Vertex V150 51MB hard disk
                           ...
```

The complete list of known storage devices is loaded by *PERQemu* at startup. This will eventually contain all the drives that have documented software support by one or more PERQ operating systems, as well as a few extra that were era-appropriate.

> To create a new type of device, the undocumented "`storage define`" subsystem is provided. Adventurous users may explore the file "`Conf/StorageDevices.db`" to see how *PERQemu* creates storage devices, but only the source code describes all of the parameters supplied and their use.

Next, use the "`create`" command to generate and format a new file. For example, to create a blank standard floppy image (double sided, single density):

```
> storage create floppy512 backup
Formatting new drive... done!
Saving the image...
Saved Disks/backup.prqm.
```

The "formatting" applied here is only the writing of empty data blocks; each device will require a specific operation to prepare it for use with the operating system. See the chapter *PERQ Operations* for more information about how each operating system uses storage devices.

This file can then be loaded using the command:

```
> load floppy backup
Assigned file 'Disks/backup.prqm' to drive 0.
```

Note that *PERQemu* always creates new media using the PRQM file format.

Using Hardware Disk Emulators

As of *PERQemu* v0.7.7, two additional storage commands are provided to import and export disk images using the "`.img`" format. This is a common raw sector format that can be used directly by (or transcoded from, using software utilities) various hardware disk emulators connected to a real PERQ. Examples of this are the Gotek floppy emulator, or David Gesswein's MFM emulator. Please see Appendix C, *Additional Resources* for more information.

Importing Disk Images

The "`storage import`" command reads data from a `.img` file into a pre-existing *PERQemu* media file. First you can use the `create` command to generate a new, empty disk image of the type you wish to import; this provides the disk geometry information the emulator needs to load the raw sector data. Note that you can overwrite an existing disk image if you choose (or by accident!) so take care not to clobber something important. *PERQemu* will check that the disk image you select is *not* currently loaded – in fact, you can use the `import` command without starting the virtual machine.

Hard disk images transcoded from the MFM emulator's native formats save the PERQ's logical header data in a separate file with a "`.metadata`" extension. So if you wish to import a file named "`v150.img`," the file "`v150.img.metadata`" must also be present. (These files are created by the "`mfm_util`" utility; see Appendix C for more information.) *PERQemu* will automatically add the extension if you don't supply the extra argument to the command, assuming that both required files can be found using the default search rules; otherwise, both pathnames may be specified. For example, to import a Vertex V150 disk image:

```
> storage create Vertex51 pnx-v150
> storage import pnx-v150 v150.img
```

In this example, a new disk image is created and saved as `Disks/pnx-v150.prqm`. Next, the data from `v150.img` and `v150.img.metadata` is loaded into it and saved. The new disk can then be configured and used with *PERQemu*.

Exporting Disk Images

To export an image to the `.img` format, use the "`storage export`" command. This offers two forms: by unit number (from the current configuration) or by file name. If the PERQ is running, it's wise to pause execution or make sure that the OS is not actively writing to the disk you wish to export. If it isn't running, the name of the currently *configured* disk is used. (This allows you to load a disk into the emulator, make changes to it, then export those changes directly from the in-memory copy with or without saving those changes back to the original media image. An esoteric use case, to be sure, but it also saves a tiny bit of typing.)

Exported data will be placed in the default `Output` directory to avoid possibly overwriting your original input data. *PERQemu* applies the same file parsing rules: given a simple filename, it will append the appropriate extension and prepend the default directory if you don't specify full paths.

Extending the example above, the command:

```
> storage export disk pnx-v150 v150
```

will load `Disks/pnx-v150.prqm` and write its raw sectors to `Output/v150.img` and `Output/v150.img.metadata`. These output files can then be transformed using `mfm_util` into the format suitable for emulation on the hardware.

Safe Mode

As of v0.7.7, the storage `create` and `export` commands respect a "safe mode" flag to help avoid overwriting existing files unintentionally. (Previously, the `create` command required an extra parameter to force overwriting an existing file; that is now removed.) By default, *PERQemu* starts up in safe mode. To change this and disable the guardrails[7]:

```
> storage unsafe
Unsafe mode: create/export commands will overwrite existing files!
> storage safe
Safe mode.
```

Import/Export Notes

PERQemu will let you import or export any hard disk geometry, but currently the only hard disk type supported by hardware emulation is for MFM drives as used with the PERQ-2/T2 or T4 systems. There isn't any known hardware emulator for the SA4000-series or Micropolis 1200-series drives, but if some enterprising FPGA hacker wanted to have a go it'd be a fantastic way to extend the life of older PERQ-1 and PERQ-2 systems!

The `import` command has also been successfully used to load FLEX archive format floppies, which use a custom format not compatible with the IBM standard 8" floppy geometries. These special diskettes are perfectly valid; *PERQemu* versions v0.7.6 and beyond now include support for the "FlexFloppy" 15 x 256-byte sector format. See chapter three, *PERQ Operations*, for information about installing and running this unique operating system.

[7] *Future versions of* PERQemu *may extend this save-me-from-my-own-fat-fingers protection to the other media saving commands, and/or only enforce the rules between the hours of midnight and 6AM.*

Working with Communication Devices

PERQemu allows several "real" devices on the host computer to be mapped to emulated devices in the virtual PERQ. These include serial ports, an audio output device, and an Ethernet adapter; others may be added in future releases. While the emulator provides the PERQ-facing side of each device interface, the C# runtime system or libraries bundled with *PERQemu* pass the data to and from the host, allowing the virtual PERQ to connect in a real way to external systems.

These mappings are established through the Settings subsystem, documented in the *Setting User Preferences* section below. Once a mapping is set up, it can be used by any configuration.[8] For some devices the association is made explicitly; for others the existence of the mapping allows it to be used implicitly without specific setup through the Configurator.

- *Note:* When establishing a device mapping, the Configurator tries to confirm that the filename you give exists, but doesn't attempt to open it or interact with it in the way the emulator does. Unpredictable or erroneous behavior may result if the mapped host interface is of the wrong type or is not supported by the runtime system.

Serial Devices

The "`settings assign rs232`*x* `device`" command is used to make the connection for serial devices. In the serial Settings and Configuration commands, the *x* designates port A (available on all PERQs) or port B (PERQ-2 models only):

```
settings> assign rs232a device <tab>
Possible completions are:
      /dev/ttyS0        /dev/ttyS1        RSX:
settings*> assign rs232a device /dev/ttyS0
Device '/dev/ttyS0' assigned to serial port A.
```

PERQemu will display a list of serial devices when you press tab, specific to your host operating system. This is generally in the form "`COM`*n*" for Windows, or "`/dev/`*xyz*" for Unix-based systems. If your device does not appear in the list, you can override the names given and type in a path. You can also optionally also set the baud rate, character size, parity, and number of stop bits. The defaults are the classic 9600-8-N-1 (9600 baud, eight bit characters, no parity, and one stop bit per character).

[8] *The rationale here is that the host-specific mapping is set up once and persisted with your other preferences, rather than for every configuration that uses the device. However, RS-232 port(s) on most "modern" computers require USB-to-serial converters, which often change their ID every damn time you plug them in, which might make this approach less than ideal. I ain't married to it.*

- *Note:* Because the PERQ Z80 firmware assumes it is directly controlling the hardware, it issues *numerous* software resets as the machine boots and may change the register programming in unpredictable ways that can make the C#/Mono runtime system *very* cranky. Now you assign the characteristics of your host device independently and *PERQemu* buffers the serial stream between the host and virtual machine so the PERQ only "sees" the data at the baud rate it can handle. Thus, configuring the host port to run at extremely high speeds (beyond the maximum 19,200 baud supported by the PERQ) increases CPU polling overhead, but not throughput.

Once saved, any configuration you load that uses RS-232 will be connected to your specified device. This may, of course, be changed at any time. If for any reason you simply wish to disassociate a particular device mapping, the "`settings unassign rs232`*x* `device`" command may be used to clear the assignment.

With the "`settings assign rs232`*x* `option`" command, option settings may be set as well. Due to quirks in the emulated SIO chip, the host/cable must either be configured to always assert the DCD signal, or one of the `DCDForceOn` or `DCDFollowDSR` option flags enabled to "fake it." Use this option if the PERQ is able to transmit but doesn't ever appear to receive data.

Additionally, flow control settings may be specified for the host device: "software" handshaking using XON/XOFF, "hardware" handshaking using the CTS/RTS pins, or both – whatever that means to your host OS and serial driver. The default is "`none`" (meaning, use the host's default behavior).

Serial devices require a separate configuration step. The "`configure enable rs232`*x*" command tells the emulator to use (or not use) the device:

```
> configure enable rs232a
RS-232 port A enabled.
> configure show
      …
IO board:       CIO
    RS-232 A:  /dev/ttyS0              ← as configured in the example above
Option board:  OIO  Options: Link, Ether
      …
```

The "`configure disable rs232`*x*" command disconnects the selected device for the current configuration. The mapping established in the Settings is preserved, of course.

Live Updates

In *PERQemu* v0.9.x and beyond, RS-232 configuration changes are also applied to the running virtual machine; earlier versions required a restart to take effect. This allows the device to be re-enabled if, for example, a USB-to-RS-232 converter cable is accidentally unplugged. In earlier versions, when the device "disappears" a fatal exception is thrown that causes a sudden emulation halt. *PERQemu* now attempts to catch and prevent this, instead immediately stopping serial communication on the affected device and disabling it in the configuration. When the situation on the host is corrected, the "`configure enable rs232`*x*" command can be re-issued to attempt to reconnect the device to the virtual machine.

Permissions and Performance

On Linux you may need to make the device file for your serial port writable, or access to it may be silently ignored. Typically this involves adding your user id to the "`dialout`" group, but please consult the documentation for your particular device and OS distribution.

As there is some overhead if the serial port is enabled and not used, you may elect to leave it unconfigured if you don't have an explicit reason to use it. Or you may want to connect the `RSX:` pseudo-device instead.

The RSX: Pseudo-device

The POS operating system offers a little known option to effect text file transfers over the serial port directly from the Shell. Copying a file to `RSX:` initiates a transfer using the DEC RSX11-M "PIP" transfer protocol. Since it's *unlikely* that you have a PDP-11 running RSX11 attached to your RS232 port, *PERQemu* provides a fake backend that simulates the remote (host-side) part of the protocol.

To enable this, simply specify `RSX:` as the device interface in the Settings `assign rs232`*x* `device` command shown above. (This is the same on Windows and Unix hosts.) Note that POS will only recognize the `RSX:` device on physical port '`a`'. More information about the use and limitations of this facility is provided in the POS section of the *PERQ Operations* chapter.

"Speech" Output

Every PERQ provides a "telephone quality" audio output capability through a Motorola MC3417 Continuously Variable Slope Delta Modulator (CVSD) chip. The output is essentially a 16kHz serial data stream pumped through one channel of an SIO chip to a DAC producing a monaural audio

signal; the CPU generates the waveforms and uses PERQ ⇒ Z80 messages and DMA to pump out the bits. No official way to record audio input was included in the hardware.

With the v0.9.x release, *PERQemu* now delivers audio output using the host's default audio driver as determined by the SDL2 library. While the available hardware for testing is extremely limited, Mac and Linux clients seem to work "out of the box." For some Windows 10 hosts, however, the default driver may not work. In this case, the "`settings assign audio device`" command can be used to specify an alternative:

```
> settings assign audio device<tab>
Possible completions are:
      [win devices here]
```

The "`directsound`" (sometimes shown as "`dsound`") is known to work on Windows 10 if the default "blah" driver doesn't produce consistent or reasonable output.

As the interface is new and still in active development, testing and tuning may result in offering some additional commands to manage the output (such as a CLI or GUI interface to mute or adjust volume). Note that *PERQemu* always attempts to play the output in real-time, regardless of emulation speed (too slow, or with rate limiting turned off, too fast).

As the CVSD output circuit is included on every PERQ I/O board, it is enabled by default. However, you can turn off the output if desired using the Configurator commands "`enable speech`" or "`disable speech`"; there is a slight performance benefit to turning off the audio output if you aren't running any PERQ software that doesn't use it (beyond the usual system "beeps"). Like the RS-232 ports, these changes take effect immediately if the virtual machine is running.

Ethernet

PERQ was an early adopter of Ethernet, and there is a decent amount of software that can use it for simple FTP, remote printing, or in the case of Accent, a whole lot more. *PERQemu* now supports the OIO Ethernet (for PERQ-1 configurations) and EIO Ethernet (PERQ-2). As with serial port setup, *PERQemu* separates the selection of a host Ethernet adapter from the virtual PERQ configuration. This allows you to set the preferred adapter to use for your PC once, then any configuration with the `Ether` option enabled will use it.

The host adapter is specified using the "`settings assign ethernet device`" command. The list of available adapters is determined by your host OS and will vary widely; *PERQemu* provides the list in the form appropriate for your machine. On a Mac or Linux machine this might produce:

```
> settings assign ethernet device<tab>
Possible completions are:
      en0         en1         null
```

On Windows 10, this might look like:

```
Possible completions are:
      Ethernet    Ethernet 2  null  VirtualBox Host-Only Network
```

You can also type in a name or identifier if the one you want isn't in the list. Note that an interface name with spaces in it (such as "`Ethernet 2`") must be enclosed in quotes.

The command "`settings show ethernet devices`" lists more information about the available Ethernet host adapters. This was enhanced in *PERQemu* v0.8.8 to show the C#/Mono runtime's view as well as the SharpPcap library's device names. This is helpful on Windows where more descriptive information is available, and the ID, name and SharpPcap device are all different!

```
> settings show ethernet devices
ID: {109C68A9-6929-4864-90AD-7417C7804ED4}  Name: VirtualBox Host-Only Network
VirtualBox Host-Only Ethernet Adapter
=====================================
  Interface type:     Ethernet
  Operational status: Up
  Hardware address:   0A0867530900
  SharpPcap device:   rpcap://\Device\NPF_{109C68A9-6929-4864-90AD-7417C7804ED4}

ID: {843128F1-FA08-4FB6-A314-DD7356D3FFD2}  Name: Ethernet
Intel(R) Ethernet Connection (5) I219-LM
=======================================
  Interface type:     Ethernet
  Operational status: Up
  Hardware address:   54BF647BEB00
  SharpPcap device:   rpcap://\Device\NPF_{843128F1-FA08-4FB6-A314-DD7356D3FFD2}

ID: {E5C00585-279F-4293-88F9-4364287409F5}  Name: PERQnet
Broadcom NetXtreme Gigabit Ethernet
===================================
  Interface type:     Ethernet
  Operational status: Up
  Hardware address:   000AF72CD100
  SharpPcap device:   rpcap://\Device\NPF_{E5C00585-279F-4293-88F9-4364287409F5}
```

The text following the `Name:` field is what the emulator uses to identify and open the adapter.

In the example above, the host machine has a second physical Ethernet controller dedicated for the emulator's use; the name "`PERQnet`" was assigned to make it easier to specify which one to use. An example of how to do this on Windows 10 is shown in the figure below:

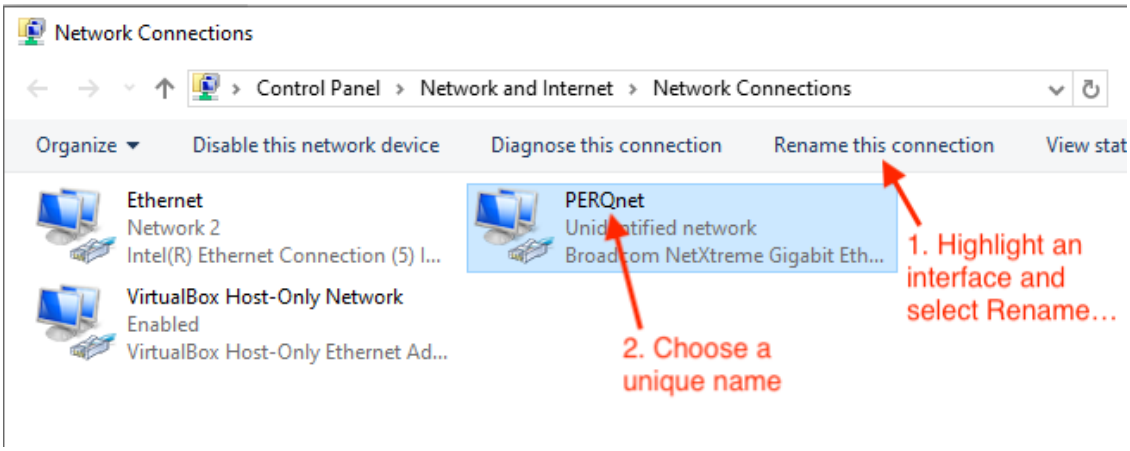

Figure 1: Renaming a Windows 10 network adapter.

The new name is then shown when "`settings assign ethernet device`" is used to make the assignment; this can help eliminate ambiguity or confusion on a machine with multiple adapters.

On all hosts the special name "`null`" is used to select the fake Ethernet driver, which spoofs enough of the hardware operation to let the PERQ think the interface is configured, but no traffic actually flows in or out. This allows Accent to boot and initialize itself properly, and is the default if no selection is made, no appropriate host adapters are found, or the program doesn't have the required privileges to open the device.

Encapsulation Options and Administrative Privileges

Currently the *PERQemu* Ethernet driver runs in "promiscuous mode," which allows raw access to the Ethernet frames being sent or received on the host's adapter. The PERQ uses a mix of proprietary protocols and has varying degrees of support for Xerox Network Systems (XNS) and PARC Universal Packet (PUP) protocols, as well as early TCP/IP (v4). The emulator attempts to pass this traffic through without interference, except for MAC address translation as described below.

A major caveat to this approach is that to access the host interface in this way *PERQemu* requires Administrator (Windows) or `root` (Mac/Linux) access to open the host device in a mode where it can send and receive raw packets. If you are understandably skittish about running the program with

escalated privileges, the emulator will detect that it cannot open the device and fall back to the `null` Ethernet implementation automatically. Ways to work around this, such as using UDP encapsulation or other ways to sandbox/limit the program's privileges to achieve the desired functionality are being explored; for now, you can review the source code and decide the risks for yourself.

Network Address Translation and the *PERQemu* Virtual Hub

As mentioned in the *Configuration* section, *PERQemu's* Ethernet implementation uses the original block of MAC addresses allocated to Three Rivers Computer in 1980. The three leading octets `02:1c:7c` are hardcoded into the low-level firmware and assumed by much of the software that runs on the machine. Rather than force the reassignment of the address on the host network adapter, and to help pre-IEEE 802.3 PERQ software coexist on modern networks, *PERQemu* instead performs a simple address and packet type translation on all Ethernet frames sent from or received by the virtual machine. The technical details of this approach are documented extensively in the source/Github distribution in `Docs/Network.txt`.

Currently there is no official command-line interface to interact with or manage the low–level Ethernet or NAT functionality, although there are plans to greatly expand the emulator's networking features in a future release. For now, the curious can use the "`debug dump ethernet`" command to see the status of the interface. (This is available in Release builds as well.)

Current Problems and Limitations

The PERQ Ethernet interface and software stack was built when 10Mbit/sec, half-duplex operation was state-of-the-art. In a typical TCP/IP-based network in the 21st century, the emulated PERQ is easily overwhelmed by the volume of background traffic; POS struggles to accomplish much of anything, while Accent is able to hang in much longer and offer some network functionality. To help alleviate this *PERQemu* attempts to filter the incoming packets to discard traffic the PERQ can't possibly handle (IPv6, for example), but the old software stacks aren't exactly "robust."

As most contemporary PCs have two or more physical Ethernet ports, it's recommended that you set up an isolated network – ideally with TCP/IP unconfigured on the private host interfaces entirely – so that *PERQemu* virtual machines (along with any real PERQs you own!) can talk to each other without the onslaught of traffic present on a noisy modern network running at speeds several orders of magnitude higher than the first Ethernet. Development of tools and techniques to mitigate this in the emulator is ongoing, and is the focus of the next milestone release.

Todo: Updates as testing/feature additions are completed
Provide a set of commands for tuning/monitoring/filtering/resetting the interface
Figure out/document how to potentially integrate Accent nodes into a home network!
Private virtual network between two *PERQemu* instances on one host!?
Work out IP encapsulation for ContrAlto interop (once 3Mbit emulation is added!)
Refer to Ops section for OS-specific Ethernet utilities

Working with Output Devices

The PERQ can work with a number of different printers using GPIB and RS-232 connections. The RS-232 driver *can* theoretically be used to drive a host-attached serial printer directly from the virtual machine, but this has not been tested (yet). A future release may include a simulation of some of the 3RCC or ICL supported GPIB printers, as these cannot currently be interfaced to the virtual machine any other way. Chapter three, *PERQ Operations*, has more details about the available hardcopy options for each operating system.

Canon Laser Printers

The first volume "desktop" laser printer from Canon was adopted very early by Three Rivers, with a custom hardware controller and driver software appearing in 1981[9]. *PERQemu* now emulates both of the supported print engines: the LBP-10 (240dpi), and the popular LBP-CX (300dpi). These are connected via a 37-pin cable and use the Canon "video interface," where the PERQ renders a bitmap in memory and sends it to the printer as a serial stream of bits. As an historical aside, this same interface is how certain "low cost" models of the Apple LaserWriter II, Sun SPARCprinter, or NeXT Laser Printer would operate years later, until integrated network interfaces and Postscript interpreters became common and more affordable.

To add the laser printer option to any PERQ, two steps are needed. First, choose the model by specifying its resolution (expressed in dots per inch):

```
> settings canon resolution 240          ← selects the LBP-10
> settings canon resolution 300          ← selects the LBP-CX
```

The default resolution is used for all configurations that include the Canon controller. While this setting may be changed at any time, the virtual machine must be powered off and back on for it to take effect. Second, configure the OIO option board:

```
> configure option board oio             ← if not already specified
> configure option add canon
```

With the `Canon` option selected, output from the PERQ's laser printing software will generate full-resolution bitmaps of each printed page. These bitmaps are captured by the Canon driver and written to the default Output directory in common image file formats.

[9] *A print ad from the time says "For hard copy output, you can add our laser printer for* just *$18,000." In* PERQemu*, it's free!*

Canon Runtime Options

The Canon driver offers two options that may be changed at any time, even when the virtual PERQ is running: output format and paper size. If the printer is busy, changing these settings will not take effect until the current print run completes.

Use the command "`settings canon output format`" to choose PNG or TIFF; JPG is not currently implemented. Additional formats may be offered in a future release. TIFF is the default.

The advantage of PNG is that compression makes the files very small, while uncompressed TIFF images are *much* larger. However, PNG can only store one image at a time, while TIFF files may store multiple images in a single file. When a multi-page document is printed, the Canon driver queues them in memory until the last page is printed then saves them all at once. The "`tumble`" utility (see Appendix C for a link) can convert a *PERQemu* TIFF file into a PDF using CCITT Group 4 lossless compression, which brings the file size down considerably.

Both Canon models support four paper cassettes: US Letter, US Legal, A4 and B5. *PERQemu* implements all four, allowing you to select the page size used when saving the output. *However*, there is no support for these in any of the standard PERQ software![10] As far as the standard microcode is concerned, *all* output is sized to print on US Letter (8.5" x 11"). Limited testing has shown that the output driver also produces reasonably good output if A4 is selected.

PERQemu limits the paper cassette to 100 sheets, just like the hardware does, and counts down with each successfully printed page. When it reaches zero the printer stalls until the paper is replenished. (This helps prevent runaway RAM consumption if a large document is printed all at once, but paper cassette capacity may be increased or made into a runtime parameter if necessary.)

You can set the default paper to be loaded using the "`settings canon paper size`" command. This size is loaded by default every time the virtual machine is started. To change the paper size when the virtual machine is running, the "`load paper`" command is available. With no arguments, `load paper` just refills the cassette in the current size; otherwise 100 sheets of the specified size are loaded. The `NoCassette` option behaves just like removing the input tray from a real printer: if a print is in progress it is interrupted and an alert symbol is shown on screen.

[10] *There are several third-party PERQ applications that can produce Canon output, but they have not been tested yet. This guide will be updated as further tests are done and possible future enhancements extend the flexibility of the Canon output driver.*

Multi-page Documents

Neither the Canon Video Interface Specification nor the PERQ microcode provides any documented, reliable method to determine the end of a page, let alone the end of a complete document. The entire apparatus is based on keeping a steady flow of data from the controller to the print engine to maintain its rated throughput of approximately 8 pages per minute.

Thus, the emulator simulates the timing of the printer's mechanical parts (rotating drum, paper feed, fuser warm-up time, etc.) to determine the start and end of a print cycle. When in "standby mode" a print request sets the machinery in motion, taking 5 seconds to bring the drum and laser up to speed. Once ready, the printer receives and saves each page as it is completed (if PNG output is specified) or adds it to a queue (for TIFF). When the PERQ stops sending data for more than 5.7 seconds, the real printer stops the moving parts and transitions back into its low-power standby mode. At this point *PERQemu* writes the entire queue as a single multi-image file (for TIFF output). If another separate document is started immediately after one completes, the emulator may combine them into one file.

The new `PrinterSpeed` rate limiting option can be turned off to limit the startup delay and reduce the timeout before returning to standby mode. Due to specific handshaking and timing constraints that the microcode expects during the active printing of each page, turning off rate limiting does not measurably decrease the actual time to print (approximately 6 seconds per page), but does save around 10 seconds in startup/standby delays.

The Canon emulation and configuration interface is still undergoing testing and refinement, so it may change prior to the next major release.

Managing the Virtual Machine

A typical session with *PERQemu* involves a few basic steps:

1. Choose a configuration to run;
2. Assign storage devices if necessary;
3. Power on the virtual machine;
4. Interact with the PERQ through the Display window;
5. Power off the virtual machine;
6. Reload the same (or load a different) configuration, rinse and repeat;
7. End the session by quitting the program.

The first three steps can be automated by writing commands into a script, so a predefined configuration may be selected and started up immediately. (See *Automating Actions with Scripts*, later in this chapter.)

When you first start *PERQemu* a default configuration is selected. Your preferences file will remember the last configuration you ran and make that the default on subsequent runs.

Starting and Stopping the PERQ

Once you have selected a valid configuration and assigned at least one bootable storage device, the emulator is started by the "`power`" command[11]:

```
> power on                    ← loads the machine but does not start it running
```

This commands the emulator to assemble the virtual PERQ according to the current configuration. Assuming the configuration is successfully set up, the assigned storage device images are loaded into memory. If any errors occur during startup, the console will display an error message to alert you to the problem. (Refer to the section *Checking for Errors* above for resolving configuration problems.)

Once the configuration is loaded, a large display window titled "PERQ" is created. At this point the virtual PERQ is "warming up," waiting for a command to start execution:

```
> start                       ← starts the CPU executing at ROM address 0x0000
```

[11] *A graphical interface that was in development featured a front panel that looks like the one used on the PERQ-2 machines, with a power switch, reset switch, and diagnostic display. After* PERQemu *v1.0 is "feature complete," work on a fully graphical v2.0 will commence!*

You may instead choose to use debugging commands, load custom microcode into the control store and jump to it, single step through the boot ROM or Z80 code one instruction at a time, or interrogate the state of the machine. Some limited configuration changes may also be made. Type "`start`" at any time to begin (or resume) execution.

As in previous versions of *PERQemu,* this shortcut will `power on` and `start` in one step:

> **`go`** ← *loads the machine* and *starts it running*

When it starts running, the PERQ first executes a complex bootstrapping operation to load an operating system. (See *Bootstrapping in Depth* below for more information about how the process works, and how to select an OS to load.)

To interact with the PERQ you must click on the display window. This directs your keystrokes and mouse movements/clicks to the virtual machine. At any time you may click on the console window to issue commands. To pause execution of the PERQ, select the console window and type:

> **`stop`** ← *pauses execution of the virtual PERQ*

At this point you can use debugger commands, save media, reset the machine, or simply pause execution to give your computer a break and go make a nice cup of tea. To restart from exactly where you left off, just type `start` again and away you go!

To shut down the emulation, use the console command:

> **`power off`** ← *stops (if running) and shuts down the PERQ*

Or you may simply press the close button on the PERQ's display window.

- *Warning:* Remember that *PERQemu* does *not* automatically save changes to modified floppies or hard disks unless you've told it to. When you power off the emulated machine without saving first, changes to the in-memory disk images are lost.

If you have requested that *PERQemu* ask for confirmation before unloading disks, a dialog box may appear over the display window giving you the option to save before closing; see the section *Working with Storage Devices*, above. See *Setting User Preferences*, below, for information about changing your autosave settings.

To exit *PERQemu,* as if you'd ever want to do such a thing:

```
> quit                    ← saves/asks if disks are modified, saves settings, exits
> quit without save       ← exits without saving settings or modified disks!
```

The `quit` commands automatically `stop` the emulator first if it's running.

The Diagnostic Display (DDS)

When first powered on the PERQ it does its best to make you think it's broken, showing no output or flashing strange patterns on the display. Only upon successful loading will the operating system clear the screen and announce itself. PERQ hardware provides a 3-digit numeric display that shows the progress of the boot sequence; *PERQemu* currently emulates this by updating the console window's title bar to read "DDS" followed by a three-digit number:

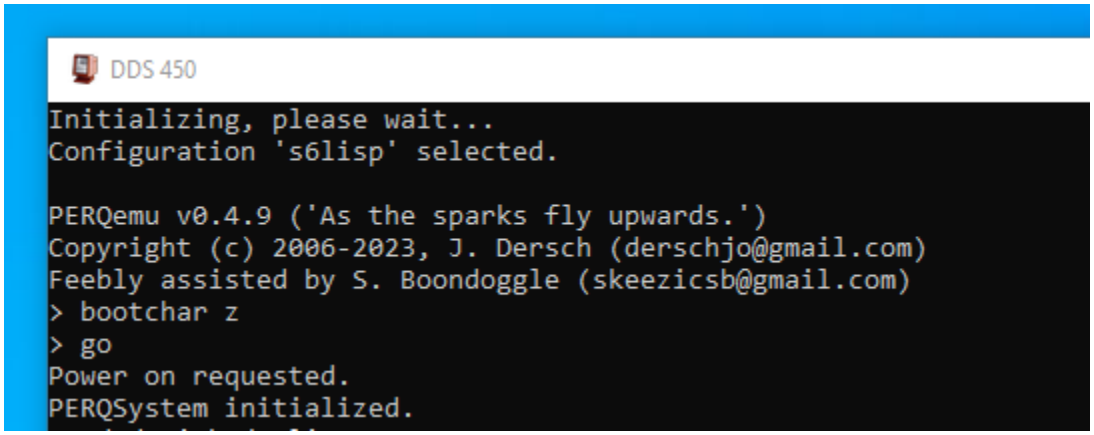

Figure 2: Console window with diagnostic display.

From 000 through 199 (roughly), a common set of diagnostic codes are used; values above 200 are specific to the operating system or diagnostic software being booted. When the boot is complete, the DDS will typically show:

PNX:	**255**
Accent:	**400**, **450** or **500**
FLEX:	**888**
POS:	**999**

The value displayed by Accent depends on the amount of memory configured (1, 2 or 4MB). While *PERQemu* allows you to create a PERQ-2/T4 with 8MB this does not appear to be supported. Yet.

Because *PERQemu* doesn't emulate the 2 minute warm-up time[12] that the old Shugart hard drives required, most hard disk boots will complete in just a few seconds. (Floppy boots, as on the hardware, take considerably longer.) There are several common milestones where it's normal for pauses to occur:

`010` Basic ROM diagnostics complete; the microcode is waiting for the disk to spin up or attempting to load the next part of the bootstrap

`030` The more extensive "VFY" microcode tests are running; this is accompanied by memory test patterns appearing on the screen

`151` The system bootstrap microcode "SYSB" is waiting for a boot character selection

If the OS doesn't come up and your PERQ really is stuck, consult the *Fault Dictionary* in the Bitsavers on-line documentation repository to figure out why. See Appendix B, *Bibliography* for links.

Boot Selection

It is not uncommon for a PERQ hard disk to contain several different operating systems, or different versions of the same OS. Certain types of POS floppies may be bootable as well, to provide a means to install a new operating system or run recovery and repair programs if the hard disk is damaged or corrupted. Each bootable device stores a table of 26 possible "boot letters," or just "boots" for short. The letters `a` .. `z` select the hard disk; `A` .. `Z` select the floppy disk (if a bootable floppy is loaded).

On the hardware, you select the boot by holding down a key on the keyboard after power on, or after pressing the boot (reset) button. The reliable way to do this is to wait until the VFY memory test begins (when the comb-like patterns first appear on the screen) at DDS 030, releasing the key after DDS 151 comes up. This way works with the emulator too; just remember to click on the display window so the PERQ gets your key presses, not the console!

PERQemu also provides a simpler method (that can be scripted):

> **`bootchar`** ← *without an argument, shows the current selection*

> **`bootchar c`** ← *sets the boot selection to any alphabetic character* c

- *Note:* This is one command where the case of the argument is significant!

[12] *If you're a real ~~glutton for punishment~~ stickler for accuracy, you* can *enable* `StartupDelay` *as a rate limit setting!*

If the boot character is unset, or if you don't press anything while the machine initializes, the default 'a' boot from the hard disk is automatically selected. *PERQemu* remembers the last selected boot character only for the duration of your session; it is unset when you restart the program.

Bootstrapping in Depth

This section is optional deep background into the periphery of PERQ geekdom that may be skipped by the casual user.

A unique feature of the PERQ is its "soft" architecture: the machine contains almost *no* software![13] When first initialized, *PERQemu* loads a very small ROM image into the CPU board and the I/O board's Z80 starts executing from its onboard ROMs. Everything else – an operating system kernel, language interpreter (which defines the instruction set), and I/O microcode – is loaded from a boot device. As mentioned above, the three phases of the microcode bootstrap are referred to as BOOT, VFY and SYSB (referring to the names typically given to the microcode used to implement them).

Only BOOT is encoded in the ROMs. At power up the ROM overlays the bottom half of the lowest 4K bank of the writable control store. It executes basic diagnostics to verify that enough of the processor is working to continue on to the next phase, reading VFY and SYSB into memory from external storage. The boot microcode tries to load from three[14] possible devices, in sequence:

Link board: A direct connection from a PDP-11 or another PERQ fitted with a special option board allows microcode to be loaded and executed under the control of a debugger. *PERQemu* currently only emulates enough of the link board to tell the microcode that nothing is plugged in;

Floppy disk: The PERQ asks the Z80 to try to boot from the floppy drive. This probes to see if a floppy with a specific "signature" is loaded and online, and if so is selected to provide the next phase of the bootstrap;

Hard disk: If no boot floppy is present, the hard disk is checked. Every PERQ hard drive has a dedicated boot area set aside (starting at cylinder 0) where the microcode for bootstrapping is written.

[13] *The author wryly acknowledges the irony of this statement. See Appendix A.*

[14] *Code to support network booting and diskless operation of Accent has been found. It would be* really *cool to add this in a future release, once Ethernet emulation is complete. Netbooting uses* ^A *through* ^Z *for boot character selection, naturally.*

If no boot floppy or hard drive is loaded, *PERQemu* (like the actual PERQ hardware) will simply loop forever. The DDS will usually display `010` if the boot cannot proceed. It may display `014` if there was an error loading from the device, perhaps indicating a corrupted or empty boot track.

VFY and SYSB are usually written into the boot area together; they are loaded as one image. The BOOT code first loads them into memory, then copies them into the high half of the control store. To indicate success, BOOT runs the DDS up to `029` when it hands off to the next stage. Once this is accomplished the ROM is disabled, and the only way to enable it again is by pressing the boot button (in *PERQemu*, using the `reset` command).

VFY's job is to run more extensive diagnostics of the processor and memory to prepare for the final stage of bootstrapping. It advances the DDS to `030`, initializes the video hardware (which also performs DRAM refresh), then runs a series of memory tests – the patterns seen on screen are the contents of main memory while the test runs! Upon successful completion, VFY runs the DDS up to `149` then jumps to SYSB.

SYSB announces itself by setting the DDS to `150`. It then resets the Z80 and polls the keyboard several times to see if a boot character is pressed, using the default '`a`' if nothing is received. SYSB uses the boot character as an index into a table of 26 possible boots from the selected drive's Device Information Block. Two OS-specific microcode binaries (usually with the extensions "`.boot`" and "`.mboot`") contain the kernel and I/O microcode, as well as the interpreter which defines the instruction set used by the system. POS and Accent run the "Q-code" interpreter, which supports Pascal, FORTRAN, C and Ada, while PNX runs "C-codes" which are optimized for C and Unix. These are read into main memory, then SYSB writes them into the control store. The Z80 is then turned off again. (You may notice that the Z80 runs at 0.0ns in the FPS display when it is turned off.)

Finally, SYSB advances the DDS to `198` and jumps to the entry point for the operating system itself. From that point the bootstrapping process is complete, and the selected OS takes over all remaining initialization of devices, filesystems, video display, and so forth. Each operating system can then use the DDS however it pleases; POS shows most of the major steps it takes as it starts up and provides an extensive list of error codes (see the *Fault Dictionary* referenced in Appendix B) while PNX and Accent are quite terse. Accent is unique in that it displays a final code based on the amount of memory detected. And there are some special bootable diagnostics that make extensive use of the DDS to display test results.

"Fascinating," I hear you cry...

Automating Actions with Scripts

PERQemu can read scripts of commands at startup or at any time from the command line. When running the Debug version of the program, scripts may be attached to breakpoints and run automatically when a specific event is triggered.

The format is simple: any plain text file containing valid CLI commands may be read and executed.[15] Note that the commands on each line must be fully specified; the interpreter will not expand or complete partial commands as it does when you type them interactively. To assist in the creation of scripts, a command to save the current history to a text file will be provided.

To run a script from the command prompt, preface it with an '@' character:

> `@`***scriptfile***

where *scriptfile* must be a qualified pathname, according to the host operating system's filesystem conventions. It may be absolute or relative to the directory where you launched *PERQemu*; currently there is no search list or file extension associated with loading scripts.

Limitations and Implementation Notes

Scripts may not be nested. Any line beginning with '`@`' in a script is ignored.

Blank lines, or comment lines starting with '`#`' are allowed – they're simply ignored.

PERQemu will always attempt to read the entire script, even when malformed, incomplete or failed commands are encountered.

While you may invoke a script at any point in the command hierarchy, scripts are always executed from the top-level prompt. You are free to navigate the hierarchy within the script, but when completed the current prompt is restored.

[15] *The astute observer will notice that the definitions of storage devices, configuration files, keyboard maps, and even the user preferences are all stored in exactly the same format... laziness is a programming* virtue *after all!*

Setting User Preferences

PERQemu offers a number of customizations that are stored between sessions in a preferences file. These are manipulated using the "`settings`" subsystem. Like other subsystems, type `commands` for a summary of the available commands, `show` to see the current settings, and `done` to return to the CLI top-level.

In the v0.8.5 release most (but not all) of the Settings commands are implemented. Please see chapter four, *Command Reference* for at least a cursory discussion of the current command set.

Autosave Options

As with prior versions of *PERQemu*, changes to disks made while the emulator runs only affect an in-memory copy; these must be saved if you wish to preserve them between sessions. For floppy or QIC tape media, the corresponding "`autosave floppy`" or "`autosave tape`" setting is applied any time a modified disk or cartridge is unloaded. For hard disks, the "`autosave harddisk`" setting is applied when the emulator is shut down with the `power off` or `quit` commands.

Choosing "`yes`" means *PERQemu* will always save modified media; "`no`" means changes are always discarded. "`Maybe`" means that a dialog will be presented only if the media is modified, allowing you to choose on a case-by-case basis. The default setting for all media types is `maybe`.

Device Assignment Options

The Settings "`assign`" and "`unassign`" commands allow you to map or unmap devices on the host for the emulator's use. These are typically set up once and apply to all PERQ configurations. The default for all currently settable device mappings is unassigned.

See the section *Working with Communication Devices* earlier in this chapter for information about mapping serial devices to the PERQ's RS-232 ports, or attaching an Ethernet host adapter to the PERQ's Ethernet interface.

Performance Options

A goal of *PERQemu*'s emulation is to run a virtual PERQ at precisely 100% of the actual hardware's speed, where the microengine executes one instruction every 170ns (5.88Mhz). Due to the way the PERQ's video timing is intimately tied to its system clock, accurate CPU emulation leads to a display refresh rate of exactly 60 frames per second. *PERQemu* provides a number of options to affect how the emulator interacts with the host to achieve this target execution rate.

The "`settings rate limit`" command accepts a number of flags:

`None`	Do not limit performance; run the emulation as fast as the host is able. This clears all the other flags.
`Default`	Sets the rate limit settings to program defaults.
`CPUSpeed`	Attempt to regulate the CPU to exactly 170ns per microcycle.
`DiskSpeed`	Accurately reflect disk drive mechanics for seek time, head settling, rotational latency, data transfer rates, etc.[16]
`TapeSpeed`	Regulate the tape streamer at the hardware's 30 inches/second rate.
`PrinterSpeed`	Accurately reflect mechanical timing of the Canon laser printer.
`StartupDelay`	Accurately reflect the disk drive spin-up delay at power on.[17]

The default settings are set for accuracy over speed. This doesn't impact performance much on hosts that can't run the emulation at full speed, but effectively locks in around 60fps on faster machines. While *PERQemu* has been run at an emulation rate in excess of 240fps (for a CPU speed of 23MHz!) with no observed incompatibilities or problems interfacing with peripherals, these purely empirical measurements are no guarantee that faster rates won't expose overruns, errors, or other timing issues.

Limiting hard disk speed with the `DiskSpeed` option is recommended for the Shugart hard disks, as seek timing is actually "baked in" to the Z80's firmware. Turning off the `TapeSpeed` limit allows the streamer to run at the 90 inches/second rate, rather than the 30IPS that the drive is rated for.

The Canon printer interface is driven by a combination of microcode, Pascal and a hardware state machine that requires specific timings to produce a properly formatted image. The `PrinterSpeed` option (on by default) tries to faithfully emulate the 8 pages-per-minute maximum throughput of the print engine. By turning off this option, *PERQemu* can shave a few seconds off of the timings to make print operations run a wee bit faster.

The emulator can be quite resource intensive on the host; *PERQemu* offers the "`pause when minimized`" setting to automatically stop emulation when the Display window is "iconified" and resume it when the application is brought back to the foreground. This *generally* works the same way on Windows, Mac and Linux hosts, where pressing the window's minimize button sends it to the dock or taskbar. The default for this setting is `true`.

---

[16] *Currently* PERQemu *only roughly estimates seek ramp timings and transfer rate limits and doesn't emulate the memory cycle stealing impact of DMA on CPU performance. "That would require an almost* fanatical *level of attention to detail," he said, updating the to-do list.*

[17] *This was useful as a debugging tool, actually, but waiting two minutes on every cold boot gets old* really *fast. But I left it in as an option if you want the full experience. Just need to play audio of the distinctive "Shugart rattle and squeal" when the SA4000 fires up...*

On the hardware, the "boot button" immediately causes the processor to restart. *PERQemu* will optionally pause the emulation when a `reset` command is typed, allowing additional commands to be issued before manually restarting the processor with `start` or `go`. The default for "`pause on reset`" is `true`.

Miscellaneous Settings

The "`settings display cursor`" option allows you to specify how the emulator displays the system cursor when the Display window is selected. On some systems the "`crosshairs`" option is less intrusive than the default arrow; or you can turn off the system cursor entirely by choosing "`hidden`". See chapter three, *PERQ Operations* for more information about how some PERQ operating systems handle mouse tracking.

By default *PERQemu* limits all of the output it generates to the `Output` subdirectory inside the installation directory. The "`settings output directory`" command allows you to specify another destination for trace logs, screenshots, printer output and other potential generated output. Trace logs are output here from Debug builds when file logging is enabled. Screen captures and bitmaps generated by the Canon laser printer are created with file names that should be obvious.

You can reset everything to default values with the "`settings default`" command, or undo changes that have not been saved by typing "`settings load`". The "`settings save`" command applies any changes you make immediately, although this is automatically done whenever *PERQemu* exits (unless you specifically choose to `quit without save`).

Let's Do The Time Warp Again

Most PERQ software is not Y2K aware, and the RTC chip on the EIO board can only be programmed with two digits for the year. The base date is assumed to be offset from 1980 by the Z80 firmware and low-level OS code. Some OSes will simply refuse to boot if the year is too far out of range!

By default, *PERQemu* sets the RTC to the host's clock when the emulator is started. In v0.7.8 a new "`settings rtc offset`" command is provided to allow the year to be computed using a later offset, thus clipping the year returned by the chip to a pre-Y2K value. The command:

```
> settings rtc offset 2020          ← subtract 40 years from the current date
```

will let the PERQ believe it's still in the 1980s (where many of us would rather be, right now, too).

This setting is persistent and applies to all PERQ-2 configurations. You can also use the "`configure rtc offset`" command to change it for just one configuration instead. See the next chapter for more info about date handling in specific operating systems.

Keyboard Options

The Settings subsystem now saves an optional default keyboard map. The "`settings keymap`" command lets you assign a custom map that will automatically be applied to all PERQs at start-up, but like the RTC offset this preference may be overridden on a case-by-case basis by the "`configure keymap`" command described earlier.

More information about the PERQ's "special keys" is given in the next chapter, *PERQ Operations*. Summaries of all the new key mapping commands is in chapter four, *Command Reference*, but the definitive treatise on the emulator's keyboard handling is Appendix D, *Keyboard Maps* – a work of Brobdingnagian intellect and inconceivable scope, truly.

Debugging Features

TBD. This is pretty "organic" and not well documented. See the *Command Reference* for the basics.

Highlights: extensive logging/tracing, breakpoints, `:variable` syntax, internal state and device driver visibility, and more. Most of these are implemented and working (well enough, with *plenty* of rough edges), though some are incomplete but aren't needed in normal day-to-day use (and the performance hit is substantial). Even in this rough state the emulator has proven invaluable for debugging microcode, though, which ought to appeal to the burgeoning class of fellow PERQ microcoders out there, right guys? Hello? <*tap tap*> Is this thing on?

3. PERQ Operations

Virtual Machine Interface

PERQemu's emulation environment translates keystrokes, mouse movements and other device interactions from the host to the specific protocols expected by the virtual PERQ being emulated. In general, the emulated machine behaves like a modern user would expect; this section describes a few areas that require special attention.

Special Keys

As described in the *Configurator* section of the previous chapter, *PERQemu* supports two keyboard types, based on the model selected. While the emulator hides the implementation details, there are several special keys that may not have direct equivalents or require remapping to work around host OS restrictions on modern keyboards:

PERQ-1	Host
HELP	**F5** or HELP
OOPS	**F6**
INS	ESCAPE or INSERT
DEL	DELETE
LINEFEED	**F7** or PAGE DOWN

PERQ-2	Host
HELP	**F5** or HELP
OOPS	**F6**
ACC/ESC	ESCAPE or INSERT
REJ/DEL	DELETE
LINEFEED	**F7** or PAGE DOWN
SETUP	PAGE UP
NOSCRL	SCROLL LOCK
BREAK	SYS REQ
PF1..PF4	F1..F4

Table 6: Special key mapping.

Bold denotes a UI change as of v0.8.2, where the default assignments for HELP, OOPS and LINEFEED were updated for consistency. On Apple extended keyboards with "*help*" and "*clear*" keys, SDL2 maps them to their default PC-101 type assignments *or* does not pass them through to the application at all! Thus, as an interim step they were relocated to the function key row.

If these defaults are not suitable for your given environment, custom key mappings may be created and assigned, as discussed in the previous chapter and at terrific length in Appendix D.

Mouse Input

Refer to the *Configurator* section in the previous chapter to review selection and configuration of a PERQ pointing device. In the discussion that follows, "mouse" will refer to the host mouse, touchpad or input device, while "puck" will refer to the PERQ's simulated 4-button BitPad puck, or the 3-button Kriz tablet puck (which looks like a mouse).

If that isn't confusing enough, the "system cursor" is the configurable cursor image that follows your mouse as you interact with the emulator, while the PERQ provides its own cursor image based on the specific software being run.

In *PERQemu* the mouse works approximately as you'd expect, though there are functional differences between a digitizer tablet and a modern mouse:

- the PERQ can use absolute tracking, while a mouse only provides relative input;
- a digitizer sends a steady stream of data, while a mouse transmits movements as they happen.

Unlike modern graphical interfaces where the system provides a cursor that is almost always active and visible, some PERQ operating systems leave tracking and using the tablet entirely up to the application being run. When enabled, the cursor may be tracked in *absolute* mode, where the position of the puck or stylus on the tablet maps directly to a coordinate on the screen, or in *relative* mode, where a series of small swipes in one direction moves the cursor relative to its last position (like a modern mouse).

To provide simulated absolute mode positioning, *PERQemu* tracks the position of the cursor based on its location in the Display window, sending coordinates 40-60 times per second based on the tablet type selected. In this mode the host cursor and active "hotspot" of the PERQ's cursor will track together. When you click outside of the Display, the emulated machine sees the cursor at the last position tracked as the mouse leaves the window; it then jumps back to the position where you click to return focus to the Display.

When operating in relative mode, the PERQ detects when the puck is off the tablet and calculates cursor movement based on the start and end points of the next mouse movement. To simulate the "off

tablet" behavior, hold down the Alt key (Option key on the Mac) while moving the mouse. You can then reposition the system cursor in the window before releasing the key to start a new swipe.

Button Mapping

Early PERQ operating systems were written to support the 4-button puck or 1-button stylus attached to a Summagraphics BitPad One. When Three Rivers introduced their own Kriz tablets with a 3-button puck, the fourth button ("blue") had no equivalent, so any function attached to that button had to be revised or a keyboard-initiated alternative provided. *PERQemu* maps the typical modern 3-button mouse in the expected way, but provides a modifier key to produce all of the BitPad buttons as shown in the figure below.

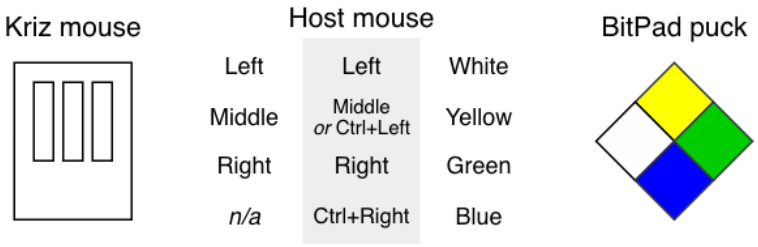

Figure 3: Host to PERQ mouse button mapping.

Display Window

With v0.8.1, *PERQemu* now resizes the PERQ's 1024-line display window to fit on screens that are too small, such as older laptops. If your host is equipped with multiple screens, the window will be resized automatically when you drag the display between them. The visible state of the Dock (or Taskbar) may affect the size calculation, so the window height should reflect the "auto hide" preference in your host's interface.

When the screen is less than full height, your mouse's scroll wheel (or the equivalent gesture on a touchpad) will smoothly pan the window up and down, adjusting the PERQ's mouse position accordingly. Alternatively, the HOME and END keys, if your keyboard is so equipped, will bring the top or bottom of the display into view immediately. Mouse wheel or HOME/END inputs are ignored when the display is full size.

- *Note*: Only vertical panning is supported; currently the window itself is not made arbitrarily resizable and a full-screen mode (while planned) is not yet available.

Floppy Formats

PERQemu doesn't "know" anything about the contents of a floppy image; it only presents it as a series of blocks to the virtual machine through a simulated floppy controller chip issuing commands to the simulated floppy disk drive. The only time the emulator peeks at the data sectors is to look for the "boot signature," a specific byte pattern that identifies a bootable diskette.

There are three[18] primary PERQ floppy formats you may encounter:

- DEC RT-11-compatible floppies;
- POS filesystem floppies;
- PNX floppies.

The RT-11 data floppies are supported in all PERQ operating systems and can be used for data interchange. To access these, each OS provides a utility described in the sections below.

PERQdisk, a standalone utility for reading and writing PERQ floppies, is now available. It takes advantage of the *PERQmedia* library and can handle all supported floppy image formats; see Appendix C, *Additional Resources* for details.

POS filesystem floppies are designed to act like a small hard disk; they can be mounted, booted from, and interacted with from POS (and MPOS) using all the standard utilities. All bootable PERQ floppies are single density, double sided, and contain a boot area for loading microcode.

PNX floppies come in many flavors and are *deeply* mysterious. Not much is known about them, but they are not to be trifled with. PNX boot floppies were seduced by the dark side of the format, twisted and *eeevill*; they seem to have a tiny POS-style boot area so the ROM/Z80 can initiate the boot load, followed by what is presumably a Unix filesystem used to mount the "miniroot" needed to bootstrap the PNX installation. Other PNX install floppies are also mountable, but it's unknown how these are created. PNX can also read RT-11 data disks with its "`fl`" utility (see below).

[18] *A* fourth *floppy format has entered the chat! FLEX uses a custom format as well as the more standard PERQ types.*

Hard Disk Formats

PERQemu is as agnostic about hard disk formats as it is about floppies; their contents are opaque to the emulator. All PERQ hard disks, however, use a unique sector format, supported by the controller and by *PERQmedia*: each 528-byte sector contains a 16-byte logical header for filesystem metadata, plus a standard 512-byte data block. The emulator does not operate at the physical level, emulating gaps and CRCs and the like, but it is able to flag bad blocks (typically from unreadable or damaged parts of archived physical disks).

POS, MPOS and Accent share a common on-disk format and use the logical header data in a consistent manner. A hard disk can contain several partitions shared by one or more of these compatible operating systems; partitions are generally limited to a maximum of 32,768 blocks (16MB) each. Software limitations are discussed in the original OS documentation. See the *Bibliography* for links.

PNX uses an incompatible on-disk format based on the Unix V7 filesystem. It is not clear if PNX takes advantage of the logical header.

FLEX can use an entire hard disk with a native (unknown) format, or can live inside a POS partition or a file inside a PNX filesystem! How cool is that?

The *PERQdisk* utility can read POS filesystems from hard disk images, and PNX support is in development; see Appendix C, *Additional Resources* for more information.

Tape Formats

As of v0.4.8, *PERQemu* emulates the one supported streaming tape drive option. The Archive Sidewinder 3020I provides 20MB of storage on DC-300XL ¼" cartridges (QIC). A library of PERQ tape images is being assembled and should soon be made available for use with the emulator; this allows for much simpler OS installations, for instance, as one tape can take the place of 20 double density floppy disks!

POS and Accent use a program called `Stut` to read and write backups of POS filesystems at the partition level. An external port of *Stut* designed to read and extract files directly on the host is being developed/updated to work with *PERQmedia*; it may be available at the time of the next *PERQemu* release. See Appendix C for more information.

PERQ tape images may be stored in the SIMH magtape format, loaded from "`.tap`" files. They may also be saved in the native *PERQmedia* "`.prqm`" format, with labels and other metadata. All streamer tapes use a fixed 512-byte data block.

Todo: Does PERQ "`tar`" work with the streamer, or only with ½" 9-track magtapes?
Is there any streamer support in PNX?
Finish the *Stut* port and start archiving tapes!

Supported Operating Systems

The PERQ can run a number of different operating systems. Efforts are ongoing to preserve old media archives and make them available in a form suitable for use with *PERQemu*. As of version 0.4.2, the following operating systems are known to boot:

- POS versions D.6, F.0 and F.1 (official 3RCC releases);
- POS version F.15 (released by Boondoggle Heavy Industries, Ltd);
- MPOS version E.29 (unreleased by 3RCC);
- Accent S4 (an early version from CMU, unreleased);
- PNX 1.3 (first public release by International Computers Ltd.).

PERQemu v0.5.0 introduces support for the CIO board and "new Z80" ROMs. This allows PERQ-1 configurations to run:

- POS version G releases through G.7 (official 3RCC releases);
- Accent S5 and S6 (Release I and II from 3RCC/CMU);
- Accent S7 (was to be Release III; later available from Accent Systems?).

Versions as of *PERQemu* v0.5.3 also support:

- PNX 2.0 (second release by ICL) for the PERQ-1/CIO.

PERQemu v0.5.8 also supports:

- PNX 2.0 and 3.0 (early access release media) for PERQ-2/EIO, 8" Micropolis disk.

Note that the available media set for PNX 3 is incomplete; only the T1 boot floppy works, so a hard disk installation is not yet possible. Patches to correct the video display list problems in ICL's non-standard microcode in PNX 2 and 3 are applied automatically.

With the *PERQemu* v0.7.*x* release, the following OSes also boot but are not yet widely available as complete hard disk images have not yet been successfully produced:

- POS version G.85 (unreleased, *incredibly* rare);
- PNX 5.02 and 5.03 (final ICL release for the PERQ-2/T2);
- FLEX (a once-thought-extinct research OS by the UK's RS&RE).

In the sections below, some basic information about each tested OS is given to help users new to the PERQ get started exploring the growing library of disk images, demos, games and applications.

POS

The basic PERQ Operating System, or POS, is a simple "no frills" single-process DOS-like OS written in Pascal. It provides the baseline development tools and programming environment for the machine. While relatively unsophisticated by modern standards, it does offer pretty much unrestrained access to the programmer: POS lets you get down to the bare metal and doesn't get in your way.

Availability

There are quite a few complete POS media distributions in the wild and most of them have been archived and made available for use with *PERQemu*. The version naming/numbering scheme is simple, with a letter representing the major release, and a number the minor release. In general, the last version of each major release was the most widely adopted; D.6, F.1 and G.6 were the releases from Three Rivers most likely to be encountered. POS F.15, G.7 and other variants are far less common.

As distributed, *PERQemu* includes a collection of hard disk images that are ready to run. Many are referenced by the predefined configurations, selected as the most representative POS vintage for the given hardware:

`d6.prqm`	The original *PERQemu* disk image archived by Josh Dersch. POS D.6 is the default boot; includes Accent S4 as '`z`' boot.
`f0.prqm`	A disk image from Skeezics' PERQ-1. POS F.0 is the default boot; MPOS is '`q`' boot (more on MPOS below).
`f1.prqm`	A fresh install of the basic POS F.1 system from floppy. Some additional applications and demos are included.
`f15.prqm`	An updated, expanded variant of POS F beamed in from an alternate reality. Details and source media available at the *PERQmedia* GitHub archive.
`g6mic.prqm`	A new install of POS G.6 for the PERQ-2 (8" Micropolis) with added games, demos, applications and source code! Supports portrait and landscape.
`g6mfm.prqm`	Same POS G.6 as `g6mic`, but for the PERQ-2/Tx (5.25" MFM).
`g7.prqm`	A new install of the unreleased POS G.7 with a few demos and applications. Also includes Accent S6 as '`z`' boot.

Unless called out as "`mic`" or "`mfm`", the hard disk images are the Shugart 24MB type, supported by the PERQ-1.[19]

[19] *As the library grows, the number of disk images bundled with the emulator may be reduced to slim down the basic package, moved instead to the* PERQmedia *archive where they can be downloaded separately.*

Configuration

Like most OS environments on the PERQ, the POS taxonomy is split into the "old Z80" and "new Z80" eras. In terms of hardware/emulator support, this means:

Old Z80 →	POS D[20] and F releases run only on the PERQ-1 with the IOB, one Shugart hard disk, and the portrait display
	POS D.6 and F.0 releases can only use the BitPad tablet; they don't support the Kriz
	POS F.1 and F.15 can support both tablet types
← New Z80	POS G requires a PERQ-1 with the CIO board or any PERQ-2 configuration
	POS G supports all hard disk types; on the PERQ-2/T*x* a second 5.25" hard drive can be provisioned
	POS G also supports both tablets *and* both display types!
	POS version G does *not* run on the 24-bit processor[21]

Table 7: POS hardware requirements.

Login and User Interface

All versions of POS indicate 999 on the DDS when they have successfully initialized. A banner is then printed that shows the available disk partitions, followed by a prompt for the date and time. This can be entered in the requested format, or you can just press `RETURN` to accept the default (which is the timestamp saved the last time the machine was shut down and the disk saved). For EIO PERQs, the on-board RTC chip may be read to automatically set the time for you!

POS then prompts for a username. In all of the disk images supplied with *PERQemu* the default "`guest`" account with a blank password is provided. Press `RETURN` at the login prompt to continue.

Interaction with POS is through a command-line interpreter called the Shell. Commands are not case sensitive (but are shown here in upper case as is the POS tradition); the filesystem is case preserving but is not case sensitive. Here are some basic commands to get you started:

[20] *POS D.5 has been recovered! No complete versions of POS prior to this are known to have survived, however.*

[21] *POS G.7 and G.85 have been recovered; they* might *run on the 24-bit CPU but not use its extended features. [To be confirmed.]*

`?`	List the commands defined in your "login profile"
`HELP`	Show usage for various commands
`DIR`	Show contents of the current directory. Executable files are suffixed with `.RUN` and directories are suffixed with `.DR`
`DIRTREE`	Display and navigate the filesystem hierarchy graphically
`PATH`	Change directory. Often aliased to `CD`, more or less like your Unix or DOS equivalent. Note that the directory delimiter is '>' (this may freak out Unix heads!) `PATH foo>bar>baz>` – change to directory `foo>bar>baz` `PATH ..` – change to parent directory `PATH sys:user>` – change to root of `user` partition
`SETSEARCH`	Add or remove directories from your search path
`TYPE`	Display a file on the screen, similar to "`more`" on modern OSes
`COMPILE`	Run the Pascal compiler. Produces a `.SEG` file which must be linked
`LINK`	Run the linker. Produces a `.RUN` file which may be executed
`COPY`	Copy a file from one location to another
`EDITOR`	A fairly simple line-oriented text editor with some mouse support
`PEPPER`	A more Emacs-like editor, available in several of the POS images

To run a program, just type its name (you can leave off the `.RUN`). Switches are prefaced by '/' as was common to many operating systems of the era. Commands and switches are not generally case-sensitive, and can usually be abbreviated as long as they are unambiguous. Most commands in POS will prompt you for arguments if you don't provide them, and most commands won't do anything harmful without asking you first.

In general, pressing `Ctrl-c` twice will abort a running program, unless it's hung.

- *Note*: Control characters on the PERQ are case sensitive! A single `Ctrl-c` is an interrupt; a second `Ctrl-c` signals an abort. A `Ctrl-Shift-C` is used to break out of command files and return you to the Shell. The Pepper editor makes extensive use of shifted control characters to make up for the lack of a "meta" key that Emacs requires. Quirky, no?

The Shell also provides a pop-up menu facility. At the prompt, you can click anywhere on the screen to bring up a menu of programs to run; the current selection is highlighted in reverse video. A small scroll bar at the bottom of the menu lets you move the menu up or down if there are more commands than will fit; the cursor changes to a small scroll icon as you hover over this area. Press and hold the mouse button and move left or right to scroll the menu. To make a selection, click once and the

highlighted command is executed by the Shell; click anywhere outside the menu to abort the selection. Note that switches or arguments cannot be provided when the menu is used.

POS doesn't impose much structure on the filesystem; it mostly lumps everything into one big `sys:boot>` directory and lets you sort it out however you like. (In later releases where Accent is bundled, they sometimes used `sys:pos>` instead.) By convention, there are usually games under `sys:user>games>`, demos under `sys:user>demo>`, and user directories are generally kept in `sys:user>`. Like DOS, there aren't really many filesystem protections; you can scribble files wherever you want! Things are arranged a little bit differently in POS F.15; "`type welcome.txt`" to learn more when you first log in.

Full POS documentation can be found on Bitsavers in PDF form; see Appendix B for links. The original text versions are also in `sys:user>doc>` in the F.1 hard disk image, or `:boot>docs>` in F.15; documentation included with POS G to be determined as those images are completed.

File Transfer via RSX:

The POS `COPY` command allows you to upload text files from the host into the running virtual machine, or copy files from the running PERQ to the host. This simple transfer method is built-in to nearly every version of POS and follows the same syntax:

`> copy` *POSfile* `~ RSX:` *hostFile* ← *from the PERQ to the host*
`> copy RSX:` *hostFile* `~` *POSfile* ← *from the host to the PERQ*

There are some notable limitations:

- POS always converts the file names to uppercase; this may not matter if your host uses a case-insensitive filesystem but may cause difficulties when uploading if you don't provide an upper-case filename;
- This is a simulated transfer over the serial port at 9600 baud, so it's not terribly fast! Error checking is minimal, and because it's an ASCII transfer you may have to check the resulting file for missing characters being stripped or line endings being changed.

Note that *PERQemu* does its best to automatically convert Unix-style line endings (LF only) to POS style (CRLF) when uploading, since otherwise the file transfer wouldn't work. When downloading, POS does not strip the carriage returns out so you may need to use an editor or command-line utility such as `dos2unix` to reformat the result.

File Transfer using RS-232

POS includes a *very* simple serial communications program called `CHATTER` that has limited file transfer capabilities. To use the serial port you must configure the host side (see the section *Working with Communication Devices* in chapter two) and enable it in your configuration. `CHATTER` should default to RS-232 port A on all machines, and can use port B on PERQ-2 configurations.

In addition, a slick, full-featured terminal emulator called `JET` exists, and a nifty-but-incomplete version of the popular `Kermit` utility is available as well. These are available in the floppy archive but will hopefully soon be included in the standard distribution as well. Neither has yet been tested extensively with *PERQemu*.

File Transfer using Floppies

The POS `FLOPPY` program is used to create and transfer files from DEC RT-11 formatted floppies. *PERQemu* supports double-sided and double-density operation, with the most typical format storing around 500KB (double-sided, single density, abbreviated "DSSD"). The program itself has extensive on-line help, and even provides pop-up menus for some commands. To create a new, blank floppy, refer to the section *Working with Storage Devices* in the previous chapter.

Once a floppy is loaded, you can prepare it for use using `FLOPPY`'s "`ZERO`" command. This simply recreates the directory structure on the floppy, erasing any files that already exist. You can also use the "`FORMAT`" command to completely erase the media first, but that's not strictly necessary given that *PERQemu's* `storage create` command does that step for you – and *much* faster. Floppies created in RT-11 format can be used with external utilities to read and write files outside the emulator, providing a more convenient method for moving binaries or larger files than the `RSX`: method.

RT-11 Floppy Limitations

- For compatibility with RT-11, files are named using the limited Radix-50 character set; you can use upper case only with few special characters in a strict "6.3" naming scheme (think early PC DOS and you won't go far wrong);
- The date format used stores a very limited range and is aggressively *not* Y2K-compliant; when prompted to enter the date when storing files, you can only specify years from 1972 through 1999!
- Files are allocated and stored in contiguous sectors – if the disk becomes fragmented you might have to move files around to coalesce the free space to make room for a larger file. `FLOPPY` can do this with its "`COMPRESS`" command... *slowly*.

POS "filesystem floppies" use a different approach altogether. These have to be created using the `PARTITION` program, which creates a filesystem on a floppy that looks and acts just like a small, slow hard disk to POS. Filesystem floppies can be made bootable by running `MAKEBOOT`. They must be "mounted" for use (the "`MOUNT FLOPPY`" command), at which point you can use the normal POS file and path syntax for accessing them. You can even run the disk "`SCAVENGER`" program to repair them. Always dismount them with "`DISMOUNT FLOPPY`" when finished before saving or ejecting.

File Transfer using Tapes

POS versions F and G support a utility for backing up and restoring hard disks called `STUT`. This Streamer Utility operates at the partition level, allowing for bulk data transfer for backups (or OS installations) of up to 20MB per cartridge[22]. `STUT` may not be present in some of the bundled hard disk images; source and binary copies are available so it can be added where needed. `STUT` has its own inline help and is fairly simple to use.

Laser Printing

The POS utility `CPRINT` drives the Canon laser printers. The POS F.15 and G.6 hard disk images bundled with *PERQemu* now include this, and source code is available in the floppy archives. Updates to the other pre-built disk images will be included in a future release.

Logout and Shutdown

You can log out of POS by typing "`bye`" to the Shell. POS will flush any cached disk buffers, record the current time, then log you out. It returns to the Login program to allow you to start a new session. Or, at this point, you can safely shut down the emulator.

> The PERQ-1 hardware can do "soft" power! Type "`bye off`" and POS will tidy up as usual, then power down the machine. (This was *really* cool back in 1981.)
>
> *PERQemu* handles the power down signal the same as pressing the close button on the Display window. Depending on your `autosave` settings, you may be prompted to save any unsaved changes to your loaded disk(s) before the emulator closes up shop and returns to the CLI.

[22] *Officially. In* PERQemu *you can create a 45- or 60MB DC-600A and Stut seems to work with the larger tapes just fine!*

MPOS

A rare and unreleased POS version E.29 was recovered. Known as MPOS, or the "multi-process" version of POS, it features a more complete window manager and can run multiple processes using a mostly-compatible POS API. We even have source code for this funky thing! Have to play with it some more to really get the hang of how it do what it do.

Availability

In a future release a clean install from scratch onto a freshly formatted hard drive will be bundled (and included in the *PERQmedia* GitHub repo). In the interim, a working copy of MPOS E.29 is provided with a POS F.0 (dump of my own PERQ's hard disk):

`f0.prqm`	MPOS is '`q`' boot!
`mpos.prqm`	A full source install of the rare MPOS E.29 release. *[TBD]*

Configuration

MPOS supports the same hardware as POS F; the default configuration works. It can use 2MB of memory. Kriz tablet, serial ports, and floppy access have not been tested.

Login and User Interface

Most interactions with MPOS are the same as POS D/F, so only the differences need to be highlighted here. The window manager needs some introduction.

Logout and Shutdown

The "`bye`" command offers most of the same options as POS. You can type `bye off` to log out and power down the PERQ.

Todo: Highlight those differences
Clean install, new media, uploaded to Github and bundled!
Window manager!
Device interaction? Do it do the RSX thang?
See if the recently found Ethernet code works.

Accent

Accent is a message-based, multi-tasking OS microkernel developed in the early 1980s by Carnegie-Mellon University as part of the *Scientific Personal Integrated Computing Environment* (SPICE) Project. Successor to RIG and precursor of Mach, Accent on the PERQ refers to both its microkernel and the overall operating system environment, consisting of a set of servers designed to be distributed over a network. *PERQemu* can run several versions of this interesting and groundbreaking operating system. Please see Appendix B, *Bibliography*, for links to a wealth of documentation.

Availability

An early release of Accent S4 from CMU is included in the bundled `d6.prqm` disk image. This is an amazing and rare find, as it pre-dates the official releases from Three Rivers/PERQ Systems in late 1984-early 1985. Accent S4 is '`z`' boot on this image.

An Accent S5 system, including SPICE Lisp version M2, is being prepared. It should be available and documented here in a future *PERQemu* release.

A fresh installation of Accent S6, paired with POS G.7 and the Accent demo, is now available and included in the standard distribution as `Disks/g7.prqm`. A version of this disk with Spice Lisp M3 is now included as `Disks/s6lisp.prqm` (with a predefined configuration file for convenience in `Conf/s6lisp.cfg`). Accent S6 is '`z`' boot on both of these images.

Accent S7, the final unreleased version known to exist, runs on the PERQ-1 as of v0.4.8; in v0.7.x it also runs on the PERQ-2, and will support the 24-bit T4! Disk images will be forthcoming *RSN*.

Configuration

All versions of Accent run best with more memory configured. On the 20-bit CPUs, that means a maximum of 2MB with a practical minimum of 1MB (though older versions will boot with less, if you insist). On a PERQ-1 24MB Shugart disk with limited swap space, more RAM is better, *especially* if you want to try out the landscape display. With each new release the OS is faster and more reliable, and hardware support is improved.

The S4 version of Accent only runs on the PERQ-1/IOB/Shugart configuration. It requires the BitPad (doesn't support the Kriz tablet) and portrait display, but it *can* run with the full 2MB of memory. While S4 *may?* support the 4K (PERQ-1) CPU, you should generally configure the 16K (PERQ-1A) CPU. Load the `AccentV4` Q-code set for accurate Q-code disassembly.

A sample dialog for loading and running Accent S4:

```
> configure default
> configure assign d6
> configure memory 2
> debug load qcodes accentv4      ← optional, for debugging
> bootchar z
```

Accent S6 runs on the 16K CPU, CIO or EIO, and all disk types. Performance is best with 2MB memory configured. It supports both tablet and display types! Selecting the Kriz tablet gives a slightly more responsive interaction with the window manager, as its binary protocol is more efficient than the ASCII data stream sent by the BitPad. To configure:

```
> configure load perq1a
> configure assign g7
> configure memory 2
> configure tablet kriz
> debug load qcodes accentv5      ← optional, for debugging
> bootchar z
```

Login and User Interface

Accent's startup is very different from POS. After the usual boot sequence, the DDS will stop after the kernel determines the amount of memory configured (400 for 1MB, 450 for 2MB). The screen is then cleared and a banner printed announcing the OS version and several startup messages. Accent will first prompt you to enter the date and time:

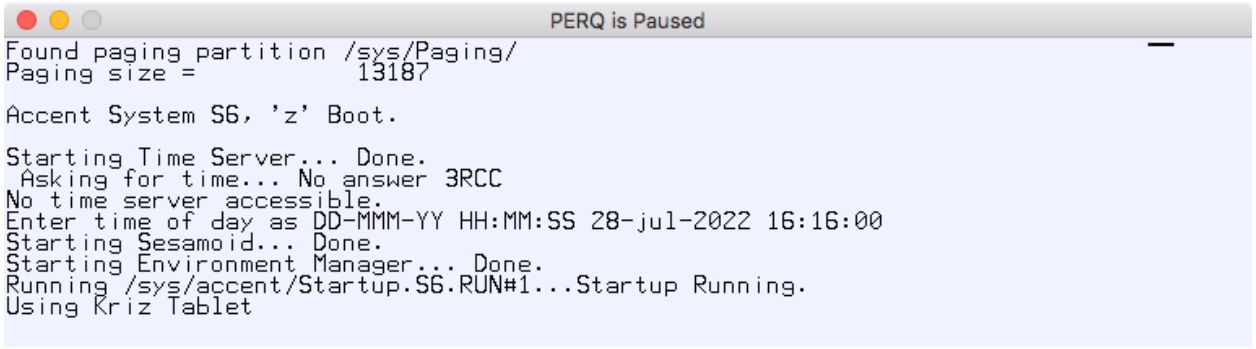

Figure 4: Accent startup screen.

Note that you can enter a 4-digit year; the time routines are Y2K compliant! No RTC offset changes are required. *However*, you should type carefully when entering date and time, and *only* use the backspace or OOPS key if you make a mistake. One less-than-endearing "quirk" is that a badly formatted date string will often crash the time server and force you to reboot.[23]

The interface to Accent is provided by the *Screen Allocation Package Providing Helpful Icons and Rectangular Environments* (SAPPHIRE), an awesome 8-letter acronym and window manager that unifies the native POS-like shell environment, a Common Lisp environment, and a Unix-compatibility layer called Qnix.[24] As it initializes, SAPPHIRE clears the screen to a stippled gray background pattern and draws several windows as it runs through a series of command files to start up the rest of the environment. Once everything is initialized Accent's Shell behaves very much like the POS Shell, with many of the same commands and options. Unlike POS, Accent provides command-line editing, scrollback, multi-tasking and "real" window management (uncovered windows redraw themselves).

Some hints for getting started with Accent:

- SAPPHIRE tracks the mouse in relative mode; the Alt/Option key is your friend!
- Moving or resizing windows requires a modal type of interaction that may feel a bit clumsy at first to those used to the "direct manipulation" paradigm common today;
- You have to click to select a "listener" window to direct where your typing will go – the listener window has a gray border around it;
- You can use the basic Emacs-like control keys to retrieve previously typed commands and edit them (`^p`/`^n` for previous/next, `^f`/`^b` forward/back, etc.);
- You can scroll back to see text that scrolls off the screen (`^V`/`^v` for backward/forward by roughly a page full of text); as with POS, control characters are case sensitive!
- Interrupting or aborting the current program goes through the window manager, so `^c` or `^C` doesn't work as in POS.

Window manager commands may be invoked by mouse click or the keyboard. Keyboard commands are prefixed by the `SETUP` key (`Ctrl-Del` on PERQ-1), then a letter. For example, "`Ctrl-Del h`" brings up a command summary. There are also pop-up menus, and the icons change depending on what SAPPHIRE wants you to do (make a window, move a window, etc). To interrupt a program, use "`Ctrl-Del c`", or "`Ctrl-Del k`" for added *oomph*.

---

[23] *Most Accent machines were connected to Ethernet networks where a time server would provide the time, eliminating this manual step. It does not appear that Accent will read the current date/time from the RTC on PERQ-2/EIO boards, unfortunately.*

[24] *Work is ongoing to make these additional Accent environments available.*

The icons in S6 are the set provided by Three Rivers/PERQ Systems, and may not match some of the CMU documentation, but their meanings should be intuitive. Like the POS D.6 installation on the "`d6`" disk image, someone replaced the default S4 cursor with a silly Opus icon; clearly a *Bloom County* fan used that machine once. In early Accent, the default cursor was an image of a sapphire ring.

Using the Accent Shell

One important change from S4 to S6 is the transition from POS-style syntax to a much more Unix-like approach. In S6 the form changes, but the underlying structure of the filesystem remains compatible:

S4: *dev*:*partition*>*dir*>*file*
S6: /*dev*/*partition*/*dir*/*file*

In addition, S6 uses the single hyphen ('`-`') to introduce command switches:

```
S4:   dir /fast /sort=size
S6:   dir -fast -sort=size
```

Type "`help`" at the shell prompt for information about Accent's command syntax. Please also refer to Appendix B, *Bibliography* for links to more Accent documentation.

Laser Printing

Accent also uses the `CPrint` software, available on floppy; it is not currently bundled.

Logout and Shutdown

There is no "clean" way to logout of Accent S4, so the best way to make sure you flush buffers to disk (if you want to save your work) is to type "`trap`" at the Shell. This will bring up the kernel debugger which will scribble in reverse video over the top of the screen. From there type '`y`' to exit to the pager process, and at that point there's no going back; switch to the *PERQemu* command line and type `stop` to pause execution, save disks, then `power off` or `reset` as desired.

In Accent S6 a "`bye`" command is provided. If you plan to save your work, it's important to run this before you save the hard disk; Accent S6 is much more aggressive with caching disk blocks in memory to improve performance, and the `bye` command makes sure modified data is flushed before shutdown. It also prints a message to tell you when it's safe to turn off or reboot the PERQ.

Limitations

Accent S4 gets *very* cranky if you try to interact with the window manager before it finishes initializing. When booting S4, let it complete *all* of its command files before switching windows or trying to start up additional shells or programs. S6 is a bit more robust, but the same advice applies.

Accent does not support the `RSX:` serial pseudo-device.

Accent S6 *does* support the streamer utility, `STUT`. More testing is needed to document its use and interoperability with POS.

Interim Ethernet Support

Designed as a distributed, networked OS, all versions of Accent require an Ethernet interface to be present. A "fake" Ethernet driver (added in v0.4.9) allows the Accent Net/Message Server to properly initialize. This means that S6 can launch its `FloppyServer` and `RS232AServer` processes, making it possible to load data from floppies and access the serial port.

All PERQ-2 configurations use the EIO board and contain a built-in Ethernet interface. To run Accent on a PERQ-1, add the following commands to your configuration:

```
> configure option board oio
> configure option add ether
```

If these files exist:

```
/sys/accent/10MhzMsgServer.S6.DIS
/sys/accent/10MhzNetServer.S6.DIS
```

they should be renamed to have the `.RUN` extension (and you'll need to `reset` the PERQ to launch them). The `g7.prqm` disk image bundled with v0.4.9 has been corrected so that the Net/Message Server starts up at boot time. The older S4 installation included on the `d6.prqm` disk image has also been updated in v0.7.5 to launch its Net/Message Servers, though it is still a bit "touchy." (Hopefully improved Ethernet support over time will enhance the Accent experience across all versions.)

Live Ethernet Support

When Ethernet support is configured and enabled, Accent can communicate with other emulated PERQs to do remote filesystem *and device* access, authentication, time service, remote execution, a simple messaging/"chat" service, FTP, and more. Later versions of S6 and S7 even incorporate early

TCP/IPv4 functionality; experimental features like network booting, live process migration between nodes, and even distributed demo programs wait to be explored.

As discussed in the earlier *Working with Communications Devices* section, even a relatively quiet switched Ethernet connection on a modern host generates a constant stream of traffic that easily overwhelms the PERQ's 10Mbit, half-duplex interface. To help alleviate this and give Accent a fighting chance to operate on busy networks, *PERQemu* provides filtering at the host interface to throw away IPv6, spanning tree, or other types of packets that the PERQ can't do anything with. It is hoped that a platform-independent way of using Win/NPcap filters to limit the traffic flow at the host interface can be devised, but the current advice is to assign a private host interface to the emulator and hook up virtual PERQs on a small hub or switch with *no* other hosts attached. Many Macs and PCs built in the last 10 years have dual Ethernet jacks; *obviously* those extra, unused interfaces are meant to be used by *PERQemu*. If your OS permits, leave TCP/IP turned off and unconfigured to minimize the amount of background traffic on the interface.

In summary: There are tantalizing hints at what Accent can do once the emulator's Ethernet interface is more fully debugged and functional!

Todo: Get S5 and Lisp M2 booting! See if MatchMaker will run too
Install the C environment for S6, and Qnix/Spoonix if available
We might have an *S3* image!? And S7!
24-bit mode is finally working; document it
Ethernet, Ethernet, Ethernet! (Progress *is* being made)
Holy crap, the Interactive Debugger runs on Accent! Centipede too!

PNX

PNX, another unfortunately named PERQ operating system, is an early Unix V7/System III mashup that includes an in-kernel window manager – possibly one of the earliest Unix variants to do so? This was primarily run on PERQs in the UK, where PNX was developed by ICL. Because the on-disk format is based on the Unix filesystem, it cannot co-reside on a disk with POS or Accent (which share a common underlying filesystem layout).

PNX also uses its own language interpreter, running an instruction set more favorable to C than the original PERQ Q-codes. However, *PERQemu's* debugger cannot yet disassemble PNX C-codes accurately, since we don't currently have access to any PNX source code or much documentation. A set pulled from the `/bin/as` binary provides an approximation that may help when debugging; use "`debug load qcodes pnxccodes`" to load them.

Availability

A Shugart 24MB disk image of PNX 1 from the *PERQmedia* Github repository has been converted to the new (compact) PRQM disk format and bundled with *PERQemu* for v0.5.0; in v0.7.5 PNX 2 has been bundled as well:

`pnx1.prqm`	A full installation of PNX 1.3
`pnx2.prqm`	A mostly-complete installation of PNX 2.0 (PERQ-1/CIO)
`pnx2mic.prqm`	An installation of PNX 2.0 for PERQ-2/EIO (8" Micropolis)

As with Accent, PNX became more capable and complete with each release, so there is great interest in bringing the new versions up on the emulator. Version 3 still supports the PERQ-1, while version 5 seems to run only on the EIO/PERQ-2 models, but more research is needed to understand their hardware requirements. A copy of PNX 4 has been found, but needs to be recovered from 8" floppies.

Configuration

The early 1.3 release of PNX has the same restrictions as early POS:

- Requires the old Z80 configuration: PERQ-1, IOB, Shugart hard drive, BitPad tablet, portrait display;
- It supports a maximum of 1MB of memory. If you configure 2MB, PNX will fail to initialize and will drop into its debugger, display an '@' prompt and only respond to a few cryptic (and undocumented) commands.

PNX 2.0 now boots on the emulator as well, with a slightly different set of constraints:

- Requires the new Z80: PERQ-1/CIO/Shugart, or PERQ-2/EIO/Micropolis;
- Portrait display only?
- Kriz tablet support available, but requires manually updating `/dev/tablet`;
- The full 1 megaword (2MB) memory option is supported;
- This version has a number of improvements over the 1.x release but is missing some components (manual pages, notably).

There is currently no extra software for these early versions of PNX besides the very basic install images. However, PNX 5 does include some demos/apps, *and* there is some exciting news about a possible software archive unearthed in the UK; watch this space!

Login and User Interface

PNX 1.3 clears the screen once the DDS reaches 255 and prints a banner to announce itself:

```
mem = 844288   M=1.4 B=1.1 K=1.3

Microcode Version 1.4
Bootstrap Version 1.1
Kernel Version    1.3

single user
login: <^Z>
```

At this point you may enter single-user mode by entering a login and password (default user `root`, default password `root`), or you can press ^Z to switch to multi-user mode. PNX 2 skips single user mode and directly proceeds to run `fsck`. Next, both versions prompt you to enter the date and time:

```
type date in form : 4 nov 5:20 or nov 4/5:20 or 11040520 etc.
use two digit numbers (or field separators) in day, hours, mins order

set the date [dd mmm hh mm] : 25 may 1:06
Thu May 25 01:06:00 GMT 1983

are date and time correct ?  y
```

Note that *it's always 1983 in PNX land!* In PNX 2, it is (ominously) always 1984.

Once entered and confirmed, the screen is cleared again and the normal multi-user prompt is displayed:

```
mem = 844288   M=1.4 B=1.1 K=1.3

International Computers Limited
PNX Operating System
Release 1


login: root
password: root                ← the password is not displayed
```

Once logged in you are presented with the familiar '#' prompt (as the root user). To start the PNX window manager, type "`winit`". The figure below shows a basic display:

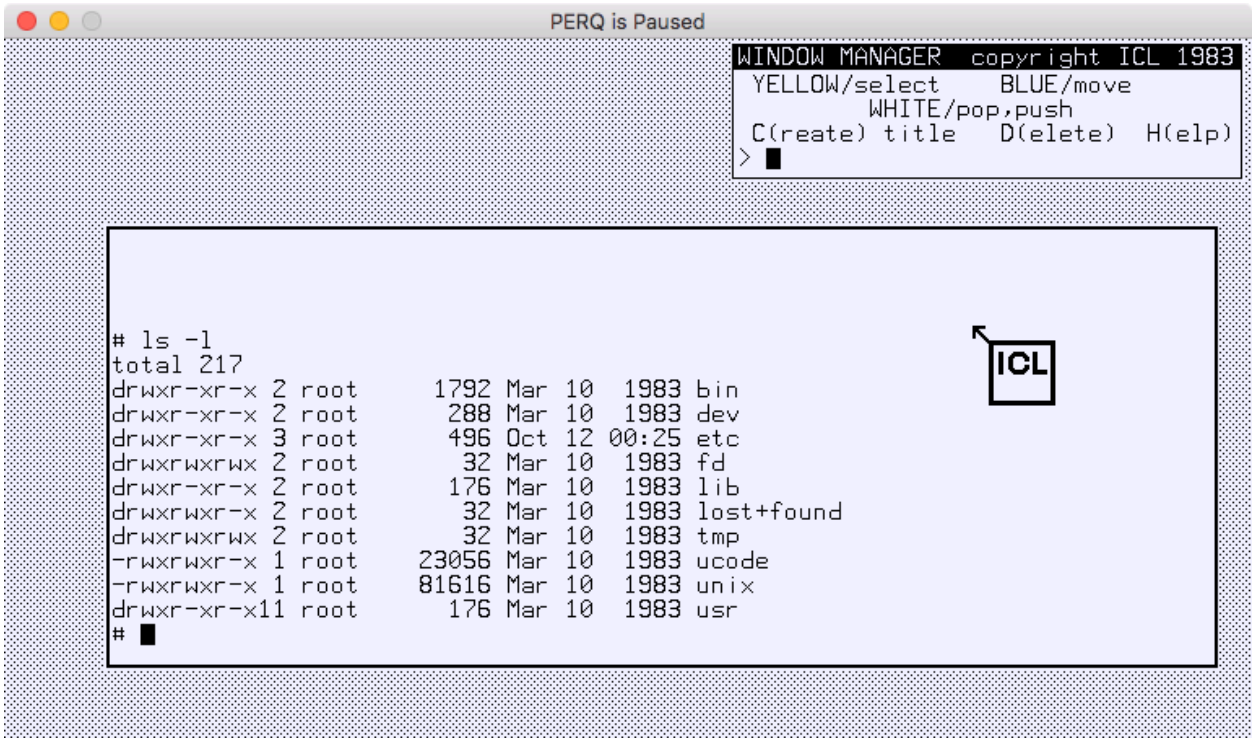

Figure 5: PNX 1.3 graphical environment.

Note that your typing is edited in a small text area at the bottom of the screen! When you press RETURN the input line is sent to the selected window (and echoed there). Use the middle button on your mouse to select the active window; its title bar is then highlighted in bold.

You may also find that output to the window pauses when it's about to scroll off the screen; there seems to be an implicit (unprompted) "more" mode that waits for you to press the space bar to continue.

❖ *Author's note:* I haven't explored this version much, and have only a rudimentary understanding of the window manager's operation. Some basic hints are provided here, but hopefully someone with more PNX 1 experience will chime in with additional pointers to documentation or tutorials!

Logout and Shutdown

PNX 1 doesn't provide a standard Unix `halt` or `shutdown` command. You can close a window by pressing ^Z at the prompt; the window manager will shut down when the last window is closed, returning you to the basic blank, non-windowed screen. From there, "`kill -1 1`" will signal the `init` process to drop you back to single user mode. Then you can "`sync; sync`", wait a few seconds, and stop the emulator.

In PNX 2 (and beyond) there's a "`bye`" command that works sort of like its POS equivalent. Note that it doesn't play around; type it (as `root`) and you'll soon see "PNX CLOSED DOWN" and the virtual machine must be reset or powered off.

Todo: Play with the window manager more and document start-up, interaction
Compile Mario Wolczko's Smalltalk for PNX (has been verified on the hardware!)
Find a definitive listing of the C-codes; update the cobbled together set
PNX 3 T1/EIO boot floppy works! The rest are all missing trk0 and must be reimaged :-(
PNX 4 set also located – boots from floppy, but same install issue as PNX 3
PNX 5 supports T2/T4, EIO/landscape/dual drives/Multibus(!) but **refuses** to boot
Didn't anyone save a copy of PNX-SR for PERQ-1? :-(
Applications! Investigate the few floppies that may have SERC utilities

FLEX

PERQ FLEX is an OS produced in the UK by the *Royal Signals & Radar Establishment* in the mid-1980s. Based on a custom instruction set optimized for Algol-68, a large set of floppy disk images has been archived and converted for use with *PERQemu*. The OS itself has successfully been bootstrapped, and additional testing is underway on both hardware and emulation. Efforts to recover the contents of several hard drives are continuing, and we hope to be able to include *PERQemu*-compatible media with the emulator or in the usual archives in a future release.

Availability

We are working to clarify with the Crown Copyright service whether the recovered images may be shared publicly. FLEX is a unique bit of computing history and this is a rare opportunity to explore another operating system on the PERQ. Watch this space.

Configuration

PERQ FLEX lives up to its name by being extremely flexible in its installation options. It can run standalone on the "bare metal," *or* be installed within a POS or PNX filesystem! Initial testing of the standalone installation required two small patches to *PERQemu*, so FLEX will not run on versions prior to v0.7.8. The OS itself is available for PERQ-2 configurations (T1/8" or T2/5.25" disks).

Note that FLEX requires that the date returned by the RTC chip not exceed some (as yet unknown) upper limit. Use the Configuration (or Settings) "`rtc offset`" override to make sure the year returned by the clock chip falls within the mid- to late-1980s if FLEX appears to hang with a plain white screen; for example, in the current year (2026) setting the offset to 2021 tells FLEX it's 1985, which works.

The FLEX graphical interface also makes extensive use of the PERQ-2 keyboard's keypad, which includes a comma key – uncommon on modern keyboards. This provided the impetus to dive into the keymapping problem. [*When released*] FLEX users will likely need to take advantage of this to adapt their host to this unique GUI environment.

HCR Unix

An early attempt to bring Unix to the PERQ, a set of HCR Unity floppies has been recovered. It is believed that this was a port of Xenix, but it was abandoned and never shipped as a product. There are no installation instructions and it's not clear whether the set is complete. Chances are slim to none that we'll ever get this running on the emulator, but it is an interesting historical find.

Of course, if this operating system is able to be installed and run under *PERQemu* this document will be updated accordingly!

More to come...

Demos and games!!
Applications
Dev tools and languages

4. Command Reference

Below is an itemized description of the command syntax for *PERQemu* v0.9.0. Command arguments are illustrated using the following typographic conventions:

argument	a required argument supplied by the user
(`this` | `that`)	a required argument from a list of specific choices
[*argument*]	an optional, user-supplied argument
[`word`...]	an optional word (or words) that extend or modify the command
[...]	a subsystem prefix; may be typed by itself or with additional arguments

Type "`commands`" at any prompt to see a summary for the current subsystem. Type "`done`" to return to the previous level in the hierarchy.

Top-level Commands

about [libraries]

Print the program's startup banner with version, copyright and contact information. If the libraries argument is given, print the version numbers of the supplemental packages used by *PERQemu*.

bootchar [*char*]

With no argument, show the current boot character (if set). With a character argument, set the desired boot character to *char* (case is significant). If unset, the PERQ defaults to '`a`' boot.

configure [...]

Prefix for configuration commands. Typed by itself, enters the Configuration subsystem.

debug [...]

Prefix for debugging commands. Typed by itself, enters the Debugging subsystem.

go

A shortcut for the sequence "`power on`" followed by "`start`" (or just `start` if already running).

help [*keyword*]

Show the (currently limited) on-line help. With no *keyword*, prints a general help message. Still a work-in-progress and likely to change in the next release of *PERQemu*.

history

Show the last 99 commands entered.

keymap [...]

Prefix for keyboard mapping commands. Typed by itself, enters the key mapping subsystem.

load (floppy | harddisk | tape) *filename* [*unit*]

Load the supplied *filename* as the current floppy, hard disk or streamer tape using the default unit numbers. If the emulator is running, removable media is brought online immediately. Updates the current configuration. A unit number may be specified when loading a hard disk in a PERQ-2/Tx system. See also: *Working with Storage Devices* in chapter two.

load paper [*size*]

If the Canon laser printer option is configured, reload the paper cassette. If *size* is not specified, the current cassette is refilled, otherwise 100 crisp new sheets of the specified type are loaded. If `NoCassette` is chosen, the paper tray is immediately *removed*, potentially interrupting any printing currently in progress. See also: *Working with Output Devices* in chapter two.

power (on | off)

Use "`power on`" to validate the current configuration, load the assigned media and start the emulator if it isn't already running. Type "`power off`" to shut down the emulator, optionally giving the opportunity to save modified media (according to your `autosave` preference settings).

quit [without save]

Exit *PERQemu*. If the emulation is running, a `power off` is done implicitly. By itself "`quit`" offers the chance to save any modified media (if configured to do so by Settings) and updates any unsaved Settings changes. If "`without save`" is specified, *PERQemu* shuts down and exits immediately; unsaved changes are lost.

reset

If the emulator is running, "`reset`" simulates the press of the PERQ's boot button to trigger a hardware reset of the virtual machine. The PERQ will then restart execution or pause, depending on user preference. See also: Settings "`pause on reset`" option.

save (floppy | harddisk | tape) [as *filename* [*format*]]

Commit changes to a modified media image using the same format and filename it was loaded from. Optionally "`as` *filename*" creates a copy of a loaded image and updates the current configuration with the new filename. The copy is created in the same format as the original image, or can be transcribed into *format* if specified. See also: *Working with Storage Devices* in chapter two.

save history

Not yet implemented.

save screenshot [*filename*]

Take an immediate screenshot of the current display, optionally saving it as *filename*. If no name is given, an obvious name is generated and the file saved to the default output directory. The screen is captured at its native resolution (100 dpi, 1 bpp). You can have any output format you like, so long as it's PNG. See also: Settings "`default output directory`" option.

settings [...]

Prefix for user preference commands. Typed by itself, enters the Settings subsystem.

start

Start or restart emulation when paused. The machine must already be powered on.

status

Print a summary of the emulator's current state. This is currently a rough debugging command and its output will be enhanced or modified in a future release.

stop

Pause emulation when running. Pressing ^C in the console window or function key F8 when the Display window has focus will also pause the emulator.

storage [...]

Prefix for storage commands. Typed by itself, enters the Storage subsystem.

unload floppy

If the emulator is running, this commands the PERQ to unload any loaded floppy media (giving the opportunity to save changes, if modified). Also updates the current configuration.

unload harddisk [*unit*]

Remove a hard disk from the current configuration. If *unit* is not specified, unload the default unit #1. Fixed hard disks cannot be unloaded when the emulator is running; hard disks with removable "packs" are not yet supported. Updates the current configuration.

unload tape

If the QIC Tape option is configured and loaded, this commands the PERQ to unload the cartridge. The default (only) unit #3 is selected. Updates the current configuration.

Settings Commands

The "`settings`" prefix allows you to tailor *PERQemu* to your system and to your preferences. At first release, a subset of the configurable settings are supported. *PERQemu* will always update your saved preferences with the current syntax as updated versions are installed.

assign audio device *hostDevice*

Map a host audio driver *hostDevice* to the PERQ's emulated "speech" output device.

assign audio option *opt val*

Fine tune the speech CVSD-to-PCM conversion parameters. Except for the "`channels`" option, you probably shouldn't mess with these. *[This interface is still evolving and will likely change.]*

assign ethernet device *hostDevice*

Map a host Ethernet interface to the PERQ's emulated network device. For Ethernet configuration options refer to chapter two, *Working with Communication Devices*.

assign (rs232a | rs232b) device *hostDevice* [*baud*] [*bits*] [*parity*] [*stopBits*]

Map a host serial interface *hostDevice* or the pseudo-device `RSX:` to a PERQ RS-232 port ('`a`' or '`b`'). You can optionally specify the baud rate, bits per character, parity and stop bits for the host device; defaults are 9600, 8, None and One. See also: chapter two, *Working with Communication Devices*.

assign (rs232a | rs232b) option *flag*

Set or reset optional *flag* for the selected RS-232 port. In v0.9.0 this includes serial handshake ("flow control") method or DCD pin settings.

autosave (floppy | harddisk | tape) (yes | no | maybe)

Set the preferred action to take when modified media is unloaded. See also: chapter two, *Working with Storage Devices*.

canon output format (raw | png | tiff | jpeg)

Set the preferred format for output from the Canon laser printer. These files are written to the default output directory at the printer's resolution (see below) and centered on the chosen paper size. PNG (single page) and TIFF (multi-page) are fully implemented; JPEG output is not yet available.
[`raw` format is a dump of the bitmap as written by the microcode for debugging and may be withdrawn.]

canon paper size (nocassette | a4 | b5 | uslegal | usletter)

Set the default size of the paper cassette for the Canon printer. Choosing `NoCassette` effectively takes the printer offline; this mechanism may be used to enable simulating "manual feed" in a future enhancement. While these are the four sizes of input tray listed and supported by the Canon hardware, the PERQ software only currently supports `USLetter` (8.5" x 11" / 216mm x 279mm).

canon resolution (240 | 300)

Set the output resolution for the Canon laser printer in dots per inch. The LBP-10 is 240 dpi; the LBP-CX is 300dpi. No other resolutions are allowed.

default

Reset all program settings to defaults.

display cursor (crosshairs | defaultarrow | none)

Change the system cursor when in the PERQ Display window.

keymap (*mapName* | none)

Apply the custom keyboard map *mapName* to all configurations. To unset this variable once set, use the reserved name `none`. May be overridden on a case-by-case basis by the "`configure keymap`" command.

load

Reload saved settings, discarding any unsaved changes.

output directory *dir*

Set directory for saving debugging logs, printer output and screenshots. *[Currently incomplete; does not yet require that* dir *exists nor will it create it for you, except when it does (when file logging is enabled).]*

pause on reset (true | false)

If true, the emulator is paused after a `reset` command is issued.

pause when minimized (true | false)

If running when the PERQ Display window is minimized, the emulation is paused and restarted when the window is restored.

rate limit *option*

Set performance *option* flags. If set to "`none`" *PERQemu* does **not** regulate the speed of the emulated CPU, peripherals or video and runs as fast as your computer can manage. If set to "`default`" the emulator strives to exactly match the 5.88Mhz rate of the processor and the 60fps display rate, with the Z80 running at the speed defined by the selected I/O board. The *option* flags are independent, so you may want to accurately emulate the CPU/video but speed up some peripheral accesses, for example.

rtc offset *year*

Change the base year offset used by the EIO RTC chip to *year*. The default is 1980. Affects all PERQ-2/EIO configurations, unless overridden by the Configure command for a specific virtual machine. See also: chapter two, *Setting User Preferences* or chapter three, *PERQ Operations*, for more information about how each OS handles dates and the clock chip.

save

Save current settings.

show

Show all current settings.

show audio devices

Show available host audio drivers.

show ethernet devices

Show available host Ethernet adapters.

unassign audio device

Unmap the assigned host audio driver; use the default device instead.

unassign ethernet device

Unmap the host Ethernet adapter (equivalent to choosing the "`null`" device).

unassign (rs232a | rs232b) device

Unmap the host device from the specified PERQ serial port.

Configuration Commands

From the top level menu, "`configure`" enters the Configuration subsystem. See *The Configurator* in chapter two for information about creating, saving and loading configurations.

assign *filename* [*unit*]

In configuration mode, *filename* is attached to the appropriate unit number based on the type of the device contained in the file. Optionally the *unit* number may be specified, useful to disambiguate which hard drive to assign in dual-drive PERQ-2/Tx configurations.

chassis *type*

Select the basic PERQ model from a list of supported *types*.

check

Validate the current configuration to make sure the machine is a valid PERQ that can be emulated. *PERQemu* will print warnings about (but allow) rare or experimental selections that may not have adequate software support, or indicate errors where configuration options are in conflict or are unsupported and will fail to boot.

cpu *type*

Select the processor to emulate from the list of supported *types*.

default

Reset the current configuration to the built-in default.

description *string*

Set an optional one-line textual description of the current configuration. The parameter *string* must be surrounded in quotes ("") if it contains spaces.

disable (rs232a | rs232b | speech)

Disable use of the specified device for this configuration. Takes immediate effect if running.

display (portrait | landscape)

Select the format of the PERQ's display. See also: the chapter *PERQ Operations* for restrictions on operating system support for the landscape display.

enable (rs232a | rs232b)

Enable the use of a host RS-232 device. In v0.9.0 and beyond, this configuration change is applied immediately if the virtual machine is running. See also: Settings "`assign rs232`" commands; the section *Working with Communication Devices* describes how the emulator supports serial ports.

enable speech

Enable audio output, which is enabled by default for all configurations. Takes effect immediately if audio is disabled and the virtual machine is running.

ethernet address *address*

Set the low word (last two octets) or "node number" of the PERQ's Ethernet address. May be any 16-bit value, but (by convention, and not enforced initially) the value should be in the range 1..65534, reserving the all-zeros and all-ones addresses. If set to zero, *PERQemu* will generate a random address. *Note*: on the PERQ-2, this value is also the machine's serial number which may have implications for some "node locked" software packages.

io board *type*

Select an I/O board for the PERQ. The *type* specified must be compatible with the chassis. See the section *The Configurator* for details.

keymap *mapName*
keymap (default | none)

Assign a custom keyboard map for this configuration. *mapName* is the "tag" for a previously defined keymap; setting this overrides any default maps set with the Settings subsystem. Use the reserved name "`default`" to remove an assigned *mapName*, or use the reserved name "`none`" so that only the default keyboard map for the machine type will be used. See also: Appendix D, *Keyboard Maps* for more information.

list

Print a list of the configurations available in the `Conf` directory.

load *config*

Load a configuration named *config*. *PERQemu* presents a selection of named configurations from a scan of the `Conf` directory at startup. You can also type a full pathname to load a *config* from another location.

memory *capacity*

Configure the PERQ's memory capacity in kilobytes (256, 512, 1024, etc.) or in megabytes (1, 2, 4, 8). *PERQemu* will round up the given argument to the nearest power of two, which must be valid for the configured CPU type. If *capacity* is not given, a helpful usage message is printed.

name *string*

Name the current configuration. The *string* specified should be a short, unique designation suitable for use as a filename. *PERQemu* will prepend `Conf/` and add the "`.cfg`" extension automatically. See also: the `description` command above; the section *The Configurator* for details of how *PERQemu* saves and loads configuration files.

option board *type*

Select an optional I/O board for the PERQ. If *type* is "`none`" the I/O Option slot is left empty.

option add *opt*
option remove *opt*

For some I/O Option boards, specific peripheral controllers may be independently enabled or disabled. Currently *PERQemu* supports a subset of options. See also: chapter two, *Configuring Sub-options*.

rtc offset *year*

Change the base year offset used by the EIO RTC chip to *year*. The default is 1980. Only affects the current configuration; overrides the Settings RTC offset if that is set too.

save

Save the current configuration if it has been modified. It must have an assigned name; see the `name` command, above.

show [*config*]

With no arguments, print a formatted description of the current configuration. If specified, *config* may be selected from the list of known configurations, or a filename; *PERQemu* will attempt to print the contents without disturbing the current configuration.

tablet (bitpad | kriz | both | none)

Select the type of digitizing tablet to attach to the PERQ. Choosing "`both`" attaches both the serial Kriz tablet and the GPIB BitPad to the PERQ, but the specific operating system loaded will decide (or

allow you to specify) which one it uses to track the mouse. You can also choose "`none`" but it isn't advised, unless greatly reduced functionality or waiting through endless I/O timeouts is your particular kink. See also: *The Configurator* section for more information about pointing devices; the chapter *PERQ Operations* for information about operating system support for the different tablet types.

unassign *filename*
unassign unit *unit*

Remove a media file from the configuration. The specified *filename* or the specific *unit* is unassigned.

Storage Commands

The "`storage`" subsystem provides commands that let you create blank media, define new types, or query the status of currently loaded storage devices. The default database of known PERQ media types is quite extensive, so some of the more esoteric commands are not (yet) documented. The most useful ones for end users are listed below.

For the "`import`" and "`export`" commands (new in v0.7.7) please see the section *Using Hardware Disk Emulators* in chapter two for examples of their use with external hardware and utilities.

create *driveType filename*

Create a new blank, formatted floppy, disk or tape image of *driveType*, selected from the given list of defined media types. A *filename* must be specified; if just a "simple" name is given, *PERQemu* automatically prepends the default `Disks` directory and adds the appropriate file extension. By default *PERQemu* will not clobber an existing file; see also: `safe`, `unsafe` to allow overwriting.

Note: This command always creates new media files using the PRQM formatter, in a not-so-subtle attempt to drive adoption of the new format.

define *driveClass driveType*

Define a new storage device in the given *driveClass* with the name *driveType*. This subsystem allows for the dynamic creation of new geometries, performance data, and media types at runtime. It is not intended for general use.

export unit *unit imgDataFile* [*metaDataFile*]
export disk *diskFile imgDataFile* [*metaDataFile*]

Export sector data from a *PERQemu* media file to a "`.img`" file. In the first form, the configured/loaded drive *#unit* is written to *imgDataFile*. In the second form, the disk image to be exported is loaded from *diskFile* instead. The `.img` extension is provided if not specified. For hard disk images, the PERQ logical headers are exported to "*imgDataFile*`.metadata`" if the optional argument *metaDataFile* isn't provided. Output is directed to the default `Output` directory if a full path is not specified. The resulting output files can be used by other utilities and hardware emulators for use on a real PERQ. See also: chapter two, *Working with Storage Devices* for more information.

import *diskFile imgDataFile* [*metaDataFile*]

Import the contents of a raw "`.img`" file for use with *PERQemu*. *diskFile* should be a pre-existing *PERQmedia* disk image created with "`storage create`" and not currently loaded. For a floppy or hard disk, *imgDataFile* is the basename of the raw image to import; the "`.img`" extension is appended if not provided. For hard disk import, *metaDataFile* contains the PERQ logical header data; the name "*imgDataFile*`.img.metadata`" is used if the optional *metaDataFile* is not provided. See also: chapter two, *Working with Storage Devices* for more information.

list

List known storage devices. *[Deprecated and will be removed or repurposed; please use* `show devices` *instead.]*

safe

Enable some basic protection from overwriting existing output files when the `create` or `export` commands are used. On by default when *PERQemu* is started. See also: `unsafe`.

show

Show information about the storage subsystem.

show aliases

Show alternate device names.

show devices

Show the names, classes and a brief description of the known storage devices.

show specifications *driveType*

Show detailed device specifications for *driveType*.

status [*unit*]

Show detailed status of a loaded storage device (when the machine is running). If *unit* is not specified, a summary of all currently attached drives is printed.

unsafe

Removes the overwrite protection for the `create` and `export` commands. Fire at will, commander!

Keyboard Mapping Commands

The "`keymap`" subsystem is used to create and manipulate custom keyboard mappings. It can be used to dynamically update the active, running virtual PERQ, so that changes can be made and tested instantly. As with the other subsystems in *PERQemu*, `commands` will show the available command summary and `done` will return to the top level of the CLI.

A great deal more information about keyboard mapping is provided in Appendix D. Too much, really. I've had complaints.

apply

If the PERQ is running and the loaded map is of the correct type, apply the changes immediately.

define *mapName*

Create a new keyboard map called *mapName* and initialize it with the default assignments. Switches to editing mode – see below.

edit [*mapName*]

Switch to editing the current map, if loaded. If *mapName* is given, it is loaded and unsaved changes to any current map are lost. See also: the `load` or `define` commands.

list

Show all currently defined keyboard maps.

load *mapName*

Select *mapName* as the current working map. Does not enter the editor. See also: `edit` or `apply`.

show [(running | summary | *mapName*)]

Display information about the current loaded keyboard map, or *mapName* if given. If the PERQ is powered on, `show running` displays the active map; `show summary` highlights changes.

save

Save the current keymap to disk. Keyboard maps are saved in the `Conf` directory with the extension "`.kbd`" by default. See also: Appendix D.

Keymap Editor

The keymap editor is a small subsystem-within-a-subsystem, where a newly defined or loaded map can be changed. It is currently command-line only. If the PERQ is running, keys pressed when the Display window has focus are displayed on the console as an aid to mapping the codes sent by the host.

The editor changes the prompt to "`keymap edit`" and adds the following additional commands. As usual, `commands` will show a help summary, and `done` will return to the Keymap subsystem.

apply

If the PERQ is running, apply the current changes immediately and remain in the editor.

default

Reset the current keyboard map to the built-in default.

description *string*

Set an optional one-line textual description of the current keymap. The parameter *string* must be surrounded in quotes ("") if it contains spaces.

map *hostKey perqKey*

Establish a mapping from the SDL2 keycode *hostKey* to the PERQ key *perqKey*. Both key codes are enumerated so that pressing TAB will show *all* of the available keycodes.

unmap *hostKey*

Remove a mapping, if any, for the SDL2 keycode *hostKey*. (Note: mapping *hostKey* to the *perqKey* "`None`" achieves the same result – pressing *hostKey* will not send anything unless remapped.)

save

Save the current keymap to disk, but remain in the editor.

show [(summary | host key *hostKey* | mapped key *perqKey*)]

Show the current map. If "`summary`" is specified, only list differences from the standard map. Show the mapping of a single key if "`host key` *hostKey*" or "`mapped key` *perqKey*" is specified.

Debugging Commands

The "`debug`" prefix provides a rich set of debugging commands. The Release build of *PERQemu* provides a subset of the total command set, mostly limited to interrogating the state of the emulator and the virtual machine. The Debug build of the program provides a much wider range of tools, including some that can modify the running state, but runs *much* more slowly.

For convenience, the Debugging subsystem provides the following execution commands:

```
go    reset      start      status     stop
```

These are the same as the top-level commands that are documented above.

All debugging commands that investigate the state of the active emulator require that the PERQ be powered on. As of this writing many of these commands are in a state of active development, or their use is fairly esoteric or directed at particular debugging scenarios. Most users won't need to explore these, so documentation is currently incomplete and their use may have unintended consequences. Careful with that axe, Eugene!

clear breakpoint *type watchpoint*

Clear a breakpoint of type *type* with the trigger *watchpoint*. *[Debug builds only]*

clear interrupt *irq*

Clear a specific CPU interrupt, if asserted; *irq* is selected from the list of available interrupts.

clear logging *category*

Clear logging for certain events; *category* is selected from the list of available logging categories.

disable breakpoints

Disable breakpoint processing. Defined breakpoints are retained but ignored until enabled. Note that breakpoints are disabled by default when the emulator starts. *[Debug builds only]*

disable console logging

Disable logging of debug messages to the console. Only normal command interactions are output.

disable file logging

Disable logging to files. *[Debug builds only]*

disassemble microcode [*address*] [*length*]

Produce a formatted disassembly of the current contents of the microstore. If no *address* argument is provided, output starts with the current microprogram counter. If no *length* is given, the default is 16 microinstructions.

dump *item*

Show internal state of a number of internal devices, states, queues, or other developer-specific information. Some of these commands *may* be available in Release builds, but all of them are subject to change or removal and should be considered experimental. *[Debug builds mostly]*

edit breakpoint *type watchpoint*

Change or reset a defined breakpoint. *type* is from the list of available types, and *watchpoint* is the value for the trigger (an address, port, etc.). This command enters a subsystem that allows modification of the breakpoint's properties; these are not yet fully documented. *[Debug builds only]*

enable breakpoints

Enable breakpoint processing. *[Debug builds only]*

enable console logging

Enable debug logging to the console. See also: the *Debugging Features* section for a complete breakdown of available logging categories and severities.

enable file logging

Enable saving debug logging messages to files in the `Output` directory. Currently up to 100 files of up to 10MB each are saved; the number, naming, destination directory and size of the log files will be made configurable in a future release. *[Debug builds only]*

find memory *address value*

Find a specific *value* in the PERQ's memory starting at *address*. *[Debug builds only]*

inst

Run one Q-code, then pause execution. This command executes microinstructions until the PERQ executes a `NextOp` (AMux select) or `NextInst` (jump instruction); it may execute one or more Z80 instructions as well. See also: Appendix B, *Bibliography* for links to documentation about the PERQ microengine.

jump *address*

Start or resume execution at the given *address* in the control store.

load microcode *binfile*

Load a microcode binary into the control store. You may want to manually `stop` the emulator before loading new microcode! (*PERQemu* should, but doesn't currently, prevent you from doing this.)

load qcodes *codeset*

Load Q-code definitions for opcode disassembly. Choose *codeset* from the given list of known versions. See also: the chapter *PERQ Operations* for more information about which operating systems use which Q-code versions.

Note: the loaded Q-code set is *only* used for accurately disassembling instructions for output and debugging, and in no way affects the execution of the emulated PERQ.

raise interrupt *irq*

Assert a specific CPU interrupt *irq* from the given list of interrupt types. Note that this is highly operating system specific, and generally harmless, but the boot process will likely crash if you inject stray interrupts prior to OS establishment.

reset breakpoints *type*

Reset breakpoint counters for all breakpoints of the given *type*. *[Debug builds only]*

set breakpoint *type watchpoint* [*pause*]

Set a breakpoint of a particular *type*. The value for *watchpoint* is specific to what you wish to trigger; a memory address, I/O port, interrupt type, etc. If true the *pause* flag stops emulation when the breakpoint is reached, otherwise it is noted/counted and execution continues. *[Debug builds only]*

set console loglevel *severity*

Set the threshold for console logging to the *severity* level from the given list. In Release builds, only messages from the selected categories at or above the severity level are printed; in Debug builds, all `Warning`, `Error` and `Heresy`-level messages regardless of category are included automatically.

set execution mode (asynchronous | synchronous)

Set the execution mode for the virtual PERQ. The default is "`asynchronous`," meaning the main CPU and Z80 run on separate threads; "`synchronous`" mode runs both processors on the same thread. *[This command may be removed in a future release.]*

set file loglevel *severity*

Set *severity* threshold for file logging. *[Debug builds only]*
See also: the "`set console loglevel`" command above.

set logging *category*

Enable debug logging for a specific *category* of events, from the provided list. Currently just one set of categories is used for both console and file logging; this may be made specific to each in a future release.
See also: the *Debugging Features* section for more information about logging.

set memory *address value*

Write a specific 16-bit *value* into the PERQ's memory at *address*. *[Debug builds only]*

set xy register *reg value*

Change the value of XY register *reg* to *value*. *[Debug builds only]*

show

Print a summary of current debugger settings.

show breakpoints [*type*]

Print the status of defined breakpoints of the given *type*. *[Debug builds only]*
With no arguments, *type* defaults to "`all`".

show cpu registers

Display the values of the basic CPU registers.

show cstack

Display the contents of the Am2910 call stack. For the 16K CPU, the extended stack is also shown.

show estack

Display the contents of the 16-level hardware expression stack.

show execution mode

Show the execution mode for the virtual PERQ. See also: "`set execution mode`" above.

show memory *address* [*length*]

Dump the PERQ's memory starting at *address*. If not given, *length* defaults to 64 words (128 bytes).

show memory state

Dump detailed information about the memory controller's state. *[Debug builds only]*

show opfile

Display the contents of the opcode cache and current Byte Program Counter (BPC).

show variables

Show debugger variables and their descriptions. The contents of the variables can be printed using the "*:var*" syntax as described in the section *Debugging Features*.

show xy register [*reg*]

Display the value of the XY register *reg*. If no argument is given, dump all 256 registers.

step

Run the next microinstruction, then pause execution. This single steps the main processor by one microcycle; the Z80 may or may not execute one opcode.

z80 [...]

Enter the Z80 debugging subsystem.

Z80 Debugging Commands

The "`debug z80`" prefix enters the Z80 debugging subsystem. Currently the most useful feature is a source-level disassembly of the instruction stream (when single stepping); the CLI doesn't offer much more, yet. The available commands are:

clear breakpoint *type watch*

Clears a Z80 breakpoint previously set with `set breakpoint` (see below). *[Debug builds only]*

dump *item*

Show internal state of a number of Z80 internal devices, states, queues, or other developer-specific information. Some of these commands *may* be available in Release builds, but all of them are subject to change or removal and should be considered experimental. *[Debug builds mostly]*

edit breakpoint *type watch*

Edits a Z80 breakpoint to set or change flags. Operates like the PERQ debugger's breakpoint editor (with slight differences); not all *type*s are implemented. *[Debug builds only]*

inst

Run one Z80 opcode, then pause execution. The PERQ will execute an equivalent number of microinstructions so that the two processors remain in sync (given the disparity of execution rates).

reset breakpoints *type*

Reset Z80 breakpoint counters for all breakpoints of the given *type*. *[Debug builds only]*

set breakpoint *type watch* [*pause*]

Set a breakpoint in the Z80's execution. This operates much like the PERQ's debugger, but the categories are mapped to the Z80 context. This feature is incomplete and not well documented; support for all breakpoint types is added as needed to debug specific instances during emulator development. *[Debug builds only]*

show breakpoints [*type*]

Print the status of Z80 breakpoints of the given *type*. *[Debug builds only]*
With no arguments, *type* defaults to "`all`".

show memory *addr*ess [*length*]

Display contents of Z80 memory starting at *address*. If not given, *length* defaults to 64 bytes.

show registers

Display contents of the Z80 registers.

top

Return directly to the top level menu. (Use "`done`" to return one level to the `debug` menu.)

Appendix A: A Brief History of PERQ

First publicly demonstrated in 1979, the PERQ's design was heavily influenced by the legendary Xerox Alto and the family of "D-machines" from Xerox PARC in the 1970s. Sometimes referred to as the "Pascalto," the machine's most distinguishing feature is its high-resolution, non-interlaced bitmapped display, supported by high memory bandwidth and a custom microcoded processor. Although early production problems delayed volume shipments until 1981, PERQ was arguably the first *commercially available* workstation-class computer to meet the "3M" criteria. Its hardware specifications were broadly in line with the requirements of Carnegie-Mellon University's SPICE Project proposal, where it was used as the development platform for the Accent kernel.

One of the strengths of the PERQ is its extremely flexible microarchitecture. A microprogrammer can program the "bare metal," even tailoring the instruction set to whatever environment is desired. The downside to this flexibility, of course, is the difficulty it posed to OEMs who had to write nearly *everything* from scratch, as POS provides little more than a thin DOS-like single user shell. While a number of factors contributed to the PERQ losing its lead in the workstation race of the early 1980s, the lack of a standard software base – specifically Unix during its meteoric rise in popularity – is generally cited as the primary failing. By the time PNX and Accent (with a System V compatibility environment called Qnix) came along, other Unix vendors like Sun Microsystems had surpassed the PERQ solely on the strength of their software support. While a PERQ T2 could hold its own against the competition in raw graphics performance, contemporary Motorola 68k-based workstations were quickly pulling away in overall speed and capacity.

By the time Three Rivers Computer (renamed in 1983 to PERQ Systems Corporation) went bankrupt in 1986, it is estimated that approximately 4,000 PERQs had been built, mostly sold to universities or a few OEMs building turnkey pre-press, page layout or document management systems. Only a handful of systems are known to exist today in museums and private collections.

PERQemu exists to preserve access to some innovative and interesting software from the heyday of modern computing, when technology was advancing rapidly and the industry as a whole was growing by leaps and bounds. Representing one of the first steps onto dry land from the primordial soup of Xerox PARC, the PERQ is a fascinating "missing link" from the world of text-based timesharing to the fully graphics-driven interactive world we take for granted today.

Appendix B: Bibliography

A growing collection of PERQ documentation is available online to help you learn more about the PERQ and the software you can run with *PERQemu*. Links are provided below to get you started; as the Bitsavers collection does not guarantee that their links won't move or change, copies (along with some additional material not yet published on Bitsavers) are provided on Github as well.

POS Documentation

The POS *Software Reference Manual* is **the** primary reference for older versions D and F:

Bitsavers:	PERQ System Software Reference Manual, Feb. 1982
Github:	POS D Collection

A more extensive collection of POS G documents breaks down the original SRM into more manageable chunks:

Bitsavers:	POS G Collection
Github:	POS G Collection

Accent Documentation

Two full sets of Accent documents exist. CMU provided full size manuscripts with whimsical artwork and supplemental information about code that was not included as part of the general releases from Three Rivers/PERQ Systems Corporation. The "official" documentation was done in the small format popular with the WoD[25] approach popular in the '80s and '90s. The content is largely the same, with a few changes between S5 and S6:

Bitsavers:	Accent S5 Programmer's Manuals Collection (CMU)
	Accent S5 User's Manuals Collection (CMU)
	Accent S5 Users Manual (Supplements, 3RCC)
Github:	Accent Collection

[25] *Popularized by the VAX VMS manual set, the* WALL OF DOCUMENTATION *approach comprised sprawling collections of annoying little half-sized binders that needlessly increased page counts, but they looked pretty all lined up on the shelf! Back then we killed trees to print paper documentation that was outdated and ignored, rather than killing salmon or burning fossil fuels to power datacenters full of broken web links and unindexed, un-OCRed scans of PDFs that nobody reads. Documentation, like QA, is a long dead tradition from a bygone era.*

PNX Documentation

There is very little PNX documentation available here in the US, as most PNX installations were supported by ICL in the UK. Some scans of the *ICL Guide to PNX* manual have recently become available, although the PNX version 3.0 floppies are incomplete and the OS itself cannot yet be installed. The documents are linked here in the hope that they will soon be relevant to *PERQemu*.

Bitsavers: *Not yet available*
Github: PNX Collection

Additional Documentation

Several "miscellaneous" or OS-independent documents are gathered to provide additional background material. The *Fault Dictionary* (available in several versions or included in a number of other docs) is especially useful for tracking down problems at boot time.

Bitsavers: POS G Fault Dictionary
Github: POS G Fault Dictionary

Applications

In November, 2024, a massive collection of nearly 1,200 floppy images was contributed to the Bitsavers software archive. These were archived by several PERQ enthusiasts and collectors (including the *PERQemu* authors) and are now available in IMD format for use with the emulator and real PERQ hardware. These have been sorted into broad categories and their contents loosely catalogued. There is a *lot* of really interesting stuff to discover here.

Efforts are ongoing to curate this raw collection to produce tested, complete installation sets for specific operating systems and applications. This additional software will in time be added to the Github PERQmedia repository, along with QIC tape dumps and many more pre-built hard disk images with demos, games, and applications to explore. Watch this space!

Todo: Links to FLEX docs and software (if publicly released)
Links to Lisp (M2) and Qnix docs?

Appendix C: Additional Resources

Additional applications are being developed to supplement the emulator, and some of these are now being made available.

PERQdisk: Filesystem Munger Tool

PERQdisk is another C# application that lets you open and explore PERQ media archives without having to run the emulator. Perhaps its most useful feature is to extract files from the virtual disk or floppy image directly to the host, from individual files to entire partitions at once.

The software requirements for *PERQdisk* are the same as *PERQemu*: any Windows, Mac or Linux environment that supports the .NET Framework v4.8 should run it. The computational and storage requirements are much, *much* lower though.

> Github: PERQdisk Repository

Currently in its "almost 1.0" early release, *PERQdisk* can read POS and PNX filesystems from hard disk images or floppies, and it can both read *and write* floppy images using the RT-11 volume format. *PERQdisk* can also view and edit text labels in the archive formats that support them, as well as save disks to different formats. The interface is similar to *PERQemu's* CLI; documentation is included with the package.

Stut: The Streamer Utility

Stut is a utility that reads PERQ tape images. It is currently being updated to use the *PERQmedia* library so that it can load 20MB QIC tape images saved in "`.tap`" or "`.prqm`" format. A tool to convert raw dumps of extracted tapes into these archive formats should be integrated, along with label editing and format transcribing capabilities. *Stut* is not yet available on Github.

Todo: Links to "tumble" for turning Canon TIFFs → PDF
Add links/basic usage for interfacing a Gotek floppy emulator to a PERQ
Add links/basic usage for using the Gesswein MFM emulator with a PERQ

Appendix D: Keyboard Maps

PERQemu v0.8.5 introduced the ability to define, save and load custom keyboard mappings. Because the current implementation of the user interface is so heavily keyboard driven, information about how to customize the emulator for your particular host hardware and OS environment is provided in several parts of this document:

- In chapter two, *The Emulator*, information about assigning custom key maps is given in the sections *The Configurator* and *Setting User Preferences*;
- The section *Special Keys* in chapter three, *PERQ Operations,* describes the default mapping of PERQ special keys to a typical PC-101 type keyboard;
- Finally, a summary of the Keymap subsystem commands is given in chapter four.

This appendix provides more detail about the user interface and gives specifics about how custom keyboard mappings can be defined and applied to the virtual machine.

Console vs. Display Input

The *PERQemu* command line interface uses the C# standard `Console` class. All input typed in the console window uses the key definitions that are built into the runtime environment, subject to the restrictions of your particular host OS. Windows, MacOS X and Linux may intercept certain function keys or other special keys for their own uses; these may change based on what particular terminal emulator you use, what version of the OS or window manager is running, or other factors. For this reason, the command-line interface is rather minimal, and tries not to rely on special keys beyond the few used for line editing and tab completion. Currently *PERQemu* provides a limited number of special translations for Turkish input, and any other "internationalization" efforts to deal with non-US English text input has to be made in the source directly.

The Display window that appears when the PERQ is started up is managed by the SDL2 library. This means that an entirely different input model is used to send keystrokes to the virtual machine, based on the standard keyboard definitions published by the USB consortium for a typical PC-101 layout. Keys pressed when the Display has focus are mapped from SDL2's codes to those encoded in PROMs built into the PERQ hardware. This allows the virtual machine to receive and process keystrokes using the same Z80 and system microcode as the real one. It's this mapping that can now be edited to better suit your host environment: the PERQ's special keys which don't have modern equivalents can be reassigned to whatever layout is most comfortable for you.

PERQemu and SDL2 Keycodes

The following tables enumerate the PERQ key caps and the default assignment of SDL2 keyboard codes. These are based on US English and the QWERTY layout used by the PERQ hardware, with the names of the SDL2 keycodes used for mapping.[26]

PERQ Key	Layout		SDL2 Keycode	Description	
	P1	P2		Unshifted	Shifted
Standard alphanumerics:					
A .. Z	✓	✓	SDLK_a .. SDLK_z	a .. z	A .. Z
Num1	✓	✓	SDLK_1	1	!
Num2	✓	✓	SDLK_2	2	@
Num3	✓	✓	SDLK_3	3	#
Num4	✓	✓	SDLK_4	4	$
Num5	✓	✓	SDLK_5	5	%
Num6	✓	✓	SDLK_6	6	^
Num7	✓	✓	SDLK_7	7	&
Num8	✓	✓	SDLK_8	8	*
Num9	✓	✓	SDLK_9	9	(
Num0	✓	✓	SDLK_0	0	)
Standard punctuation and symbols:					
Minus	✓	✓	SDLK_MINUS	-	_
Equals	✓	✓	SDLK_EQUALS	=	+
LeftBracket	✓	✓	SDLK_LEFTBRACKET	[	{
RightBracket	✓	✓	SDLK_RIGHTBRACKET	]	}
Quote	✓	✓	SDLK_QUOTE	'	"
BackQuote	✓	✓	SDLK_BACKQUOTE	`	~
Slash	✓	✓	SDLK_SLASH	/	?

[26] *These are* not *the SDL2* scancodes *(which are different, of course).*

PERQ Key	Layout		SDL2 Keycode	Description	
	P1	P2		Unshifted	Shifted
BackSlash	✓	✓	SDLK_BACKSLASH	\	|
Comma	✓	✓	SDLK_COMMA	,	<
Period	✓	✓	SDLK_PERIOD	.	>
SemiColon	✓	✓	SDLK_SEMICOLON	;	:
Space	✓	✓	SDLK_SPACE		*Note 1*
Tab	✓	✓	SDLK_TAB		*Note 1*
Pad0 .. Pad9		✓	SDLK_KP_0 .. SDLK_KP_9	0 .. 9	0 .. 9
PadPeriod		✓	SDLK_KP_PERIOD	.	.
PadComma		✓	SDLK_KP_COMMA	,	,
PadMinus		✓	SDLK_KP_MINUS	-	-
PadPlus		✓	SDLK_KP_PLUS	+	*Note 2*
PadMultiply		✓	SDLK_KP_MULTIPLY	*	*Note 2*
PadDivide		✓	SDLK_KP_DIVIDE	/	*Note 2*
PadEquals		✓	SDLK_KP_EQUALS	=	*Note 2*

Table 8: Printing characters.

Note 1: POS does not use the TAB character, while other PERQ operating systems may. A text editor or the POS `EXPANDTABS` utility can convert them when files are copied from a different environment.

Note 2: The PERQ-2 keypad doesn't have these keys, but for convenience they are defined so that a modern keypad can send an appropriate keystroke. For instance, the FLEX OS assigns navigation or other commands to the keypad, so it can be useful to map a different host key to the emulator's `PadComma`, as a comma is uncommon on modern US-layout keypads. The PERQ's keyboard layouts are shown at the end of this appendix.

PERQ Key	Layout		SDL2 Keycode	Description
	P1	P2		
Standard editing keys:				
Ins	✓	✓	SDLK_INSERT, SDLK_ESCAPE	AccEsc *on PERQ-2*
Del	✓	✓	SDLK_DELETE	RejDel *on PERQ-2*
BackSpace	✓	✓	SDLK_BACKSPACE	*Note 3*
Return	✓	✓	SDLK_RETURN	*Note 4*
Enter		✓	SDLK_KP_ENTER	*Note 4*
LineFeed	✓		SDLK_PAGEDOWN, SDLK_F7	LF *on the PERQ-1*
LineFeed		✓	SDLK_PAGEDOWN, SDLK_F7	*Note 4*
Up		✓	SDLK_UP	
Down		✓	SDLK_DOWN	
Left		✓	SDLK_LEFT	
Right		✓	SDLK_RIGHT	
Special function keys:				
Help	✓	✓	SDLK_HELP, SDLK_F5	*Note 5*
Oops	✓	✓	SDLK_CLEAR, SDLK_F6	*Note 6*
Setup		✓	SDLK_PAGEUP	
Break		✓	SDLK_SYSREQ	*Note 7*
NoScroll		✓	SDLK_SCROLLLOCK	
PF1 .. PF4		✓	SDLK_F1 .. SDLK_F4	

Table 9: Editing and special keys.

Note 3: On Apple extended keyboards there are often two keycaps labeled "***delete***." The wider one above the RETURN key sends BACKSPACE to the PERQ; the smaller one on the center column is mapped to the DEL or REJ/DEL key on the PERQ.

Note 4: POS (and MPOS) text files have CRLF line endings (like Windows), while PNX and Accent use just LF (Unix newline) style.

Note 5: Many modern keyboards don't have a HELP key, and for Apple extended keyboards that do SDL2 usually sends `SDLK_INSERT` instead. Sigh.

Note 6: Shouldn't *every* keyboard have a key labeled OOPS? Attempts to map the keypad key "*clear*" on Apple extended keyboards are futile, as SDL2 treats that as NUM LOCK, which *PERQemu* tracks as a modifier key. But it doesn't modify anything yet. Oops.

Note 7: The PROM in the PERQ-2 keyboard has a bug in the encoding where the arrow keys overlap with BREAK – rendering it useless? Most PERQ software doesn't seem to use it, and as there's no PERQ-1 equivalent it's generally ignored.

Reserved Keys

The following table lists the codes that are intercepted by *PERQemu* itself, which aren't available to be remapped. The PERQ hardware sees the keyboard as a byte-parallel (PERQ-1) or serial (PERQ-2) device and cannot distinguish shift or control key up/down events anyway, so the emulator tracks these and applies the correct modifier bits before sending the encoded byte to the virtual machine.

Reserved SDL2 Keycodes	**Emulator function**
`SDLK_LALT, SDLK_RALT, SDLK_MENU`	*Mouse off tablet mode*
`SDLK_LCTRL, SDLK_RCTRL`	*Control key modifier*
`SDLK_LSHIFT, SDLK_RSHIFT`	*Shift key modifier*
`SDLK_HOME, SDLK_END`	*Scroll display window*
`SDLK_CAPSLOCK, SDLK_NUMLOCKCLEAR`	*Caps and Numpad lock state*
`SDLK_PAUSE, SDLK_F8`	*Pause emulation*
`SDLK_PRINTSCREEN`	*Reserved*

Table 10: Reserved keys.

The `SDLK_PRINTSCREEN` code (often labeled PrintScreen/SysReq on a PC keyboard) is intended to allow a screenshot to be taken without requiring typing `save screenshot` in the console window. This feature is not yet implemented. A future release of *PERQemu* may provide additional "pseudo keys" so that other emulator functions can be enhanced, such as keystrokes to eject the PERQ's floppy, invoke debugger commands, or control audio volume and screen brightness. *Watch this space!*

Unassigned SDL2 Keycodes		
Shifted keys:		
SDLK_AMPERSAND SDLK_ASTERISK SDLK_AT SDLK_CARET SDLK_COLON SDLK_DOLLAR	SDLK_EXCLAIM SDLK_GREATER SDLK_HASH SDLK_LEFTPAREN SDLK_LESS SDLK_PERCENT	SDLK_PLUS SDLK_QUOTEDBL SDLK_QUESTION SDLK_RIGHTPAREN SDLK_UNDERSCORE
Miscellaneous keys:		
SDLK_AC_HOME SDLK_AC_BACK SDLK_AC_BOOKMARKS SDLK_AC_FORWARD SDLK_AC_REFRESH SDLK_AC_SEARCH SDLK_AC_STOP SDLK_AGAIN SDLK_ALTERASE SDLK_APP1 SDLK_APP2 SDLK_APPLICATION SDLK_CANCEL SDLK_CALCULATOR SDLK_CLEARAGAIN	SDLK_COMPUTER SDLK_COPY SDLK_CRSEL SDLK_CURRENCYSUBUNIT SDLK_CURRENCYUNIT SDLK_CUT SDLK_DECIMALSEPARATOR SDLK_DISPLAYSWITCH SDLK_EXECUTE SDLK_EXSEL SDLK_F9..SDLK_F24 SDLK_FIND SDLK_LGUI SDLK_MAIL SDLK_MEDIASELECT	SDLK_MODE SDLK_OPER SDLK_OUT SDLK_PASTE SDLK_POWER SDLK_PRIOR SDLK_RETURN2 SDLK_RGUI SDLK_SELECT SDLK_SEPARATOR SDLK_SLEEP SDLK_STOP SDLK_THOUSANDSSEPARATOR SDLK_UNDO SDLK_WWW
Multimedia keys:		
SDLK_AUDIOFASTFORWARD SDLK_AUDIOMUTE SDLK_AUDIONEXT SDLK_AUDIOPLAY SDLK_AUDIOPREV SDLK_AUDIOREWIND	SDLK_AUDIOSTOP SDLK_BRIGHTNESSDOWN SDLK_BRIGHTNESSUP SDLK_EJECT SDLK_MUTE	SDLK_KBDILLUMDOWN SDLK_KBDILLUMTOGGLE SDLK_KBDILLUMUP SDLK_VOLUMEDOWN SDLK_VOLUMEUP
Custom keypad keys:		
SDLK_KP_00 SDLK_KP_000 SDLK_KP_A..SDLK_KP_F SDLK_KP_AMPERSAND SDLK_KP_AT SDLK_KP_BACKSPACE SDLK_KP_BINARY SDLK_KP_CLEAR SDLK_KP_CLEARENTRY SDLK_KP_COLON SDLK_KP_DBLAMPERSAND SDLK_KP_DBLVERTICALBAR SDLK_KP_DECIMAL	SDLK_KP_EQUALSAS400 SDLK_KP_EXCLAM SDLK_KP_GREATER SDLK_KP_HASH SDLK_KP_HEXADECIMAL SDLK_KP_LEFTBRACE SDLK_KP_LEFTPAREN SDLK_KP_LESS SDLK_KP_MEMADD SDLK_KP_MEMCLEAR SDLK_KP_MEMDIVIDE SDLK_KP_MEMMULTIPLY SDLK_KP_MEMRECALL	SDLK_KP_MEMSTORE SDLK_KP_MEMSUBTRACT SDLK_KP_OCTAL SDLK_KP_PERCENT SDLK_KP_PLUSMINUS SDLK_KP_POWER SDLK_KP_RIGHTBRACE SDLK_KP_RIGHTPAREN SDLK_KP_SPACE SDLK_KP_TAB SDLK_KP_VERTICALBAR SDLK_KP_XOR

Table 11: Unmapped keys.

Note that the codes for the shifted numeric and punctuation keys don't seem to be generated by SDL2 for the standard US/English keyboard.

For completeness, the emulator key `None` is mapped to `SDLK_UNKNOWN`; these are never passed to the virtual machine. Mapping a host key to `None` is the same as unmapping it; mapping `SDLK_UNKNOWN` to a PERQ key removes *all* the mappings to it!

Using the Keymap Editor

PERQemu began life as a "Console app" with a strictly command-line driven interface, and this has been reasonably quick and efficient to learn and use. However, there are two areas that would benefit greatly from a rewrite as a fully *GUIfied* application: the Debuggers, and now the Keymapper. Using the CLI to painstakingly remap keys using their textual description is admittedly tedious, but the status quo (requiring one to edit and rebuild the source) was no longer tolerable. Something had to be done.

An obvious shortcut would have been to externalize the default keymaps to make them customizable outside of the emulator using a text editor. But *no!* I would not subject you, my beloved fellow PERQ users, to such indignities. Instead, enter the warped realm of my disordered mind to see how two thousand lines of additional code turn what could have been a quick hack into a masterpiece of boondogglery, truly befitting a feature that will likely be used once and forgotten.

Editing Keymaps for Fun and Profit

The new Keymap subsystem works much like the Configurator (or the still inadequately documented Storage subsystem). Key maps are stored as plain text files with the extension "`.kbd`" and are gathered up from the `Conf` directory when *PERQemu* is launched. As is common to the Settings, the device database, Configurations, or other command files and debug scripts, the "format" of the keymap file is just a series of commands as typed into the CLI. You must admit there's a certain indolent elegance about this, sprinkled with a *soupçon of stubbornness* to resist using JSON or XML or some other common file format. This provides a measure of grace to those who find the process cumbersome: simply create and save a new map, then edit the resulting file directly! *Violà!* Everyone is happy.

Because the PERQ-2 keyboard is a superset of the original, *PERQemu* lets you define a map that encompasses both types, then applies only the relevant changes at load time depending on the type of machine that's running. The PERQ-1 won't receive or respond to the keys that are only present on the PERQ-2 keyboard.

To create a new, default keymap, enter the Keymapper:

```
> keymap
```

Next, use the "`define`" command to create a new map with a type and name:

```
keymap> define MyKeymap          ← creates a new map from the default
keymap editor>                   ← enters the editor
```

As with starting a new Configuration, you're strongly advised to choose a brief, fairly short name for the map and don't include a path or extension – let *PERQemu* save it in the default location. This way your custom configurations will be loaded and tracked automatically. (You can also include a `description` which will show up in the `list` command output to remind you of what the new map is for.) Once you're in the editor mode, the usual `commands` help is available too, or refer to the *Command Reference* in chapter four.

The "`map`" or "`unmap`" commands let you make changes based on the keycode names:

```
keymap editor> map sdlk_f1 oops  ← maps your F1 key to the PERQ's OOPS key
keymap editor*> unmap sdlk_f6    ← don't send anything if F6 is pressed
```

Note that the Keymapper changes the prompt to show a '`*`' if unsaved modifications have been made.

After you've made any desired changes (or if you just want to save a new copy of the default keymap) use the "`done`" command to leave the editor. Be sure to use the "`save`" command if you want to preserve your new or updated keymap:

```
keymap editor*> done
keymap*> save
Keyboard map saved to Conf/mykeymap.kbd.
keymap>
```

Note that for convenience the Keymapper lets you issue the `save` command while in edit mode as well. It preserves your changes in memory, so you can return to editing using the "`edit`" command. Unsaved changes are *lost*, however, if you `load` a different keymap use `define` to start a new definition. Of course, *PERQemu* will remind you if you leave the subsystem with unsaved changes.

Using an External Editor

Rather than just store the changed key assignments, *PERQemu* always writes out the full map when saving. Thus, you're free to edit `Conf/`*`yourmap`*`.kbd` with the external editor of your choosing, simply replacing the key codes with their text descriptions from the tables above. The emulator will ignore or reject any typos or unknown keycodes when reading in the files, to prevent the emulator from failing to start up due to a simple keymap error.

Other Features, Quirks and Notes

If you enter the keymap editor while the virtual machine is powered on, *PERQemu* will show you the SDL2 keycode for any keys pressed when the Display window is selected. In this way you can see what code is sent for your particular host platform, though the clicking back and forth between windows can be clunky. The CLI does tab completion for any editor command that accepts an SDL2 code as a parameter, but the list is quite unwieldy – it helps to make the console window big enough to see them all. Or you can just type in the full name from the tables above for the key you wish to change.

The "`apply`" command lets you instantly update the keymap of a running virtual machine. This gives immediate feedback so you can see if the new arrangement is suitable. To get *Inception*-level meta, you can boot a POS disk that has the `KEYTEST` diagnostic utility in `sys:boot>diags`. This graphical tool is used to show what keyboard input the PERQ is receiving at the application level, so if you open the keymap editor while `KEYTEST` is running the result is far out, man.[27] Screen captures shown in the figures below show how the keyboards are arranged.

So why did I do all of this? Insanity, mostly, but also because I am a throwback to an age where Unix networks shared home directories via NFS and I haven't given up my Netapp fileservers yet. And because I try to test *PERQemu* on at least a handful of platforms, including a MacBookPro that doesn't have an extended keyboard. I also toyed with per-*host* Settings files or automatic detection of keymaps based on hostname, but this was already so far off the deep end I scaled it back (if you can believe that). For those reasons *and* one very specific need – accommodating the FLEX OS user interface that assigns specific UI functions to the PERQ 2's keypad by location/keycode rather than ASCII equivalent – this sledgehammer-on-gnat solution will hopefully suffice.

[27] *Well, it would be if I implemented the SDL2 keyboard editor as a semi-transparent graphical overlay that I'd originally planned!*

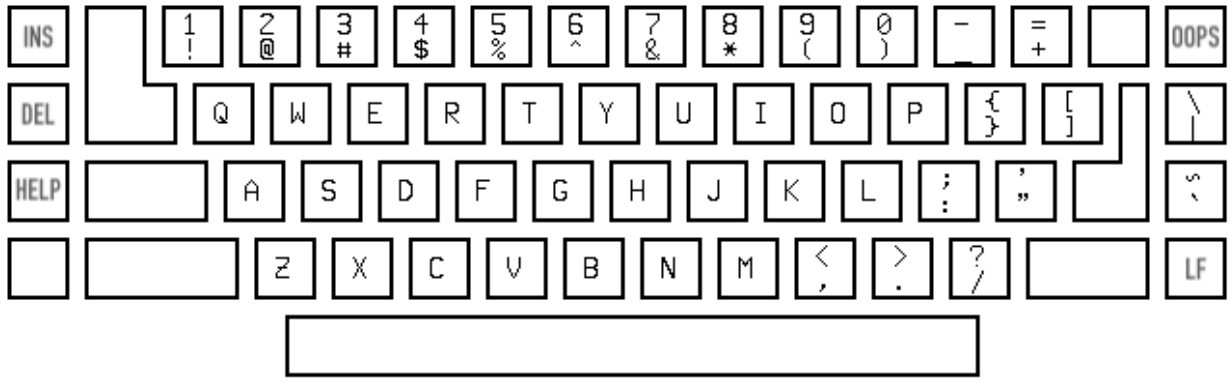

Figure 6: Screen capture of the PERQ-1 keyboard layout.

The earlier version of the `KeyTest` program doesn't label the special keys and inverts the labels of the numeric keys, for some reason. (Special key labels added here for clarity.)

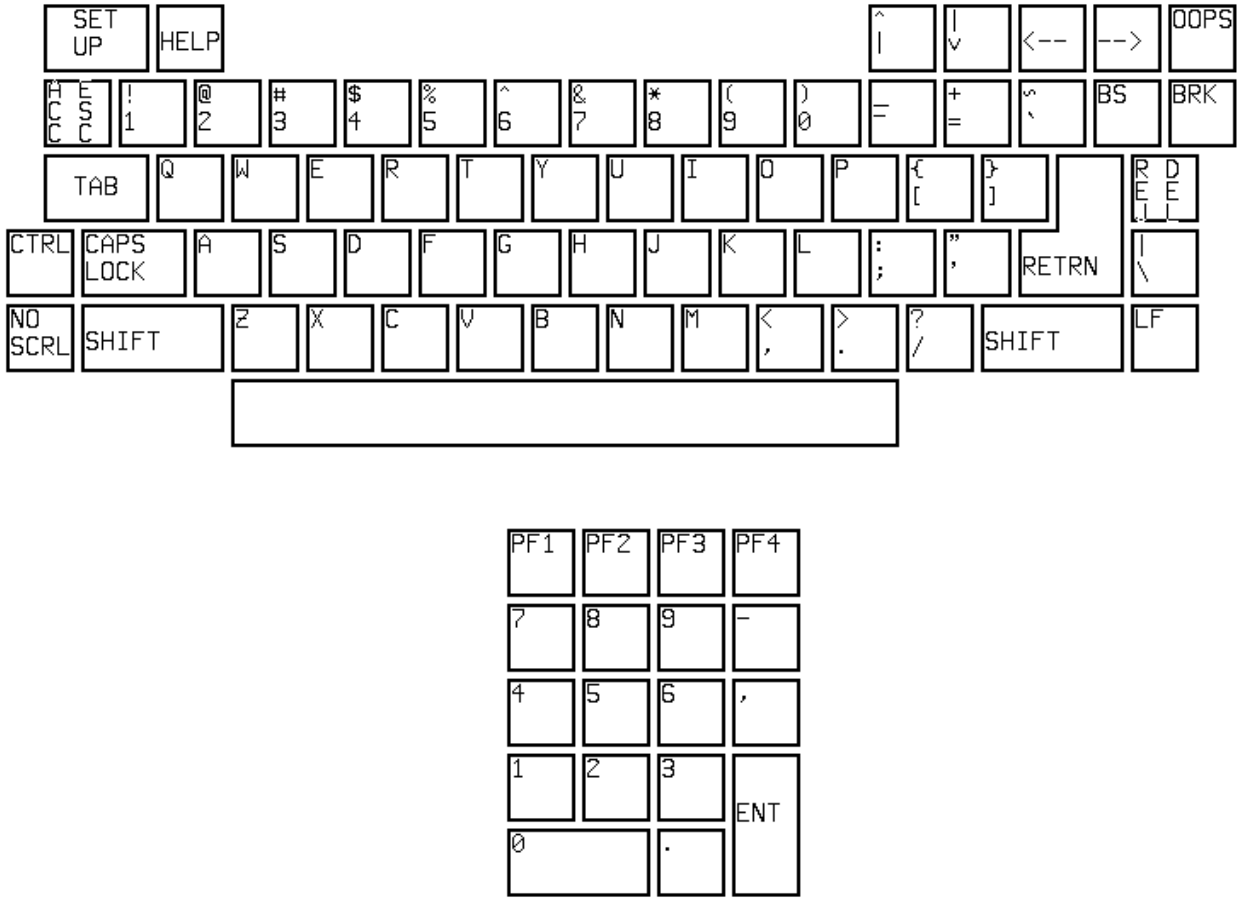

Figure 7: Screen capture of the PERQ-2 VT-100 style layout.

Appendix E: Release History

Summaries of changes in prior releases on the Boondoggle branch, gathered here for posterity.

What's New in *PERQemu* v0.8.5

This release incorporates all of the fixes and features since v0.7.8, plus:

- The display now resizes itself when dragged between screens, with support for panning using the mouse wheel or HOME/END keys when the window is less than full height;
- Support for re-mapping the PERQ's keyboards to accommodate different host keyboard layouts or user preference, plus updated defaults for HELP, OOPS key assignments;
- Refactoring of the Ethernet controller for improved reliability and future enhancements;
- Updates to the storage subsystem commands, and other small fixes and updates.

What's New in *PERQemu* v0.7.8

The PERQ FLEX OS has been recovered and is running on the emulator! This interim release includes patches and new media conversion commands, along with many *extremely* esoteric low-level fixes that enhance emulator accuracy, but *still* don't solve the PNX 5 problem. Highlights:

- The 24-bit "T4" configuration now boots and runs Accent with 4MB of RAM!
- New floppy format plus patches to the DDS and Z80 DMA controller to support the installation of PERQ FLEX – see chapter three for information on this interesting OS;
- Import from and export to "`.img`" files (raw image captures of disk media) for converting the custom FLEX archive floppy format and for reading/writing MFM hard drive images compatible with the Gesswein MFM emulator;
- New Setting/Configuration commands to set the RTC chip's base year offset;
- Numerous reliability enhancements to the Ethernet controller.

This release allows the new Spice Lisp M3 disk image posted on the *PERQmedia* Github archive to run with the 24-bit PERQ-2/T4 configuration with 4MB of RAM, or with the usual 20-bit/2MB/T2 config.

What's New in *PERQemu* v0.7.5

The focus of this release is on testing, stability, and bundling additional PERQ-2 hard disk images so that those configurations can be used "out of the box." This release wraps up the changes since v0.5.0 and marks the completion of *most* major features of the emulator:

- All of the PERQ-2/EIO changes, Canon laser printer, and other additions since v0.5.0;
    - Several new bundled PERQ-2 hard drive images (PNX 2, POS G.6);
- The *final* RasterOp rewrite[28] to fix an extremely obscure edge case;
- Many small bug fixes, performance and accuracy improvements, plus debugging enhancements.

What's New in *PERQemu* v0.6.5

The v0.6.5 interim release is a fairly large step forward toward completing the hardware support for the full lineup of PERQ-2 configurations. Completing the work begun with v0.5.8, this release offers:

- PERQ-2 emulation:
    - Full support for the MFM 5.25" hard drive controller for PERQ-2/T2 configurations (one or two drives);
    - Low-level formatting support for both EIO DIB types (8" and 5.25");
- User Guide updates and numerous small bug fixes.

As of this writing it appears that .NET Framework 4.8.1 will remain supported by Microsoft for quite some time, so for now the target environment remains unchanged. With the v0.7.x release the baseline for MacOS X compatibility will move up to 10.13 (High Sierra) and Visual Studio 2019, as Mac OS X 10.11 has become untenable as a development platform. For now at least cursory testing on Ubuntu Linux (LTS 20.4) continues using the Mono 6.x runtime.

What's New in *PERQemu* v0.5.8

This interim release provides the first working PERQ-2 emulation. While incomplete and still in active development, snapshots posted to the "experiments" branch offer several new features:

- PERQ-2 emulation:
    - Ethernet I/O board (EIO) emulation now allows the basic PERQ-2 model to be configured and run; so far POS G, Accent S6 and PNX versions 2 and 3 are all confirmed to boot!
    - Micropolis 8" hard drive controller provides PERQ-2 and PERQ-2/T1 configurations with a larger 35MB disk vs. the 24MB Shugart in the PERQ-1.

[28] *This statement will not* ever *come back to haunt me, surely.*

What's New in *PERQemu* v0.5.5

This interim release completes the peripheral support for the PERQ-1 models and refactoring to integrate PERQ-2 support. It incorporates numerous bug fixes and refinements of the user interface. Significant new features include:

- Ethernet support:
  - For PERQ-1 configurations, the I/O Option board (OIO) 10Mbit Ethernet interface is fully emulated but still under active development;
  - Currently requires host `root` / `Administrator` privileges;
- Canon laser printer support:
  - The LBP-10 and CX models can be configured as an I/O Option, with software available in POS and Accent to drive them;
  - Output may be in PNG (single page) or TIFF (multiple pages) format;
  - Hardware allows multiple paper sizes (but software limited to US Letter);
- Screen captures from the CLI are available:
  - Portrait or landscape display saved as 100 dpi, 1bpp, PNG format.

There have been other bug fixes, refinements, and UI changes since v0.5.0 that will be enumerated at the next major release when I have time to write them all up.

What's New in *PERQemu* v0.5.0

Release v0.5.0 marks the "halfway" point in the emulator's development. This release does not offer many new emulation options compared to prior *PERQemu* releases; it still emulates the original PERQ-1 model and standard peripherals. However, a *huge* amount of work has been done "under the hood" to enable the addition of all of the PERQ-2 series options going forward:

- True Z80 emulation, running from actual PERQ Z80 ROM images:
  - The original IOB with v8.7 ROMs ("old Z80") supports the PERQ-1 options that prior versions of *PERQemu* can run;
  - The enhanced CIO board with v10.17 ROMs ("new Z80") now enables newer versions of all of the PERQ-1 operating systems to run;
- Uses the SDL2 library for the display and keyboard/mouse interface:
  - Runs on 64-bit Mono/MacOS and eliminates 32-bit WinForms limitations;
- A new, unified *PERQmedia* storage architecture and file format:
  - Provides a flexible framework for managing *all* existing PERQ floppy, hard disk and tape image formats;

    - Adds a new common "PRQM" format with support for text and graphical labels, data compression, and checksums;
- Dynamic runtime configuration of all PERQ models and features:
    - Quickly load and run from a library of predefined machine configurations, or easily customize your own;
    - New peripheral support: emulated streaming QIC tape interface added;
- Enhanced command line interface:
    - More prompts, in-line help, and interactivity;
    - Basic Unix/Emacs control keys added for editing;
- Expanded debugging facilities:
    - Extensive logging options, to the console and/or log files;
    - Breakpoint support in Debug builds for watching I/O ports, memory locations, micro addresses, and more; breakpoints can trigger the execution of user-supplied scripts to perform more complex actions;
- Persistent user preference settings, and much, much more!

Please refer to the Readme files or Github Releases page for additional pre-v0.5.0 information.